PROJECT GUIDE | PORTFOLIO PROJECT 4

Growth Experimentation Platform

A complete, in-depth guide to what was built:
A/B testing, SQL, causal and uplift modelling,
real randomised data, a REST API and a live interactive app.

Author	Deep Shah
Live app	growth-experimentation-platform.streamlit.app
Code	github.com/DeepShah111/growth-experimentation-platform
Stack	Python, SQL (DuckDB), pandas, SciPy, scikit-learn, Streamlit, FastAPI, Docker, pytest, GitHub Actions
Data	Hillstrom e-mail test and Criteo ads test (real, randomised) plus a simulator with a known truth

Every number in this guide comes from the project's own code, tests or the live app. Where something is an assumption, it says so.

Contents

Part A: The big picture . . . 7

1. What this project is . . . 8

1.1 The idea in one paragraph . . . 8
1.2 The business story we built around it . . . 8
1.3 What an A/B test is, and why people get it wrong . . . 8
1.4 What we built (the components) . . . 9
1.5 Real data and the simulator: why both . . . 10
1.6 Which resume gaps it closes . . . 10

2. The tools and why we chose them . . . 11

3. The data . . . 12

3.1 The simulated warehouse (the test bench) . . . 12
3.2 Hillstrom e-mail test (real, randomised) . . . 13
3.3 Criteo ads uplift test (real, randomised) . . . 14

4. Architecture and project structure . . . 15

4.1 The ten-stage data and model flow . . . 15
4.2 The folder structure . . . 16

Part B: How it works, step by step . . . 17

5. Steps 1 to 3: simulate, load and clean . . . 18

5.1 Step 1: the simulator . . . 18
5.2 Step 2: loading into DuckDB . . . 18
5.3 Step 3: cleaning (the staging view) . . . 18

6. The SQL layer in depth . . . 20

6.1 SQL concepts index (what you can say you used) . . . 20
6.2 The eleven analysis reports . . . 20
6.3 The marts: one row per user . . . 22
6.4 A whole experiment analysis in pure SQL . . . 22
6.5 Two SQL bugs found by reading, and fixed (good interview stories) . . . 23

7. A/B testing and statistics, concept by concept . . . 24

7.1 Randomisation and the three kinds of number . . . 24
7.2 Sample-ratio mismatch (SRM): was the split fair? . . . 24
7.3 Effect size, uncertainty and the p-value . . . 25

7.4 CUPED: less noise from information you already have . . . 25
7.5 Multiple testing: when you run many experiments . . . 26
7.6 Power, minimum detectable effect and sample size . . . 26
7.7 Peeking and sequential testing . . . 27
7.8 Novelty effects: does the lift fade? . . . 27
7.9 Interaction between overlapping experiments . . . 28
7.10 Heavy-tailed revenue . . . 28
7.11 Guardrails and unit economics . . . 28
7.12 The decision rule: six gates in order . . . 29
7.13 Is the platform itself trustworthy? (validation by re-simulation) . . . 30

8. Uplift modelling: who should get the offer? . . . 31

8.1 Why an average effect is not enough . . . 31
8.2 The three learners . . . 31
8.3 Cross-fitting: nobody is scored by a model that saw them . . . 31
8.4 Scoring without knowing anyone's individual effect . . . 32
8.5 From scores to money: the profit curve and an honest cut-off . . . 32
8.6 Stability: repeating with 10 seeds . . . 32
8.7 What exp_008 found . . . 33

9. Real-data work . . . 34

9.1 Criteo: the same checks on a real advertising experiment . . . 34
9.2 Hillstrom: the full engine on a real e-mail test . . . 34

10. The platform layer: engine, API, security and tests . . . 36

10.1 The engine modules . . . 36
10.2 The REST API . . . 36
10.3 Security of the SQL box . . . 37
10.4 Tests: 167 automated checks . . . 37
10.5 Continuous integration, Docker and deployment . . . 37

Part C: The app, page by page . . . 39

11. A tour of the live app . . . 40

11.1 Overview . . . 40
11.2 Analyze an experiment (interactive) . . . 42
11.3 Plan an experiment (interactive) . . . 44
11.4 Peeking simulator (interactive) . . . 47
11.5 Live targeting (interactive) . . . 48
11.6 SQL workbench (interactive) . . . 50
11.7 How it works . . . 53

11.8 SQL layer . . . 54
11.9 Decision scorecard . . . 54
11.10 Trust checks . . . 56
11.11 Statistics lab . . . 57
11.12 Who gets the offer? (exp_008) . . . 58
11.13 Real data (Criteo) . . . 60
11.14 Platform validation . . . 61
11.15 Decision memos . . . 62
11.16 Limits and honesty . . . 66

Part D: What happens when you change a parameter . . . 67

12. The seven levers behind every result . . . 68
12.1 Direction of every control (quick reference) . . . 69

13. What-if experiments (computed numbers) . . . 70
13.1 Plan page: what each lever costs in users and days . . . 70
13.2 Peeking: false wins by the number of looks . . . 71
13.3 Analyze page: economics decide the verdict (Hillstrom, conversion) . . . 71
13.4 Analyze page: the other controls . . . 72
13.5 The effect of sample size: demo warehouse vs full warehouse . . . 72
13.6 Live targeting: how cost moves the best reach (Hillstrom, Men's e-mail, spend, T-learner) . . . 73

Part E: Explaining and defending the project . . . 74

14. Explaining the project . . . 75
14.1 The 60-second version . . . 75
14.2 The five-minute walk-through (a suggested order) . . . 75

15. Resume bullets you can honestly write . . . 76
15.1 Skills lines this project supports . . . 76

16. Interview questions and model answers . . . 77

Part F: Appendices . . . 80

Appendix A. Glossary . . . 81

Appendix B. Formula cheat-sheet . . . 83

Appendix C. Commands . . . 84

Appendix D. Troubleshooting . . . 85

Appendix E. Honest limits and what to do next 86

How to read this guide

This guide explains the **Growth Experimentation Platform** from the ground up: what it is, how it was built, every concept it uses, every page of the live app, and what happens when you change a parameter. It is written so that you can understand the project in detail and explain it to someone else.

If you want to...	Read
Understand the project quickly	Chapter 1 (what it is), chapter 4 (architecture), chapter 14 (how to explain it)
Understand the A/B testing and statistics	Chapter 7, then chapter 12 (the seven levers)
Understand the SQL	Chapter 6
Understand the machine learning (uplift)	Chapter 8
Know what every page of the app does and what each control changes	Chapter 11 (page by page), chapter 12 and chapter 13 (computed what-ifs)
Prepare for an interview or write the resume bullets	Chapters 14 to 16
Run, test or deploy it	Appendix C and D

How to read the boxes

Colour guide. Blue boxes explain an idea in plain words. Green boxes are key ideas or decisions. Amber boxes are warnings and assumptions. Purple boxes describe a parameter. Dark boxes are interview tips. Red boxes flag common mistakes. Screenshots come from the live app at growth-experimentation-platform.streamlit.app.

PART A

The big picture

What the project is in plain language, which tools it uses, what data it runs on, and how the pieces fit together.

1. What this project is

1.1 The idea in one paragraph

Companies constantly change their products: a new checkout button, a different homepage, a discount e-mail. Before rolling a change out to everyone, they test it on a random part of their users and compare the result with a group that did not get the change. This is called an **A/B test** (or an *experiment*). The hard part is not running the test; it is **reading the result correctly**. A result can look like a win and be pure luck, a broken test can fake a win, a real win can lose money once costs are counted, and an average win can hide users who are harmed.

The **Growth Experimentation Platform** is a system that takes the data of any such test and returns a trust-checked, plain-English decision: **SHIP**, **ITERATE**, **INCONCLUSIVE** or **DO NOT TRUST**, together with the reasons, the money, and (when it makes sense) *which users* should get the change. It is built from a SQL warehouse, a statistics engine, uplift (machine-learning) models, a REST API and an interactive web app, and it is deployed live.

> **In plain words**
> Think of it as a **referee for experiments**. A product manager hands over the raw numbers; the platform checks that the game was fair (was the split of users really random?), counts the score carefully (lift, uncertainty, noise reduction), checks whether the team could have seen the result at all (power), checks that nothing else got worse (guardrails), and finally asks whether the change pays for itself.

1.2 The business story we built around it

The project imagines a **grocery-delivery app** with 100,000 users. Over 12 weeks the growth team runs **nine experiments** (a redesigned checkout button, a delivery-fee banner, a homepage redesign, a win-back offer, and so on). The team needs three answers, and the whole platform is organised around them:

#	The question	What answers it in the platform
1	**Can we trust the result?**	Sample-ratio check, data-quality monitor, peeking and multiple-testing controls, power analysis, and a validation of the platform's own statistics in 100 re-simulated worlds.
2	**What should ship?**	A decision scorecard: six ordered gates (trust, evidence, power, safety, validity, economics) that end in SHIP / ITERATE / INCONCLUSIVE / DO NOT TRUST / NO EFFECT FOUND / DO NOT SHIP.
3	**Who should get the offer?**	Uplift models (S, T and X learners) that estimate, per user, how much extra the offer causes, plus a profit curve that decides how many users to reach.

1.3 What an A/B test is, and why people get it wrong

In an A/B test, users are assigned **at random** to a **control** group (the old experience) or a **treatment** group (the new one). Because assignment is random, the two groups are alike in everything except the change, so any difference in the outcome (orders per user, conversion, revenue) can be attributed to the change. That is the core of **causal inference**: random assignment lets us measure cause, not just correlation.

Teams still go wrong in predictable ways. Each of these is something the platform detects or prevents:

Mistake	What happens	Where the platform handles it
The split was not really random (a bug)	Any lift is meaningless. In the simulator, exp_006 secretly drops low-activity treated users and fakes a +13% lift.	Sample-ratio mismatch (SRM) test; scorecard says DO NOT TRUST.
Too few users	A real effect exists but the result is 'not significant', and the team wrongly concludes 'no effect'.	Power and minimum detectable effect (MDE); the scorecard says INCONCLUSIVE, not 'no effect'.
Peeking	Checking daily and stopping at the first p below 0.05 turns a 5% false-positive rate into about 20 to 28%.	Peeking simulator and sequential (O'Brien-Fleming) boundaries.
Many experiments at once	Run 20 harmless tests and about 6 in 10 times one looks like a winner by chance.	Holm correction across experiments.
Noisy metrics	Heavy-tailed revenue hides real effects.	CUPED noise reduction; winsorising and bootstrap estimators.
A win that loses money	A real +7% lift still costs more to deliver than it earns.	Unit-economics gate; profit curve; uplift targeting.
An average hides harm	The offer helps some users and hurts others; the average looks fine.	Uplift deciles and models.
Novelty effect	A new design looks great in week 1 and fades.	Weekly lift and trend test.

1.4 What we built (the components)

- **A simulator** of a grocery app: 100,000 users, 84 days, nine experiments with *hidden, known* true effects and planted problems (duplicate events, late events, a broken assignment).
- **A SQL warehouse (DuckDB)**: raw tables, a cleaning layer, one-row-per-user marts, eleven analysis reports, and a complete experiment analysis written in pure SQL.
- **A statistics engine** (Python): sample-ratio test, CUPED, Holm, power, peeking, novelty, interaction, heavy-tail revenue, guardrails, economics, and the decision rule.
- **Uplift models**: S, T and X learners with cross-fitting, honest evaluation, profit curves and a hold-out cut-off.
- **Real-data checks**: the Criteo ads experiment (balance, naive vs adjusted vs doubly-robust effects, uplift) and the Hillstrom e-mail experiment (the full scorecard on real data).
- **A generic engine** (`src/engine/`): the same logic as pure functions that work on *any* experiment table, including a CSV you upload.
- **A REST API** (FastAPI + Docker) that exposes the engine, and **a live interactive web app** (Streamlit, 16 pages).
- **Quality controls**: 167 automated tests, linting, continuous integration on GitHub Actions, and two written decision memos.

> **Where it lives**
> Live app: **growth-experimentation-platform.streamlit.app**. Source: **github.com/DeepShah111/growth-experimentation-platform**.

1.5 Real data and the simulator: why both

The simulator is **not** presented as evidence about the real world. It is a **test bench**: because we wrote the true effect of every experiment, we can check whether the platform finds it. A statistics tool that has never been graded against a known answer cannot claim to be correct. The platform is then run on **real randomised data** (Hillstrom and Criteo), where the true answer is unknown but the same checks apply.

> **Interview tip**
> The honest framing for interviews: "*The simulator proves the methods work when the truth is known. The real datasets show they run on real, messy data. I never claim the simulated numbers describe a real business.*"

1.6 Which resume gaps it closes

Gap on the resume	How this project closes it	What an interviewer can ask you
SQL was listed as a skill but not used in any project	Staging view, marts, 11 reports and a full experiment analysis in SQL, each checked by a unit test against a hand-computed answer; a live SQL workbench.	Window functions, gaps and islands, cohort retention, sessionisation, QUALIFY, handling ties, EXPLAIN.
A/B testing appeared only as the word "Statistics"	SRM, CUPED, Holm, power, peeking, novelty, interaction, guardrails, economics, a decision rule, a validation of all of it.	How do you size a test? What is SRM? What is CUPED? Why does peeking inflate errors?
Causal inference / uplift was missing	Randomised-experiment effects, doubly-robust (AIPW) estimation, S/T/X learners, cross-fitting, Qini, honest profit.	Why uplift and not churn prediction? How do you evaluate an uplift model?
Business impact framing	Unit economics, break-even arithmetic, two decision memos, the 'is it worth shipping' bar.	Walk me through a decision you would make from these numbers.
Deployment / engineering	FastAPI, Docker, CI, 167 tests, a live Streamlit app, a read-only SQL box with a security guard.	How would you productionise this? How do you test statistics code?

2. The tools and why we chose them

Everything is free and open source, and runs on a laptop. The table lists each tool, what it does in *this* project, and the reason it was chosen.

Tool	Used for	Why this one
Python 3.11/3.12	All logic: simulation, statistics, models, API, app	The standard language of data science; the whole stack is available.
pandas, NumPy	Tables and numerical arrays	Fast, familiar, and what every DS interviewer expects.
SciPy	Statistical tests (Welch t-test, chi-square), distributions	Reliable, well-tested implementations.
scikit-learn	Gradient-boosting models inside the uplift learners; cross-validation folds	HistGradientBoosting handles mixed numeric/categorical tabular data well.
DuckDB	The SQL warehouse (runs inside Python, reads Parquet directly)	A real analytical SQL engine with window functions, QUALIFY, FILTER and statistical aggregates, with zero setup.
Parquet / PyArrow	Storing the simulated tables (about 85 MB for 100,000 users)	Compact, columnar, typed; the standard format for analytics data.
Streamlit + Altair	The interactive web app and its charts	Fast to build, deploys free, charts are declarative and testable.
FastAPI + Pydantic + Uvicorn	The REST API with validated inputs and automatic docs at /docs	Modern, typed, and the same stack as the CLV project on the resume.
Docker	Packaging the API as a container	One command runs it anywhere; shows deployment skill.
pytest	167 automated tests	Statistics code must be tested against hand-worked answers.
GitHub Actions	Runs tests, lint and the Docker build on every push	Continuous integration; a green badge builds trust.
ruff	Linting (catches unused code and errors)	Fast, one tool.
Streamlit Community Cloud	Hosting the live app for free	Deploys straight from GitHub on every push.
VS Code, Git, GitHub	Development and version control	Standard professional workflow.

3. The data

3.1 The simulated warehouse (the test bench)

The simulator (`src/simulation/simulate.py`) generates seven tables for a fictional grocery-delivery app. Day 0 is a Monday (5 Jan 2026). Days 0 to 27 are a **pre-period** in which nobody is in an experiment: this gives every user 28 days of behaviour *before* any test, which later powers CUPED and the uplift features.

Table	One row is...	Main columns
users	a user	user_id, signup_date, is_new (joined during the 12 weeks), city, device (android/ios/web), channel (organic/paid/referral)
products	a product (300 of them)	product_id, category
sessions	a visit to the app	session_id, user_id, session_start, duration_sec, device
events	something that happened in a session	event_id, session_id, user_id, event_type (product_view, add_to_cart, checkout_start, purchase), event_ts, product_id, ingested_at
orders	a purchase	order_id, session_id, user_id, order_ts, n_items, gross_value, discount_pct, order_value
experiments	an experiment	experiment_id, name, start_date, end_date, primary_metric, exclusive_group
assignments	a user placed in an experiment	assignment_id, experiment_id, user_id, variant (control/treatment), assigned_at

An eighth and ninth file, the **answer key** (`ground_truth` and `user_truth`), live in a separate database schema called `answer_key`. The analysis never reads them, so it cannot cheat; they are used afterwards only to judge the results. A test even builds the whole scorecard from a database that has no answer-key tables at all.

The planted problems

- **Duplicate events**: the tracking pipeline double-logs about 0.60% of events (identical session, type, timestamp and product).
- **Late events**: about 1.51% of events arrive more than a day after they happened. They are kept but flagged, because a report run yesterday would not have seen them.
- **An assignment bug (exp_006)**: low-activity treatment users are silently dropped from the assignments table, which makes the treatment group smaller and fakes a lift.
- **Overlapping experiments**: exp_001 and exp_004 both claim the homepage surface and share users, so they might interact.

The nine experiments and their hidden truths

ID	Name	Hidden truth	What it tests about the platform
exp_001	checkout_button_v2	Constant +8% on checkout-to-purchase; overlaps exp_004	A clear win: does the platform ship it, and note the overlap?
exp_002	delivery_fee_banner	Constant +1.5% (tiny)	Is the sample big enough to see a tiny effect? Built to be underpowered.
exp_003	recs_carousel	Exactly 0 (a true null)	Should come out 'not significant' about 95% of the time.
exp_004	homepage_redesign	Novelty: +12% at first, fading to about +1% (half-life 6 days); overlaps exp_001	Can the platform notice a fade?
exp_005	onboarding_flow	+18% for users who joined during the window, 0 for others	A segment-only effect that the average can miss.
exp_006	search_ranking_v3	0, but a planted assignment bug fakes a positive lift	Must be flagged DO NOT TRUST.
exp_007	free_delivery_threshold	+6% on basket value (revenue), conversion unchanged	A heavy-tailed revenue metric.
exp_008	winback_offer_profile	Lift depends on channel (organic +4%, paid +30%, referral -10%) plus device (android 0, ios -5%, web +15%)	A costly offer; a learnable uplift test bed.
exp_009	personalised_offer_latent	Lift depends on a hidden engagement segment (casual +30%, medium +10%, loyal -3%)	Deliberately hard: models cannot see the segment.

Assumptions
The business assumptions used for money are **INR 50 contribution per extra order** (about 12% of a roughly INR 400 basket) and **INR 3 cost per user shown the win-back offer**. They are assumptions, not measurements, and every page that uses them says so.

3.2 Hillstrom e-mail test (real, randomised)

Kevin Hillstrom's MineThatData challenge dataset: **64,000 customers** of an online retailer who had bought in the previous 12 months were randomly split into three groups: **Men's e-mail** (21,307), **Women's e-mail** (21,387) and **No e-mail** (21,306). Two weeks later the retailer recorded whether each customer visited the website, converted (bought) and how much they spent.

Column	Meaning
segment	The arm: Mens E-Mail, Womens E-Mail or No E-Mail (the control)
visit, conversion, spend	Outcomes in the 2 weeks after the e-mail: visited the site (0/1), bought (0/1), dollars spent
recency, history, mens, womens, newbie	Pre-experiment facts: months since last purchase, dollars spent in the past year, bought men's / women's products, new customer
zip_code, channel	Urban/Suburban/Rural; Phone/Web/Multichannel

Watch out
Caveats stated in the app: the data is from 2008 and one retailer; spend is zero for about 99% of customers, so it is heavy-tailed and noisy.

3.3 Criteo ads uplift test (real, randomised)

Criteo's public advertising experiment (Uplift v2.1, 13,979,592 rows in the full file): about **85% of users were targeted** with ads and 15% were held out at random. Each row has 12 anonymised features (f0 to f11), the treatment flag, whether the user **visited** the advertiser's site and whether they **converted**. The project uses a random 1,000,000-row sample. Two honest cautions: the features are anonymised, and only about 3.6% of treated users actually saw an ad, so all effects are *per user assigned*, not *per ad seen*. The Criteo pages in the app display results computed earlier on the 1M sample (artifacts); the live Analyze page can also use a 100,000-row sample when that file is added.

4. Architecture and project structure

The platform has four layers. Data goes in on the left; decisions come out on the right.

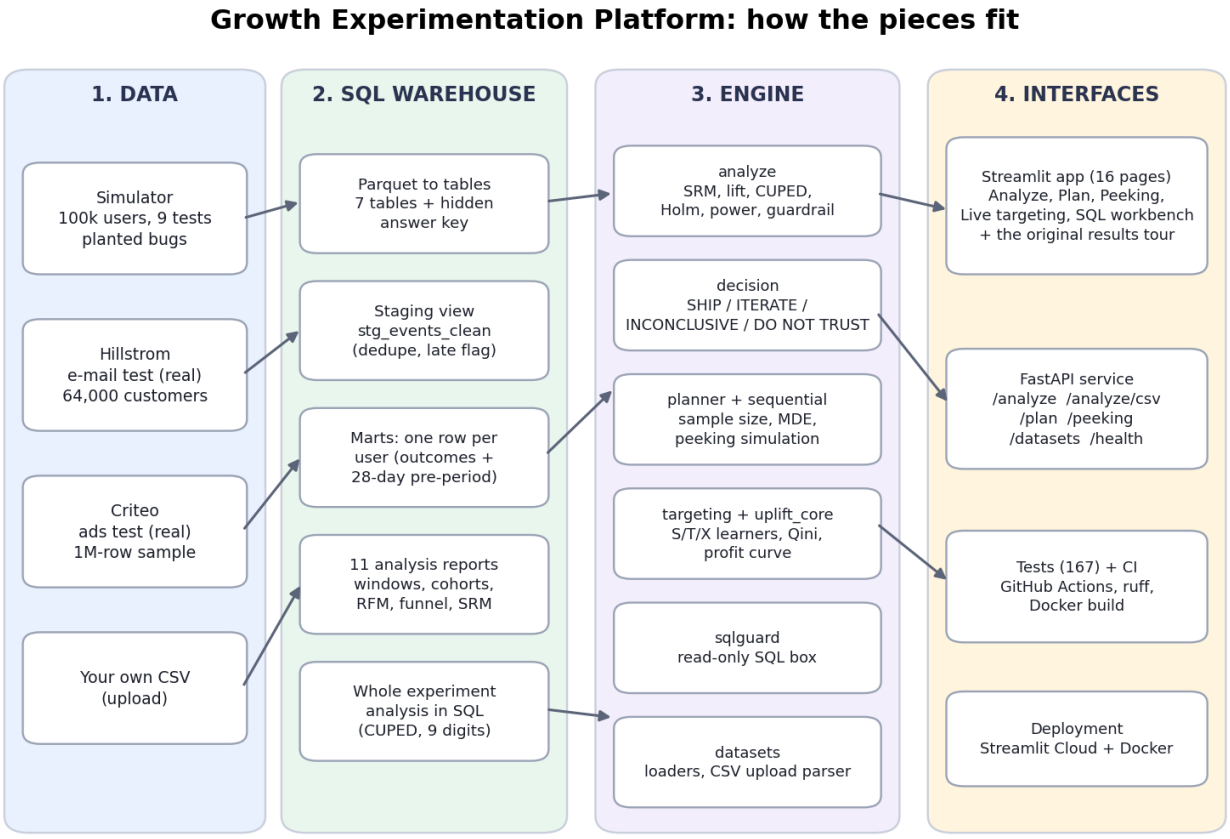


Figure 1. The four layers: data, SQL warehouse, engine, interfaces.

4.1 The ten-stage data and model flow

#	Stage	Goes in, comes out	Where
1	Simulate	Config to 7 Parquet tables (about 85 MB)	src/simulation
2	Load	Parquet to DuckDB tables; hidden answer key in its own schema	sql/schema, src/warehouse/build_warehouse.py
3	Clean	events to stg_events_clean: 5,275,310 raw events become 5,243,847	sql/staging
4	Report	Clean tables to 11 SQL result CSVs	sql/analysis, src/warehouse/run_sql.py
5	Model the data	Clean tables to one row per assigned user: outcomes after assignment, features from the 28 days before	sql/marts
6	Statistics	Marts to a decision per experiment	src/experiments, src/engine
7	Machine learning	Marts to an uplift score per user (S, T, X learners, 4-fold cross-fitting)	src/targeting, src/engine/uplift_core.py
8	Evaluate	Scores to honest numbers: Qini, bootstrap intervals, 10 seeds, hold-out profit	src/targeting
9	Real data	Criteo sample and Hillstrom through the same checks	src/criteo, src/engine
10	Deliver	Artifacts, memos, the API and the Streamlit app	app/, api/, docs/memos

4.2 The folder structure

```
growth-experimentation-platform/
  .github/workflows/ci.yml   tests on Python 3.11 and 3.12, lint, Docker build and smoke test
  Dockerfile, .dockerignore  packages the REST API
  api/main.py                FastAPI service (6 endpoints, validated inputs)
  app/                       Streamlit app: streamlit_app.py (original pages), live_pages.py (5
      interactive pages), charts.py
  artifacts/                 result CSVs and charts that the original pages display
  docs/                      decision memos, deployment guide, diagram sources
  notebooks/                 thin notebooks that call src/
  sample_data/               hillstrom.csv.gz (about 0.4 MB) so the live app has real data
  sql/  schema/ staging/ marts/ analysis/   all SQL, one file per query
  src/
    config.py                sizes, the nine experiments, business assumptions
    simulation/              the simulator
    warehouse/               build the DuckDB database, run all SQL reports
    experiments/             statistics: first_look, novelty, interaction, revenue, studies,
        validation, scorecard, stats_utils
    targeting/               uplift.py, multiseed.py (analysis scripts)
    criteo/                  prepare, validate, diagnostics (real-data checks)
    engine/                  PURE functions used by the API and the app:
                             analyze, planner, sequential, targeting, uplift_core,
                             decision, datasets, sqlguard, make_samples
  tests/                     mirrors src/, plus one test per SQL query, API tests and app tests
```

Design decision

Why an engine layer? The notebooks, the command-line scripts, the API and the app all call the same pure functions in `src/engine/`. That is why the Analyze page and the REST API can never disagree, and why a test can check the SQL result against the Python result to nine digits.

PART B

How it works, step by step

The simulator, the SQL layer, every A/B-testing concept, the uplift models, the real-data work and the engine, API and tests.

5. Steps 1 to 3: simulate, load and clean

5.1 Step 1: the simulator

The simulator builds users first (when they signed up, device, channel, city, an engagement level), then walks through 84 days creating sessions, events and orders, and applies each experiment's hidden effect only to users assigned to it and only inside its date window. Everything is driven by a fixed random seed, so the same command always produces the same warehouse (a test checks this). Planted problems are switched on by `inject_bugs=True`; a test confirms that turning them off removes the sample-ratio bug.

Realism checks are tests too: session-to-order conversion must be between 3% and 8%, iOS must convert better than Android and Android better than web, retention must decay over time, every primary key must be unique, every foreign key must point to a real row, and the number of orders must equal the number of distinct sessions that have a purchase event.

> **Interview tip**
> **Why is this not 'fake data, so worthless'?** Because the point of the simulator is that the *truth is known*. It is the same idea as testing a smoke detector with a controlled fire: you can only grade a detector if you lit the fire yourself.

5.2 Step 2: loading into DuckDB

`sql/schema/00_schema.sql` creates the database objects with `CREATE OR REPLACE TABLE ... AS SELECT * FROM read_parquet(...)`. DuckDB reads Parquet files directly, so loading about 5 million events is fast. The observable tables sit in the default schema; the hidden truth sits in `answer_key`, so a query cannot touch it by accident.

`src/warehouse/build_warehouse.py` runs that file and then applies every file in `sql/staging/`. `src/warehouse/run_sql.py` runs every query in `sql/analysis/`, fills any `$parameters` (for example `$as_of` becomes the day after the last order) and saves each result as a CSV in `artifacts/sql/`.

5.3 Step 3: cleaning (the staging view)

Raw events are not trustworthy: the tracking pipeline double-logs some events and some arrive late. `stg_events_clean` is a **view** (a saved query) that every later query reads instead of the raw table.

```
CREATE OR REPLACE VIEW stg_events_clean AS
SELECT event_id, session_id, user_id, event_type, event_ts, product_id, ingested_at,
       (ingested_at > event_ts + INTERVAL 1 DAY) AS is_late
FROM (
    SELECT e.*,
           ROW_NUMBER() OVER (PARTITION BY session_id, event_type, event_ts, product_id
                              ORDER BY event_id) AS rn
    FROM events e
)
WHERE rn = 1;
```

- **ROW_NUMBER() OVER (PARTITION BY ... ORDER BY event_id)** numbers the rows inside each group of identical events (same session, type, timestamp, product). Keeping `rn = 1` keeps exactly one row per group, the one with the lowest event_id. This is the standard way to de-duplicate in SQL.

- **PARTITION BY** treats NULL values as equal, which matters here because `product_id` is NULL for checkout and purchase events; duplicates of those are still caught.
- **is_late** is a flag, not a filter: late events are kept but marked, because a report run the day after would not have seen them.
- **Result:** 5,275,310 raw events become 5,243,847 distinct events. The difference is 31,463, which is exactly 0.60% of 5,243,847, the amount the simulator injected, so the clean-up removes the planted duplicates and nothing else.

6. The SQL layer in depth

This chapter is the answer to "where is the SQL?". There are **11 analysis reports**, **2 marts** (per-user tables used by the statistics) and **1 complete experiment analysis in SQL**. Each report has a unit test that runs it on a tiny hand-built database and compares the output with an answer computed by hand.

6.1 SQL concepts index (what you can say you used)

Concept	Where it appears	In one line
CTEs (WITH ...)	Every report	Name each step of a calculation so it reads top to bottom.
ROW_NUMBER, RANK, DENSE_RANK	stg_events_clean, activity_streaks, top_products, rfm_segments	Number or rank rows inside groups; they differ in how ties are treated.
LAG and running SUM() OVER	daily_revenue_trends, sessionization_check	Compare a row with the previous one; accumulate totals.
Window frames (ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)	daily_revenue_trends	A 7-day moving average.
QUALIFY	top_products_per_category	Filter on a window function without a wrapper query.
Gaps and islands	activity_streaks	Find runs of consecutive days with (day number minus row number).
Self-joins	overlapping_experiments, experiment_interaction	Join a table to itself to find pairs.
Conditional aggregation (MAX/SUM of a flag, FILTER)	funnel_by_device, srm_counts, experiment_user_metrics	Turn row-level facts into per-group counts and per-user features.
LEFT JOIN + COALESCE	experiment_user_metrics	Keep users with no orders and treat them as zero.
Grid construction with range()	weekly_retention, experiment_user_weekly	Generate every (cohort, week) pair so empty cells show as zero.
Date functions (DATE_TRUNC, DATE_DIFF, INTERVAL)	retention, streaks, marts	Cohort weeks, day gaps, 28-day windows.
NULLIF, CASE, CAST	Many	Safe division, rule-based labels, type conversion.
Statistical aggregates (COVAR_SAMP, VAR_SAMP)	experiment_in_sql	Compute the CUPED coefficient inside the database.
Views, schemas, parameters ($exp_id)	staging, answer_key, marts	Layered, reusable, safe-by-design SQL.
EXPLAIN	SQL workbench	Read the execution plan of a query.

6.2 The eleven analysis reports

activity_streaks.sql (Gaps and islands)

Question: What is the longest run of consecutive active days per user?

Distinct (user, day) pairs; for each user compute *day number minus ROW_NUMBER*. For consecutive days that value is constant, so it labels each unbroken run (an 'island'). Count the days in each island, take the maximum per user, then count users by longest streak. Shown in the app: most users' longest streak is 1 day (4,237 users in the 12,000-user demo warehouse), tapering off as streaks get longer.

daily_revenue_trends.sql (Window functions on a daily series)

Question: How is revenue trending?

Daily orders and revenue, then **SUM() OVER (ORDER BY day)** for the running total, **AVG() OVER (... ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)** for a 7-day moving average, and **LAG(revenue, 1)** and **LAG(revenue, 7)** for day-over-day and week-over-week change, with NULLIF to avoid dividing by zero.

data_quality_events.sql (Monitoring)

Question: How many events are duplicated or late, per day?

Counts raw events per day and distinct events per day (from the staging view), derives the duplicate rate and the late rate. The lines should be flat; the monitor finds the 0.60% and 1.51% the simulator planted.

experiment_interaction.sql (Self-join, 2x2 design)

Question: Does experiment A work the same with and without experiment B?

Finds users in both experiments, splits them into the four cells of (variant in A) x (variant in B), restricts orders to the window where both ran (GREATEST of the starts, LEAST of the ends) and returns users, orders and the sum of squared orders per cell so a variance can be computed in Python.

funnel_by_device.sql (Conditional aggregation)

Question: Where do users drop off, by device?

One row per session with flags (viewed, carted, checkout, purchased) built from **MAX((event_type = 'x')::INT)**; then step-to-step conversion per device: cart rate, cart-to-checkout, checkout-to-purchase and overall session conversion.

overlapping_experiments.sql (Self-join on time)

Question: Which experiments claim the same surface and share users?

Joins experiments to themselves on the same `exclusive_group` with overlapping dates (and e1.id below e2.id so each pair appears once), then counts distinct shared users.

rfm_segments.sql (Customer segmentation)

Question: Who are the champions, the at-risk and the lost customers?

Recency (days since last order), frequency (orders) and monetary value (revenue) per user as of a date, each scored 1 to 5, then rule-based labels with CASE: champions, loyal, new_or_promising, at_risk, lost, needs_attention, plus each segment's share of revenue with **SUM(SUM(monetary)) OVER ()**. **Fixed in v2:** the first version used NTILE, which splits users with *identical* values into different scores by user_id. Most buyers have one order, so frequency was scored arbitrarily. It now uses RANK, so ties get the same score (a test with eight tied users proves it).

sessionization_check.sql (LAG and running sums)

Question: If we rebuilt sessions from raw events, would we match the logged sessions?

LAG(event_ts) gives each event's previous timestamp for the same user; a gap of more than 30 minutes marks a new session; a running **SUM** of those flags numbers the sessions. Compares rebuilt and logged session counts by device (the ratio should be near 1).

srm_counts.sql (Conditional counts)

Question: How many users are in each arm of every experiment?

The input to the sample-ratio check: control_users and treatment_users per experiment, from **SUM((variant = 'control')::INT)**.

top_products_per_category.sql (Top-N per group)

Question: Which 3 products are carted most inside each category?

Cart-add counts per product, then **DENSE_RANK() OVER (PARTITION BY category ORDER BY cart_adds DESC)** and **QUALIFY rank_in_category <= 3**. Ties share a rank, so a category can return more than three rows.

weekly_retention.sql (Cohorts)

Question: Of users who signed up in a given week, how many are still active k weeks later?

Cohort = signup week. **Fixed in v2 (right-censoring):** a cohort can only be judged at week k if that week has already happened. The query builds a grid of every (cohort, week) pair that is observable, counts zero active users explicitly, and so never lets late cohorts or the unfinished final week drag later-week retention down. Two tests cover it.

6.3 The marts: one row per user

`sql/marts/experiment_user_metrics.sql` is the table every statistic reads. For one experiment (parameter `$exp_id`) it returns **one row per assigned user**:

Columns	Meaning	How it is computed
variant, user_id	Arm and user	From assignments
orders_in, revenue_in	The OUTCOME	Orders and revenue from the user's own assignment time to the end of the experiment. COUNT(*) FILTER (WHERE ...) and SUM(...) FILTER (...); users with no orders become 0 via LEFT JOIN + COALESCE.
orders_pre, revenue_pre, sessions_pre	The COVARIATES (CUPED and uplift features)	The same user's behaviour in the 28 days *before* the experiment started. Information from before assignment can never be affected by the treatment.
is_new, device, channel, city, days_since_signup	Profile	From users

The final line is `ORDER BY a.user_id`: a fixed row order so cross-fitting folds and bootstraps do not change between runs. `experiment_user_weekly.sql` returns one row per (user, *exposure week*), where week k is days 7(k-1) to 7k after the user's *own* assignment time (each user's novelty clock starts when they first saw the change), keeping only complete weeks inside the experiment window. It feeds the novelty analysis.

6.4 A whole experiment analysis in pure SQL

`sql/analysis/experiment_in_sql.sql` does the full variance-reduced analysis inside the database. The per-user mart is wrapped as a CTE called `metrics` and the query computes:

```
theta  = COVAR_SAMP(orders_in, orders_pre) / VAR_SAMP(orders_pre)        -- CUPED coefficient,
    pooled over both arms
y_adj  = orders_in - theta * (orders_pre - AVG(orders_pre))             -- same mean, smaller
    variance
lift   = mean_treatment / mean_control - 1                               -- relative lift
half   = 1.96 * SQRT( var_t/n_t/mean_c^2 + mean_t^2 * var_c/n_c/mean_c^4 )   -- delta-method 95%
    half-width
```

SQL equals Python
A test (and the SQL workbench page) runs this SQL and the Python engine on the same experiment and checks that the CUPED lift, the interval and the naive lift agree to **nine digits**. For exp_001 on the demo warehouse both give a CUPED lift of 0.092311 and a half-width of 0.089745 (the app shows this).

6.5 Two SQL bugs found by reading, and fixed (good interview stories)

Bug	Why it was wrong	Fix and proof
RFM scored with NTILE	When many users tie (most have 1 order), NTILE splits them across different scores by user_id, which is arbitrary, so segments were partly an artifact.	RANK-based scores: ties get the same score. A new test with 8 tied buyers and 2 clear champions checks that all eight land in one segment.
Retention counted unfinished weeks	A cohort that signed up recently cannot have been observed 8 weeks later; pooling it dragged later-week retention down.	A grid of observable (cohort, week) pairs with explicit zeros; two tests (one for censoring, one for zero counts).

7. A/B testing and statistics, concept by concept

Every concept below follows the same pattern: **the problem** in plain language, **the idea**, **the formula** (short), and **how this project uses it**. Read this chapter once slowly; the page-by-page tour later will then feel obvious.

7.1 Randomisation and the three kinds of number

Random assignment is the foundation. Because chance decides who is in which group, the groups are alike on average in *everything*, including things you never measured. Any difference in the outcome is then caused by the change. Every analysis reports three things: the **effect size** (how big), the **uncertainty** (how sure) and the **decision** (what to do). A p-value alone is only a fragment of the second.

Term	Meaning	Example from the project
Control / treatment	The old experience / the new one	No e-mail vs Men's e-mail
Outcome (metric)	What we measure per user	orders per user, revenue per user, conversion, spend
Relative lift	(treatment mean / control mean) minus 1	Men's e-mail conversion: 1.25% vs 0.57% is about +119%
Absolute effect	treatment mean minus control mean	+0.0068 conversions per user
Confidence interval	A range that would contain the true effect in about 95% of repeated experiments	+72% to +166% for Men's e-mail conversion
p-value	How surprising the data would be if there were truly no effect	3e-13 after Holm correction

7.2 Sample-ratio mismatch (SRM): was the split fair?

Problem. If you planned 50/50 but got 46/54, something is broken (a bug in assignment, bots, logging loss). Then the groups are not comparable and any lift is meaningless, however significant it looks.

Idea. Compare the observed arm sizes with the planned ones using a **chi-square goodness-of-fit test**: X^2 = sum of (observed - expected)^2 / expected. A tiny p-value means the sizes are too unequal to be chance.

Worked example. 5,200 vs 4,800 users with an equal plan: expected 5,000 each, $X^2 = 200^2/5{,}000 + 200^2/5{,}000 = 16$, p = 0.000063. A test in the project checks exactly this number.

In the project. Two thresholds are used: **p below 0.001 = DO NOT TRUST** (the whole analysis stops), and **p below 0.10 = a borderline note**. In the simulator, exp_006 shows only about 46% of users in the treatment arm and is flagged DO NOT TRUST; exp_004 has p = 0.062, shown amber on the Trust page. The test also works for more than two arms and for *unequal planned splits* (you can tell the Analyze page the split was 85/15, as in Criteo).

> **Common mistake**
> A very significant SRM is never fixed by 'adjusting' the analysis. It means the experiment itself must be investigated and re-run.

7.3 Effect size, uncertainty and the p-value

Relative lift is mean(treatment) / mean(control) - 1. Because it is a *ratio of two noisy averages*, its standard error needs the **delta method**:

```
se(lift) = sqrt( (se_t / mean_c)^2 + (mean_t * se_c / mean_c^2)^2 )        95% interval = lift +/-
    1.96 * se(lift)
p-value  = Welch two-sample t-test (does not assume equal variances)
```

With tens of thousands of users the normal approximation is excellent, which is why the project uses it (and bootstraps as a cross-check for revenue). **Reading an interval:** if it is entirely above zero the effect is real; if it is entirely above the 'worth-shipping' bar the win is clear; if it straddles zero we cannot tell.

7.4 CUPED: less noise from information you already have

Problem. Users differ enormously in how much they buy. That natural variation is *noise* that makes the interval wide, so you need more users or more days.

Idea. Each user's behaviour in the 28 days **before** the test (x) predicts how much they will buy during the test (y). Subtract the predictable part. Because x was measured before assignment, it is the same in both arms on average, so removing it does **not** change the average effect; it only removes noise. This is called **CUPED** (Controlled-experiment Using Pre-Experiment Data).

```
theta = cov(y, x) / var(x)
y_adjusted = y - theta * (x - mean(x))          mean unchanged,  variance multiplied by (1 - rho^2)
rho = correlation between y and x          (rho = 0.6 means 36% less variance, roughly 36% fewer
    users needed)
```

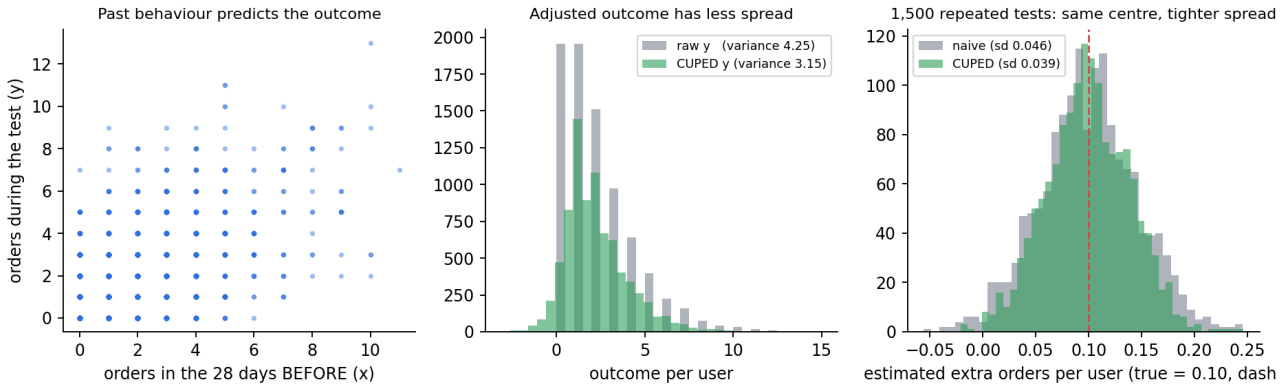


Figure 2. CUPED on simulated data: past behaviour predicts the outcome (left), so the adjusted outcome is less spread (middle); repeating the test 1,500 times gives the same centre but a tighter spread (right).

Several covariates. The platform generalises this: with a list of pre-experiment columns it fits an ordinary regression of y on them (pooled over both arms) and subtracts the fitted part. With one covariate this equals CUPED exactly (a test proves it). The result also reports how much narrower the interval became. **In the project:** in the simulator exp_001's interval narrows by about 4.5% (0.0940 to 0.0897 half-width on the demo warehouse). On the real Hillstrom spend outcome, adjusting for five pre-columns narrows it by only about 0.3%: spend is zero for nearly everyone, so past behaviour explains almost nothing. The platform reports that honestly instead of claiming a gain.

> **Watch out**
> Only use columns measured **before** assignment. A column that the treatment can change (for example, 'visited the site during the test') would bias the result. The app warns about this next to the covariate selector.

7.5 Multiple testing: when you run many experiments

Problem. Each test has a 5% chance of a false win. Run several and the chance that *at least one* is a false win grows quickly: $1 - 0.95^{m}$.

Number of tests (m)	1	5	10	20
Chance of at least one false win	5.0%	22.6%	40.1%	64.2%

Corrections. Bonferroni uses alpha/m for every test (simple, conservative). **Holm** is a step-down version that is never less powerful: sort the p-values from smallest, compare the smallest with alpha/m, the next with alpha/(m-1), and so on, stopping at the first failure. Both control the *family-wise error rate* (the chance of any false win). **Benjamini-Hochberg (BH)** controls the *false-discovery rate* (the share of wins that are false) and is less strict, used when you accept a few false discoveries.

Example from the tests. The p-values 0.001, 0.008, 0.039, 0.041, 0.042, 0.06, 0.074, 0.205, 0.212, 0.216: Bonferroni and Holm declare 1 win, BH declares 2. **Holm-adjusted p-values** are what the app displays: reject at the chosen level exactly when the adjusted p is at most that level. **In the project** the scorecard applies Holm across the 8 experiments whose assignment passed the sample-ratio check (exp_006 is excluded because it cannot be read at all). On the Analyze page, Holm is applied across the treatment arms (for example Men's and Women's e-mail vs control).

7.6 Power, minimum detectable effect and sample size

Problem. A test can fail to find a real effect simply because it had too few users. 'Not significant' then means *unknown*, not *zero*.

Idea. Power is the chance of detecting an effect of a given size (80% is the usual target). The **minimum detectable effect (MDE)** is the smallest relative lift the sample can reliably detect. They are two views of the same equation:

```
rate metric:    n per arm = (z_(1-alpha/2) + z_power)^2 * ( p0(1-p0) + p1(1-p1) ) * (1 - rho^2) / (p1
    - p0)^2
mean metric:    n per arm = 2 * ((z_(1-alpha/2) + z_power) * sd * sqrt(1 - rho^2) / (lift * mean))^2
MDE:            (z_(1-alpha/2) + z_power) * sqrt(2) * sd * sqrt(1 - rho^2) / ( sqrt(n) * mean )
```

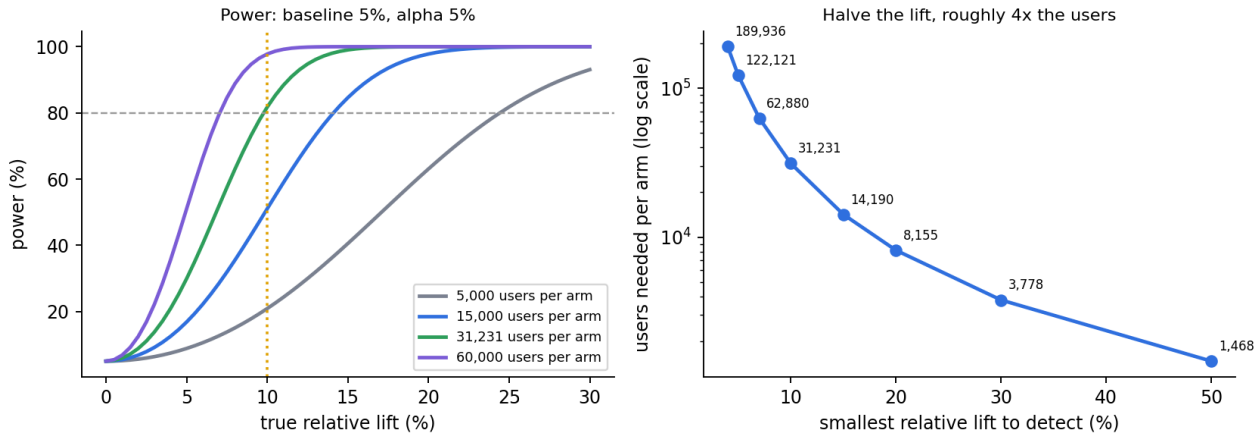


Figure 3. Left: power rises with sample size (baseline 5%, alpha 5%; 31,231 users per arm give 80% power at a 10% lift, dotted line). Right: users needed grow roughly four-fold each time the detectable lift is halved.

- **Three rules of thumb:** halve the lift you want to detect and you need about 4x the users; ask for 90% instead of 80% power and you need about a third more; add a covariate with correlation 0.6 and you need about 36% fewer.

- **In the project:** exp_002 (true +1.5%) had 47,446 users per arm but would have needed 350,424 without CUPED (296,022 with it). The smallest lift it could detect was about 4.1% (3.8% with CUPED), so it was *built to be underpowered* and the scorecard correctly says INCONCLUSIVE.
- The formulas are checked by simulation (power and false-positive rates measured by re-running thousands of tests), and by textbook examples in the tests (a 0.2-standard-deviation effect at 80% power needs about 392 users per arm).

7.7 Peeking and sequential testing

Problem. Opening the dashboard every day and stopping the first time p dips below 0.05 sounds harmless. It is not: each look is another chance for luck to cross the line.

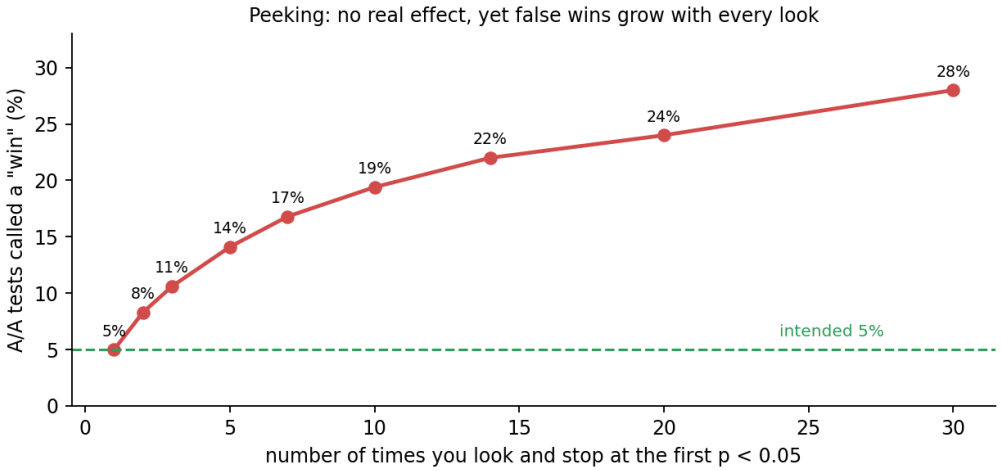


Figure 4. False-win rate of an A/A test (no real effect) as the number of looks grows. One look: 5%. Ten looks: about 19%. Thirty looks: about 28%.

Fixes. (1) Decide the sample size in advance and look once. (2) Use a stricter threshold at every look (**Bonferroni**, alpha divided by the number of looks: safe but conservative). (3) Use a **sequential boundary** such as **O'Brien-Fleming**, which is very strict early and relaxes toward the end, with the overall error held at 5%. The project calibrates the boundary c*sqrt(K/k) by Monte Carlo (bisection on c) and checks in a test that its overall false-positive rate is 5% within 0.6 percentage points.

In the project: with 10 looks the simulator gives naive 19.5%, Bonferroni about 2.7%, O'Brien-Fleming about 5.4% and a single look 5.0%. On the real 100,000-user warehouse the study found about 25% false wins for daily peeking over 28 days (the Overview page shows 25.0% vs 5.8% for one look).

7.8 Novelty effects: does the lift fade?

Problem. A new design can look great in week 1 because it is new, then fade. Averaging the whole window over-promises.

Idea. Measure the lift **by exposure week**, where week k means days 7(k-1) to 7k after each user's own assignment (so every user's novelty clock starts when they first saw the change). Weeks are **not independent** (the same users appear in several weeks), so the trend test first reduces each user to *one number*, the slope of their weekly orders against the week, and compares the average slope between arms with a Welch t-test. A negative difference means the effect is fading.

In the project. exp_004's planted truth is +12% fading to +1%, but the weekly noise is too large for the test to see it (p about 0.19), and the scorecard says SHIP. The platform lists this as a **known**

miss rather than hiding it; on exp_001 the trend is flat to slightly rising (+0.0017 orders per user-week per week, p = 0.25) and the latest full week is +14.9%. The test checks the method on synthetic data: it detects a fading effect (change per week about -0.027, p below 0.001) and does not flag a constant one.

7.9 Interaction between overlapping experiments

Problem. Two experiments on the same surface (exp_001 checkout button, exp_004 homepage redesign) share users. Does the effect of A depend on whether the user also got B?

Idea. A 2x2 table of (variant in A) x (variant in B). **Interaction = (effect of A when B is on) minus (effect of A when B is off).** Zero means the effects simply add. The standard error comes from the four cell variances (the SQL returns orders and the sum of squared orders so Python can compute them).

In the project: the effect of exp_001 is +0.0414 orders per user when exp_004 is on and +0.0288 when it is off; the interaction is +0.0126 ± 0.0203 (p = 0.22). No interaction was detected, which is *not proof* there is none, and the memo keeps a 5% holdback for that reason. A test proves the method: additive effects give p above 0.99; a planted -0.2 interaction is recovered exactly with p below 0.001.

7.10 Heavy-tailed revenue

Problem. Revenue per user is mostly zero with a few huge baskets: in the simulator about 74.7% of users spent nothing and the top 10% of users hold about 76% of the revenue. Means of such data are noisy.

Estimator	What it does	Caution
Naive mean ratio	Plain difference of means with the delta-method interval	Wide when a few big spenders dominate
CUPED	Removes the part explained by pre-period revenue	Helps only if past revenue predicts present revenue
Winsorised at the pooled 99.5th percentile	Caps extreme values at the *same* cap in both arms	Changes WHAT is measured (a capped mean): shown next to, never instead of, the uncapped estimates
Bootstrap percentile	Resamples users 1,000 times; no normality assumption	Slower; confirms the interval
Purchase rate x revenue per buyer	Splits the lift into 'more buyers' and 'bigger baskets'	Descriptive only: buyers are a post-treatment group

exp_007 (free-delivery threshold) is built to move basket value, not conversion, so the decomposition shows the lift coming from bigger baskets.

7.11 Guardrails and unit economics

Guardrail. A second metric that must not get worse. For an orders test the guardrail is revenue per user; for a revenue test it is orders. A *significantly negative* guardrail (the upper end of its interval is below zero) blocks shipping.

Economics. A statistically significant lift can still lose money. For an offer that costs money to send:

```
profit per user = value_per_unit * (extra outcome per user) - cost_per_treated_user          (with a
    95% interval)
```

If the whole interval is below zero (provably unprofitable) the decision is **ITERATE**: do not ship to everyone, target a subset. If the interval includes zero it is not proven unprofitable and shipping is not blocked. **In the project** the win-back offer (exp_008) raises orders +7.1% (95% interval +4.4% to +9.8%) but, at INR 50 per extra order and INR 3 per user, loses INR 1.29 per user (interval -2.03 to -0.55), about INR 129,000 per 100,000 users. It breaks even only if an order is worth about INR 88 or the offer costs under INR 1.71 per user (simple arithmetic on the same figures).

7.12 The decision rule: six gates in order

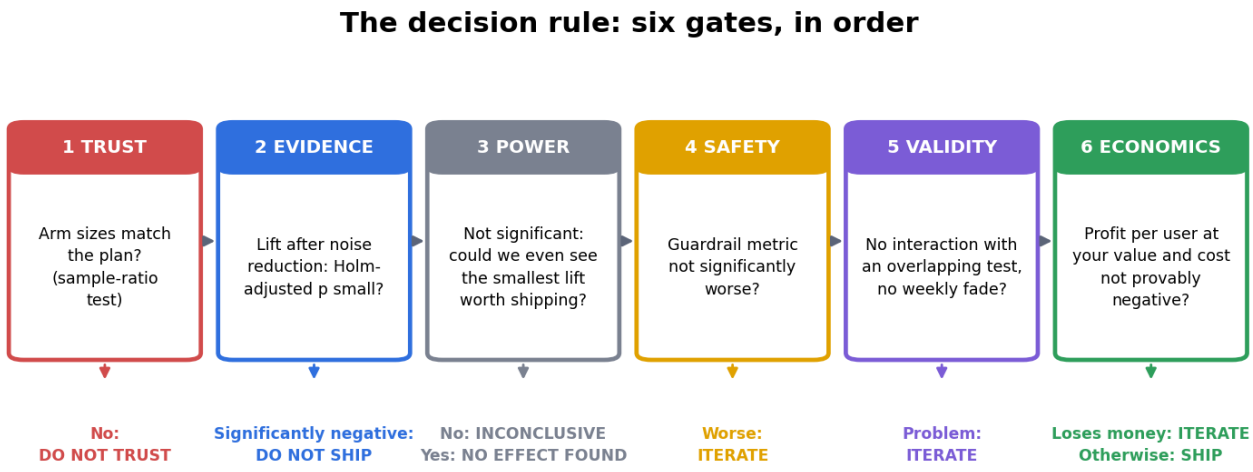


Figure 5. The scorecard's decision rule. A gate that stops the process means the later gates are not read.

Decision	When	What it tells the team
DO NOT TRUST	Sample-ratio p below 0.001	Assignment is broken; fix and re-run. Nothing else is read.
DO NOT SHIP (harmful)	Significantly negative lift	The change hurts.
SHIP	Significant positive lift, no blocker	Ship, with notes (borderline SRM, interval below the bar, overlap, profit, fading week).
ITERATE	Significant, but a guardrail is worse, an interaction is found, the effect is fading, or it is provably unprofitable	Real effect, but do not ship as is.
INCONCLUSIVE (underpowered)	Not significant and the sample could not detect a lift as small as the worth-shipping bar	Unknown, not zero. Needs more users.
NO EFFECT FOUND	Not significant and the test could have detected the bar	A real null, with the lift interval.

The result on the nine simulated experiments (100,000 users) and how it compares with the hidden truth:

Experiment	Decision	Hidden truth	Verdict on the platform
exp_001 checkout button	SHIP (+11.1%, 95% interval +8.0% to +14.3%)	+8%	Right
exp_002 delivery-fee banner	INCONCLUSIVE	+1.5%	Defensible (built underpowered)

Experiment	Decision	Hidden truth	Verdict on the platform
exp_003 recs carousel	INCONCLUSIVE	Exactly 0	Defensible
exp_004 homepage redesign	SHIP	+12% fading to +1%	**Miss:** weekly noise too large to see the fade
exp_005 onboarding flow	INCONCLUSIVE	+18% for new users only	**Miss:** a segment-only effect
exp_006 search ranking	DO NOT TRUST	0, planted assignment bug	Right
exp_007 free-delivery threshold	SHIP	+6% basket value	Right
exp_008 win-back offer	ITERATE	Varies by channel and device	Right: real lift, unprofitable for everyone
exp_009 personalised offer	INCONCLUSIVE	Hidden-segment effect	Defensible

Interview tip
Two misses are written on the Limits page and in the table above. Showing where a tool is blind is what makes the rest of it credible. Both are about *heterogeneity* (an effect that exists only for some users or only for a while), which the scorecard does not examine.

7.13 Is the platform itself trustworthy? (validation by re-simulation)

A tool that gives confident answers must be checked itself. `src/experiments/validation.py` re-runs the simulator with a fresh random seed, repeats the whole analysis (naive and CUPED), and does it **100 times** with 20,000 users each, for the experiments whose true effect is constant and known (exp_001, 002, 003, 007) plus the bug experiment exp_006 on purpose.

Check	Question	Good result
Bias	Is the average estimate equal to the built-in truth?	Within about 2 standard errors of zero
SE ratio	Is the reported standard error equal to the real spread of the estimates?	About 1 (below 1 means intervals are too narrow)
Coverage	How often does the 95% interval contain the truth?	About 95% (with a Wilson interval, honest with few repeats)
Rejection rate	How often is p below 0.05?	Power for real effects; the false-positive rate (about 5%) for exp_003
CUPED gain	How much narrower is the CUPED interval than the naive one?	Below 1

Result (shown on the Platform validation page): clean experiments are unbiased (Yes); interval coverage is 94% to 100%; and the planted bug experiment is flagged (Yes), with an average error of roughly +13% to +19% and coverage near zero, exactly what a broken assignment should produce. The validation code itself is tested by feeding it synthetic results with intervals that are too narrow or biased estimates and checking that it complains.

8. Uplift modelling: who should get the offer?

8.1 Why an average effect is not enough

The win-back offer (exp_008) raises orders by about 7% on average, yet at the assumed economics sending it to everyone *loses* money. The reason: the offer helps some users a lot, does nothing for many, and **hurts** some. **Uplift modelling** estimates, *for each user*, how much extra the offer causes. That is different from predicting who will buy.

Type of user	Buys without the offer?	Buys with the offer?	Offer worth sending?
Persuadable	No	Yes	**Yes**: the offer causes the purchase
Sure thing	Yes	Yes	No: you pay for something that would have happened
Lost cause	No	No	No: wasted cost
Sleeping dog	Yes	No	**Never**: the offer drives them away

The lowest-scored tenth of users in exp_008 actually ordered about 0.16 *fewer* orders per user when shown the offer; the highest-scored tenth ordered about 0.24 *more*. Excluding the first group matters as much as finding the second.

8.2 The three learners

All three use **gradient-boosted trees** (scikit-learn `HistGradientBoostingRegressor`; Poisson loss for counts; 150 trees, depth 4, minimum 200 users per leaf, L2 regularisation 1.0). Settings were fixed, not tuned, and no deep learning is used. The features are only things known **before** the experiment: orders_pre, revenue_pre, sessions_pre, days_since_signup, is_new, and the categories device, channel and city (8 in all).

Learner	How it works	Strength / weakness
S-learner (single)	One model sees the features plus a 'got the offer' flag. Uplift = prediction with the flag on minus prediction with it off.	Simple and stable; can shrink the effect toward zero if the flag is a weak feature.
T-learner (two)	One model trained on treated users, one on control users. Uplift = the difference of their predictions.	Flexible; each model sees only half the data, so it can be noisy.
X-learner (cross)	Uses the two models to *impute each user's missing outcome*, then regresses those imputed effects and blends with the propensity.	Strong when treatment and control groups differ in size; more moving parts.

8.3 Cross-fitting: nobody is scored by a model that saw them

Users are split into 4 folds. For each fold, the models train on the other three and score the held-out fold. Every user therefore gets a score from a model that never saw their outcome. Without this, the model would look brilliant simply by memorising its training data.

8.4 Scoring without knowing anyone's individual effect

We never observe a user's effect (a user is either treated or not). Because the experiment is randomised with a known share treated *p*, we can build an **unbiased per-user pseudo-effect**:

```
tau_i = y_i * ( t_i / p  -  (1 - t_i) / (1 - p) )          average of tau_i = the average treatment
    effect
```

Sort users from the highest score down and add up tau_i as you go: that is the **gain curve** (a Qini-style curve) of estimated extra orders when you treat the top-k users. The **Qini area** is the average gap between the model's curve and random targeting, in **extra orders per 1,000 users**. Bootstrap resamples give intervals, and a model only counts if its *paired* difference from random excludes zero.

> **Watch out**
> A weakness shown honestly: with heavy-tailed outcomes (such as Hillstrom spend) even the 'random' baseline can score tens of units either way. The page therefore shows intervals, and says plainly when no model clearly beats random.

8.5 From scores to money: the profit curve and an honest cut-off

```
profit(k) = value_per_unit * (cumulative estimated extra outcome from the top k users) -
    cost_per_user * k
```

The best reach is where this curve peaks. But picking the peak on the same data that scores it is **optimistic** (you chose the luckiest point). The honest version, **hold-out profit**, picks the best cut-off on a random half of the users and scores it on the other half. Both numbers are shown so the optimism is visible, as profit per 1,000 users.

8.6 Stability: repeating with 10 seeds

One run depends on the random folds and model seeds. `multiseed.py` repeats the whole evaluation with 10 seeds and reports the spread, how often each learner is best, and a **paired bootstrap** of the *differences* between learners (the same resampled users score both models). Seed spread measures algorithmic noise; the bootstrap measures the noise from having only 100,000 users. Both matter.

8.7 What exp_008 found

Question	Answer (assumed economics: INR 50 per extra order, INR 3 per user)
Overall lift	+7.1% (95% interval +4.4% to +9.8%), significant after Holm
Send to everyone	INR -1.29 per user (interval -2.03 to -0.55), about INR -129,000 per 100,000 users
Send to the model's top slice (cut-off chosen on one half, scored on the other)	S-learner: INR 1,107 per 1,000 users (INR 1,260 when scored on the same users, the optimistic figure)
Other models, same method	X-learner INR 944, T-learner INR 647, random targeting INR -138 per 1,000 users
Learner ranking	S beats T clearly (paired bootstrap +9.3, 95% interval +4.1 to +15.0); S vs X marginal (+5.1, +0.3 to +10.3); X vs T cannot be separated; all three beat random by a wide margin
Stability	S-learner scored 22.6 extra orders per 1,000 users vs random (sd 1.2 across seeds) and was best of the three in 10 of 10 seeds
Who benefits (S-learner, by predicted tenth)	Lowest tenth -0.158 extra orders per user; the next four tenths close to zero (at most +0.012); ninth tenth +0.158; top tenth +0.242 (predicted 0.222)

Decision

The recommendation in the targeting memo: do **not** send the offer to everyone; send it to roughly the top 24% ranked by the S-learner; then run a fresh randomised test of that targeted policy against a no-offer holdout before rollout. The fresh holdout is listed as **not yet tested**.

9. Real-data work

9.1 Criteo: the same checks on a real advertising experiment

The Criteo checks (`src/criteo/`) re-use the same methods on 1,000,000 real users. The order of the checks matters:

1. **Was it really randomised?** Compare every feature between treated and control with the **standardised mean difference (SMD)**. Under perfect randomisation at this size SMDs are about plus or minus 0.003; common practice treats below 0.1 as acceptable. Here all 12 features are slightly unbalanced (the largest is 0.052), small but systematic, which means the naive estimate is a little biased.
2. **Can the features predict treatment?** A cross-validated model tries to predict the treatment flag from the features; an AUC of 0.5 means perfectly randomised.
3. **Three ways to estimate the effect. Naive** difference in rates; **regression-adjusted (Lin)**, which regresses on centred covariates and their interactions with treatment, with robust standard errors; **AIPW (doubly robust)**, which combines an outcome model and a propensity model so that one can be wrong without ruining the answer. The three should agree if randomisation were perfect.
4. **Analyse by assignment, never by exposure.** Only about 3.6% of treated users actually saw an ad. Exposure happens *after* assignment, so conditioning on it breaks the randomisation (post-treatment bias). Effects are per user assigned.

Outcome	Naive effect	Adjusted (doubly robust) effect	Difference
Visit	+1.15 percentage points	+0.83 percentage points	28% lower: part of the naive number was bias from the imbalance
Conversion	+0.12 percentage points	+0.11 percentage points	13% lower

Who responds. Ranking users with the models and measuring the effect by predicted tenth, the top tenth gains about 6 percentage points of visits and holds about **72%** of all incremental visits (the first-pass method said 82%; the robust AIPW-scored figure is the one to quote). This is real, learnable heterogeneity, in contrast to Hillstrom below.

9.2 Hillstrom: the full engine on a real e-mail test

The live Analyze page and the REST API run the generic engine on the Hillstrom file. Results below come from that engine (no covariates, Holm across the two e-mails), computed for this guide.

Outcome	Arm	Mean (arm vs control)	Relative lift (95% interval)	Holm-adjusted p	Decision*
conversion	Mens E-Mail	0.0125 vs 0.0057	+118.8% (+72.2% to +165.5%)	3.0e-13	SHIP
conversion	Womens E-Mail	0.0088 vs 0.0057	+54.3% (+19.3% to +89.3%)	1.6e-04	SHIP
visit	Mens E-Mail	0.1828 vs 0.1062	+72.1% (+63.8% to +80.4%)	2.7e-112	SHIP
visit	Womens E-Mail	0.1514 vs 0.1062	+42.6% (+35.4% to +49.8%)	2.4e-44	SHIP
spend	Mens E-Mail	1.4226 vs 0.6528	+117.9% (+54.4% to +181.4%)	2.3e-07	SHIP

Outcome	Arm	Mean (arm vs control)	Relative lift (95% interval)	Holm-adjusted p	Decision*
spend	Womens E-Mail	1.0772 vs 0.6528	+65.0% (+14.9% to +115.1%)	1.1e-03	SHIP

*Without unit economics (the economics gate is switched off here), so a significant lift ships. With the page defaults (USD 30 per conversion, USD 0.15 per e-mail) Women's e-mail turns to ITERATE, see chapter 13. **Sample-ratio check: p = 0.904** (arm sizes 21,306 / 21,307 / 21,387), so the split is clean.

- **Both e-mails clearly raise visits, conversion and spend; Men's e-mail raises them more.**
- **Covariate adjustment barely helps spend** (interval 0.3% narrower for Men's e-mail, no change for Women's): spend is zero for almost everyone.
- **No uplift model beats random targeting** on this data, for conversion or spend (every bootstrap interval for the difference from random includes zero). The Live targeting page therefore says so and recommends deciding between 'everyone' and 'nobody' from the economics.
- **The contrast is the lesson:** learnable heterogeneity exists in Criteo and in the simulator's exp_008, but not here, and the platform tells you which situation you are in.

10. The platform layer: engine, API, security and tests

10.1 The engine modules

File (src/engine/)	What it does
analyze.py	`analyze_experiment(df, variant, outcome, covariates, config)`: SRM (any number of arms, optional unequal plan), per-arm lift (naive and adjusted), Holm across arms, MDE, optional guardrail and economics, then the decision for each arm. Clear errors for bad input (missing control, non-numeric column, an arm with fewer than 30 users, a zero control mean).
decision.py	The six-gate decision rule as one pure function with the thresholds as arguments (the same code the scorecard uses).
planner.py	Sample size, duration (rounded up to whole weeks), MDE for a fixed sample, power curve and days-versus-lift curve; Bonferroni split of alpha for several arms.
sequential.py	The peeking simulator: false-positive rates for naive, Bonferroni and O'Brien-Fleming rules.
uplift_core.py	S, T and X learners, the pseudo-effect, gain curve, Qini area and hold-out profit (no database or plotting code).
targeting.py	Cross-fitted scoring for any arm vs control, profit curves for any value and cost, bootstrap Qini table, honest policy summary.
datasets.py	Loaders for Hillstrom and Criteo, the in-memory demo warehouse, CSV upload parser with a row cap, guessing which column is the arm.
sqlguard.py	Validation and lock-down for the public SQL box.
make_samples.py	Builds the small committed sample files the live app needs.

10.2 The REST API

Endpoint	Input	Returns
GET /health	none	{status: ok, version}
GET /datasets	none	The bundled datasets, their columns and whether they are available
POST /analyze	JSON: records, variant, outcome, covariates, config (control, alpha, min_effect, planned_split, value_per_unit, cost_per_treated, guardrail ...)	SRM result and one decision object per treatment arm
POST /analyze/csv	A CSV file upload plus the same mapping as form fields	Same as /analyze
POST /plan	kind (rate or mean), baseline, rel_lift, sd, alpha, power, rho, daily_users, n_arms	n per arm, total users, days, weeks, and a power curve
POST /peeking	n_looks, alpha, n_sims	False-positive rate per stopping rule

Inputs are validated by Pydantic: alpha must be between 0 and 0.5, power between 0.5 and 0.999, and so on; a request outside the limits gets a clear **422** error with a readable message (for example *control arm 'nope' not found; arms in the data: [...]*). Limits protect the free server: 200,000 JSON records, 30 MB or 1,000,000 rows per upload. Nothing is stored.

Example. Request to POST `/plan` and the (shortened) response:

```
request:  {"kind": "rate", "baseline": 0.1, "rel_lift": 0.2, "daily_users": 1000}
response: {"n_per_arm": 3839, "total_users": 7678, "alpha_used": 0.05, "sd": 0.3, "days": 8,
    "weeks": 2}   (power_curve omitted)
```

10.3 Security of the SQL box

A public page that lets strangers type SQL needs defence in depth. Three layers protect it:

1. **Text guard** (`validate_readonly`): one statement only; must start with SELECT, WITH, EXPLAIN, DESCRIBE or SHOW; forbidden words such as DROP, DELETE, INSERT, COPY, ATTACH, INSTALL, PRAGMA, SET and the file-reading functions (read_csv, read_parquet, glob, ...) are rejected; comments and string literals are handled so 'drop' inside a string is not a false alarm.
2. **Locked connection**: external (file and network) access is switched off, configuration is locked so a query cannot switch it back on, memory is capped at 1 GB and threads at 2. A test proves a locked connection cannot read a file and cannot re-enable access even if the text guard were bypassed.
3. **Limits and data**: results are capped at 1,000 rows, and the database holds only simulated data. The hidden answer key is not even loaded into it.

10.4 Tests: 167 automated checks

Area	What is checked
Statistics helpers	Textbook values (sample size 392 for a 0.2-SD effect, power 80% at the required n, MDE and n are inverses), multiple-testing worked example, O'Brien-Fleming false-positive rate.
Scorecard	Every branch of the decision rule, including 'exactly at the threshold is not a failure', and that the scorecard builds from a database with *no* answer-key tables and catches exp_006.
SQL layer	One test per query on a tiny hand-built database, including ties (RFM), censoring (retention), the gaps-and-islands trick, LAG sessionisation, QUALIFY ties and that RFM ignores orders after the as-of date.
Simulator	Unique keys, valid foreign keys, no experiment before the pre-period ends, the planted SRM only in exp_006, realistic conversion, retention decay, deterministic with a seed.
Novelty, interaction, revenue, validation, uplift, multiseed	Detect a planted fade but not a constant effect; additive effects give p above 0.99; winsorising uses one pooled cap; the validation code flags narrow intervals and bias; a leakage guard checks that only the 8 pre-experiment features reach the models; hold-out profit is less optimistic than in-sample for a useless score.
Engine	Known lifts recovered, CUPED narrows intervals and keeps the mean, SRM blocks even a huge lift, unequal plans, Holm matches a hand calculation, guardrail and economics change the decision, bad input gives clear errors, real Hillstrom arm sizes.
SQL equals Python	The pure-SQL analysis and the Python engine agree to nine digits on three experiments.
API	All endpoints, 422 errors, CSV upload, config changes the decision.
SQL guard	Allowed and forbidden statements, and the lock-down of the connection.
App	Every page renders with no artifacts and with the real ones; widgets are driven (plan sizes scale with the sliders; the decision flips from SHIP to ITERATE as the cost per e-mail rises; DO NOT TRUST appears for exp_006; DROP TABLE is refused).

> **Interview tip**
> A principle used throughout: **test statistics code against answers you worked out by hand**, and test the tests (for example, feed the validation code deliberately broken inputs and check that it complains).

10.5 Continuous integration, Docker and deployment

- **CI** (`.github/workflows/ci.yml`): on every push, run ruff and the full test suite on Python 3.11 and 3.12, then build the Docker image and smoke-test `/health` and `/plan` inside the container.

- **Docker**: a slim Python image, dependencies first (so rebuilds are fast), non-root user, health check, port taken from the environment.
- **Streamlit Community Cloud**: deploys `app/streamlit_app.py` from GitHub on every push; dependencies come from `app/requirements.txt`. Free apps sleep when idle, so open the link a few minutes before sharing it.
- **Data in the cloud**: the app reads only small committed files (`artifacts/`, `docs/`, `sql/`, `sample_data/hillstrom.csv.gz`) and builds a 12,000-user demo warehouse in memory; no database server and no large files are needed.

PART C

The app, page by page

A guided tour of every page of the live application: what it is for, what you see, every control and what it changes, and how to read the result.

11. A tour of the live app

The live app has **16 pages** in a left-hand menu. They fall into two groups: five **interactive pages** (new in v2) where you change inputs and the answer changes immediately, and eleven **results pages** that explain how the platform was built and what it found. Every page opens with a box of three short sentences: *What this does*, *Why it matters* and *How to use it*.

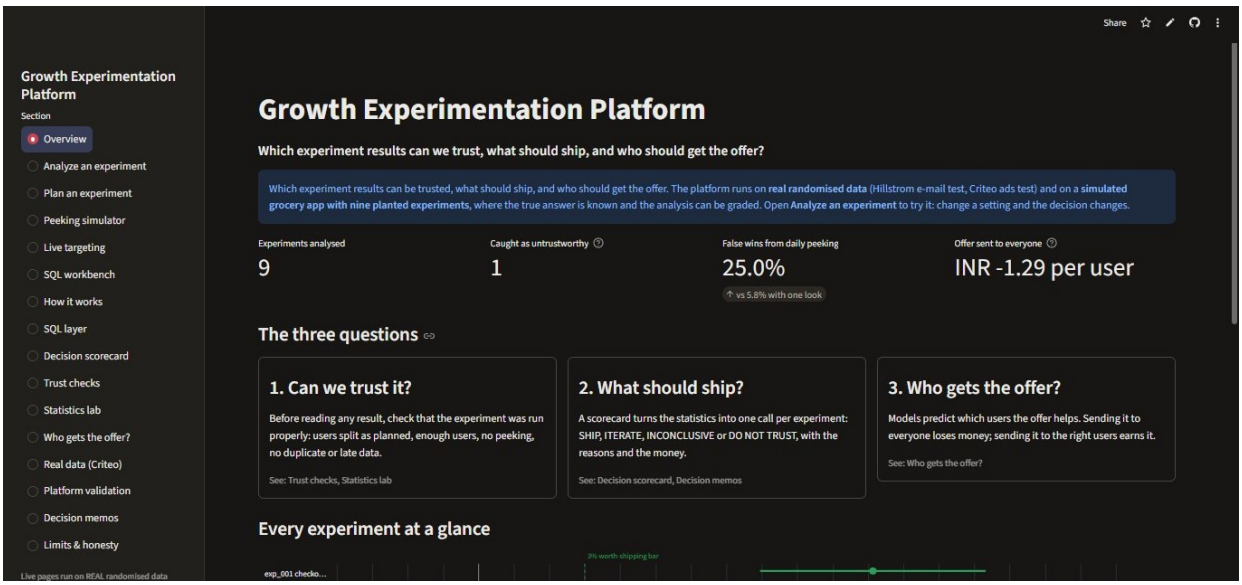
Group	Pages
Interactive (change a control, the answer changes)	Analyze an experiment, Plan an experiment, Peeking simulator, Live targeting, SQL workbench
Orientation	Overview, How it works, SQL layer
The decision work	Decision scorecard, Trust checks, Statistics lab, Who gets the offer?, Real data (Criteo), Platform validation, Decision memos
Honesty	Limits and honesty

Navigation

The menu. The left panel is one list; click a name to open that page. The panel is sized to about 18% of the screen (never below 15 or above 18.5 rem) so the content keeps the room, and you can drag its edge to resize it.

11.1 Overview

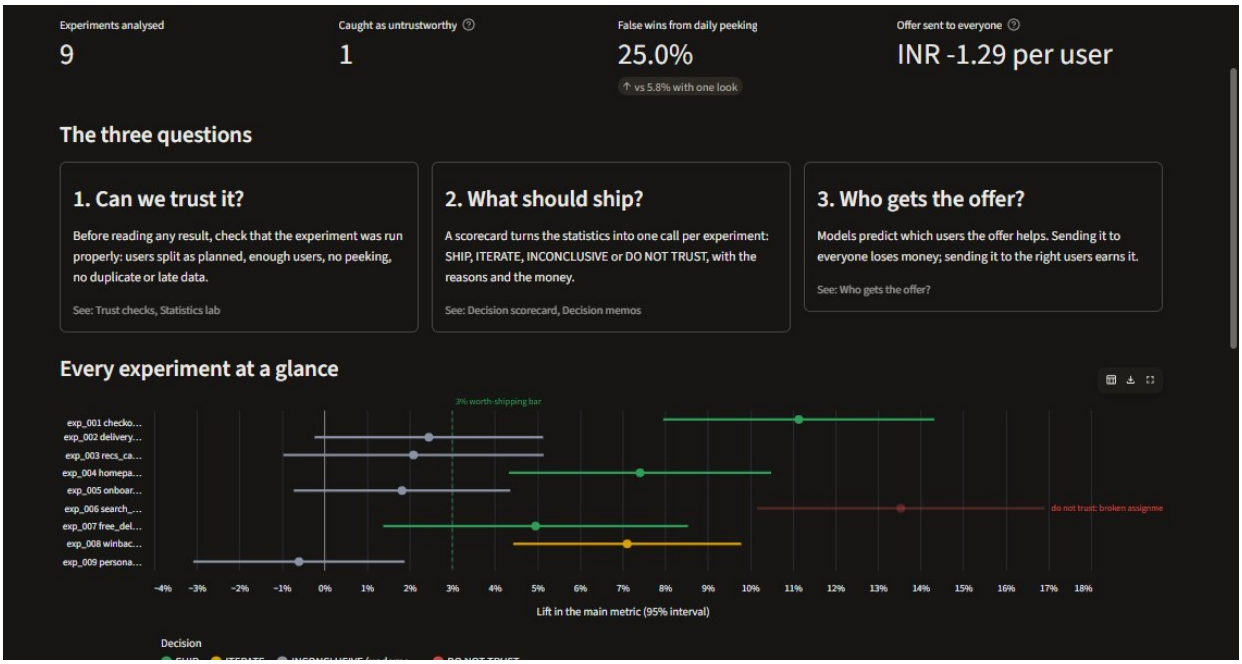
At a glance	
Purpose	The front door: the three questions the platform answers, four headline numbers, and every experiment at a glance.
Controls	None. It is a summary computed from the saved results.
Reads from	artifacts/scorecard.csv and other result files



Screenshot 6. The Overview page with the menu on the left, four headline numbers and the three questions.

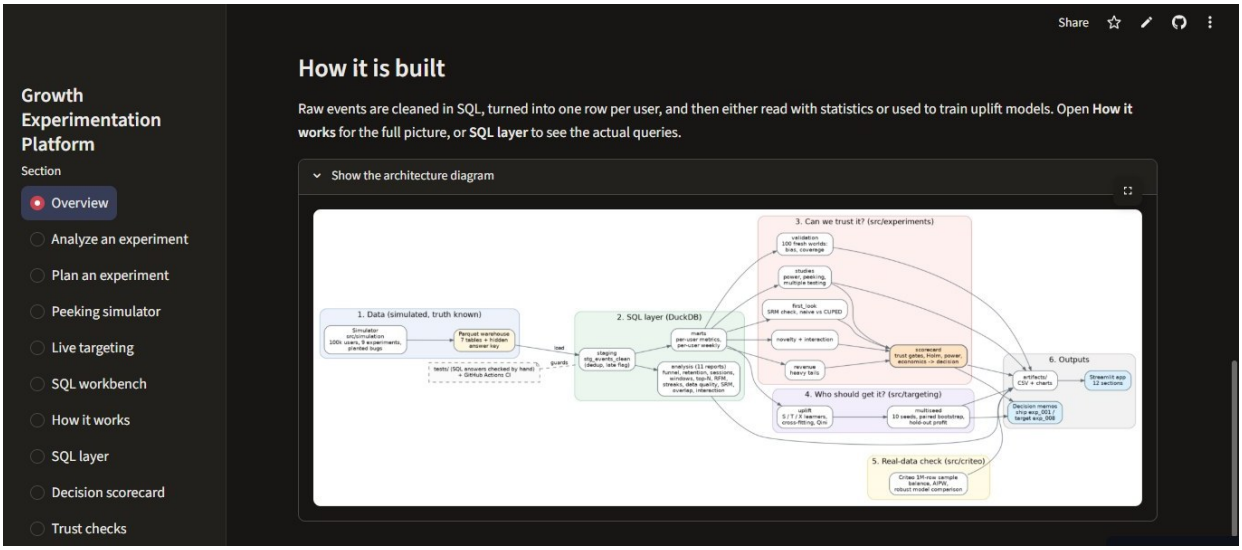
The four headline numbers

Number	Meaning
Experiments analysed: 9	The nine simulated experiments in the warehouse.
Caught as untrustworthy: 1	exp_006, flagged DO NOT TRUST by the sample-ratio check (the planted assignment bug).
False wins from daily peeking: 25.0%	If you check an A/A test every day for 28 days and stop at the first p below 0.05, 25% of tests with no real effect are called wins, against 5.8% with one look (the small gap from exactly 5% is simulation noise).
Offer sent to everyone: INR -1.29 per user	The win-back offer raises orders but loses money per user if sent to all, at INR 50 per extra order and INR 3 per user.



Screenshot 7. 'The three questions' cards and 'Every experiment at a glance'.

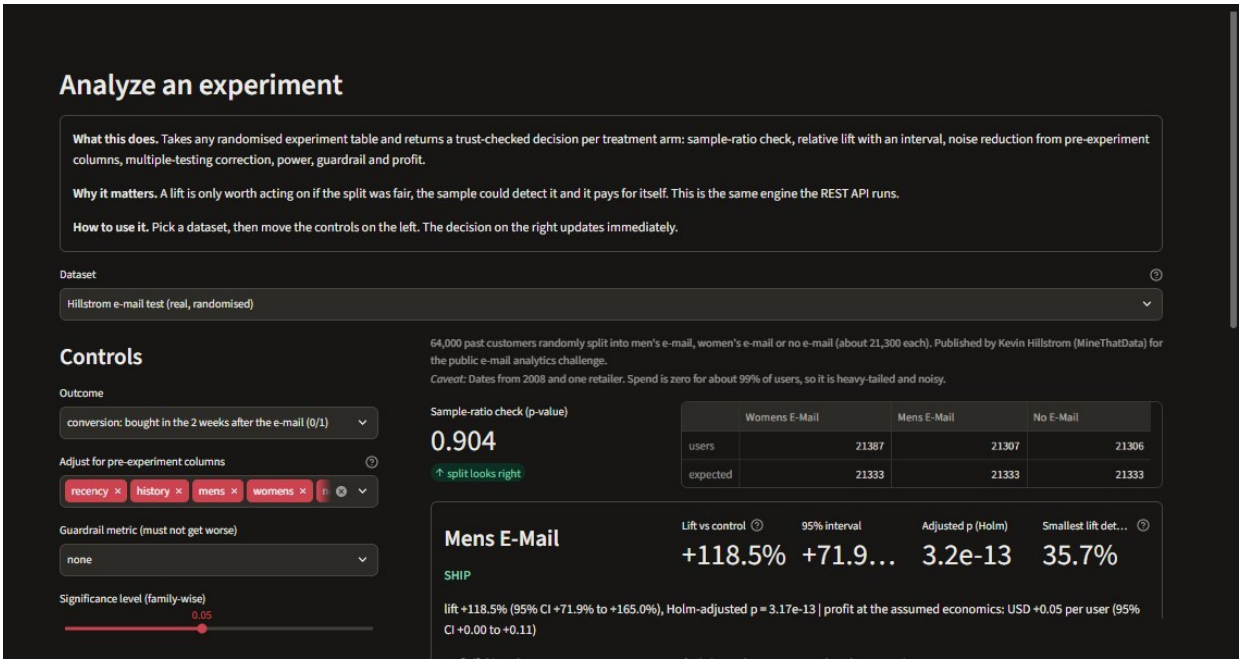
Reading 'Every experiment at a glance'. Each line is one experiment's 95% interval for its main metric's lift; the dot is the estimate. The dashed green vertical line is the **3% worth-shipping bar**. Colours are the decisions: green = SHIP, amber = ITERATE, grey = INCONCLUSIVE, red (faded, labelled) = DO NOT TRUST. A line entirely to the right of the dashed bar is a clear win; a line that crosses zero cannot be told apart from no effect.



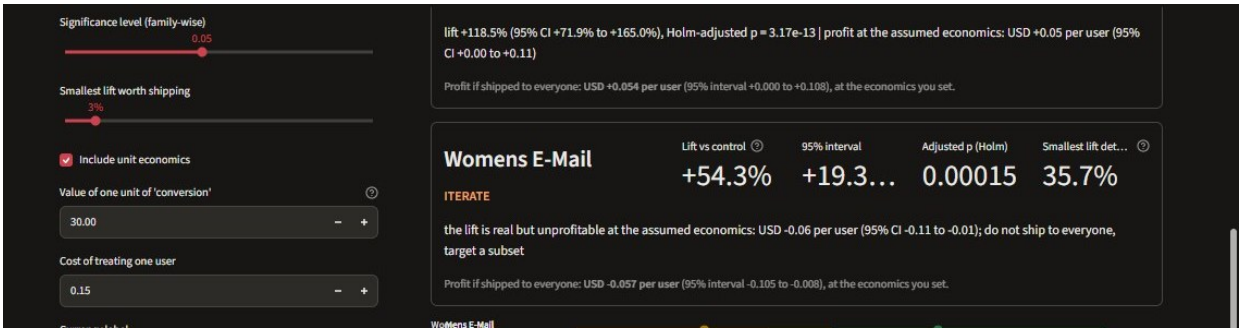
Screenshot 8. 'How it is built' on the Overview page: an expandable architecture diagram of the original (v1) pipeline.

11.2 Analyze an experiment (interactive)

At a glance	
Purpose	Takes any randomised experiment table and returns a trust-checked decision for every treatment arm. This is the same engine the REST API runs.
Data you can use	Hillstrom e-mail test (real), Criteo ads test (real, if its sample file is present), the simulated app (nine experiments with known truth), or your own CSV.
Output	Sample-ratio result, one decision card per treatment arm (lift, interval, Holm-adjusted p, detectable lift, reasons, profit), an interval chart, notes and a downloadable table.



Screenshot 9. Analyze an experiment: choosing the Hillstrom dataset, the controls on the left and the sample-ratio check on the right.



Screenshot 10. The Controls column continues (significance level, worth-shipping bar, unit economics) and the second decision card appears. (The interval chart in this screenshot is squashed by a display bug that is fixed in the accompanying ui-fixes update.)

Every control, what it means and what it changes

Control	Meaning	What happens when you change it
Dataset	Which table to analyse.	Switches the whole page. Hillstrom is real; 'Simulated app' lets you also reveal the planted truth; 'Upload my own CSV' opens a mapping form.
Experiment (simulated only)	Which of the nine experiments.	exp_006 turns every card to DO NOT TRUST; exp_001 usually gives SHIP. The demo warehouse has 12,000 users (not 100,000), so some verdicts differ from the Scorecard page (smaller samples are less powerful).
Outcome	The metric being compared (for Hillstrom: conversion, visit, spend).	Changes every number. Conversion and visit are 0/1 flags; spend is dollars.
Adjust for pre-experiment columns	Columns measured before the test, used for CUPED/regression noise reduction.	More useful columns means a narrower interval (the page prints how much narrower). Never select a column the treatment could change.
Guardrail metric	A second metric that must not get worse.	If it is significantly worse for an arm, a would-be SHIP becomes ITERATE with the reason written out.
Significance level (family-wise) 0.01 to 0.10	The tolerated chance of any false win across the arms (Holm).	Raising it makes 'significant' easier (more false-win risk); lowering it makes it harder. With very small p-values (Hillstrom) the verdict does not change.
Smallest lift worth shipping 0% to 30%	Your business bar, as a relative lift.	It never changes the lift or the p-value. It decides INCONCLUSIVE vs NO EFFECT FOUND when nothing is significant (a test that cannot detect the bar is underpowered), and it adds a note when the interval's low end is below the bar.
Include unit economics	Switches the profit gate on.	When on, a significant lift that is provably unprofitable becomes ITERATE.
Value of one unit of the outcome	Money earned per unit of the outcome (per conversion, per order, per dollar of spend as margin).	Raising it makes profit per user larger; low values make shipping unprofitable.
Cost of treating one user	What it costs to give one user the treatment (an e-mail, an offer).	Raising it lowers profit; past break-even the decision flips from SHIP to ITERATE.
Currency label	Text only (USD, INR ...).	Changes labels, not numbers.
The planned split was NOT equal	Tell the sample-ratio test the intended shares per arm (for example 85/15).	A wrong plan makes a fair split look broken. Hillstrom with the correct equal plan has p = 0.904; entering a 50/25/25 plan gives a tiny p and every card becomes DO NOT TRUST.
(Upload) Arm column, Control arm, Outcome column, Pre-experiment columns	Maps your CSV to the engine; the page guesses sensible defaults.	Up to 1,000,000 rows; the CSV is analysed in memory and discarded.

What the results area shows

- **Sample-ratio check (p-value)** and a table of users per arm vs expected. Green 'split looks right' or red 'SPLIT IS BROKEN: do not read the results'.
- **One card per treatment arm** with the decision (coloured), **Lift vs control**, the **95% interval**, the **Adjusted p (Holm)**, the **Detectable lift** (the smallest relative lift this sample could reliably detect), the plain-English reasons, and 'Profit if shipped to everyone' with its own interval.
- **An interval chart** of all arms against zero and the dashed green worth-shipping bar.
- **Notes** (for example 'adjusted for 5 pre-experiment covariate(s); the interval is 0% narrower on average', and 'Holm correction applied across 2 treatment arms').
- **All numbers** (an expandable table, with a Download as CSV button) and, for the simulator, **Reveal the planted truth**.

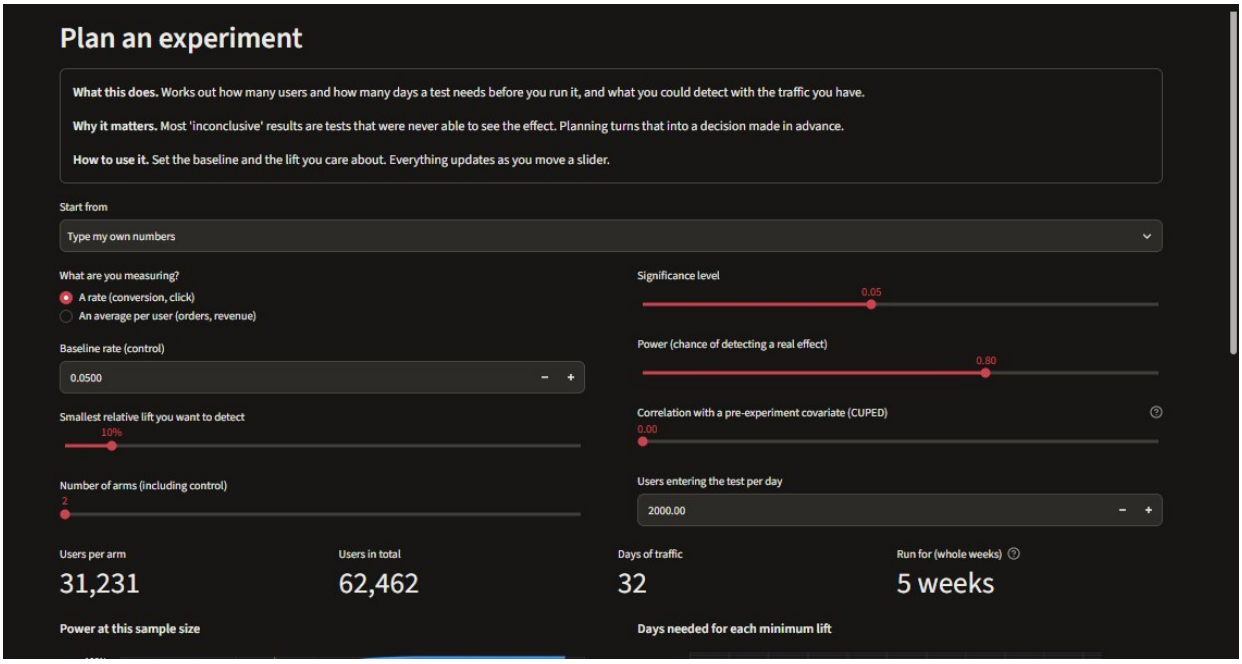
Worked example from the screenshots (Hillstrom, conversion)

Arm	Result	Why
Men's e-mail	**SHIP.** Lift +118.5% (95% interval +71.9% to +165.0%), Holm-adjusted p = 3.2e-13; profit +USD 0.05 per user (interval 0.00 to 0.11).	Clearly significant; the lift is above the 35.7% detectable lift; at USD 30 per conversion and USD 0.15 per e-mail it pays.
Women's e-mail	**ITERATE.** Lift +54.3% (interval +19.3% to ...), Holm-adjusted p = 0.00015; profit -USD 0.06 per user (interval -0.11 to -0.01).	The lift is real, but at these economics it is *provably unprofitable*, so the advice is 'do not ship to everyone; target a subset'.

Assumptions
The page's default economics (USD 30 per conversion, USD 0.15 per e-mail) are **assumptions**. Move the cost slider and the decision for Women's e-mail flips back to SHIP at a low enough cost (see chapter 13). That is the point: the decision is the combination of evidence *and* economics.

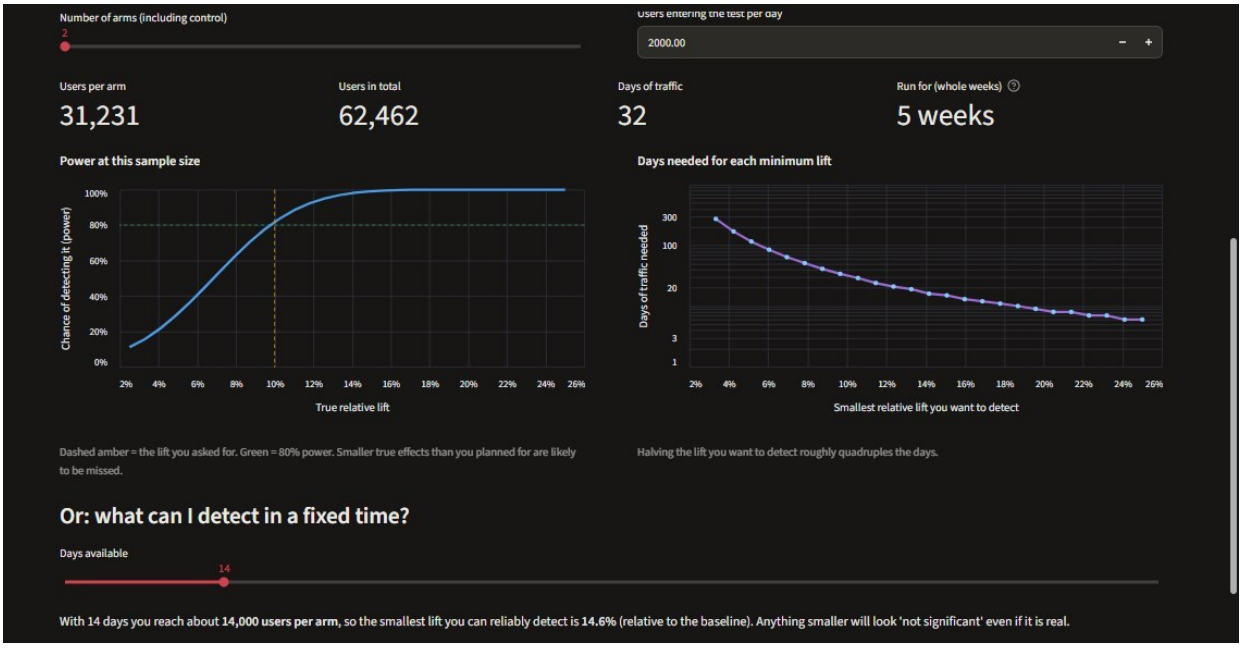
11.3 Plan an experiment (interactive)

At a glance	
Purpose	Before running a test: how many users and days it needs, and what it could detect with the traffic you have.
Output	Users per arm, total users, days of traffic, whole weeks, a power curve, a days-versus-lift curve, and the smallest detectable lift for a fixed number of days.

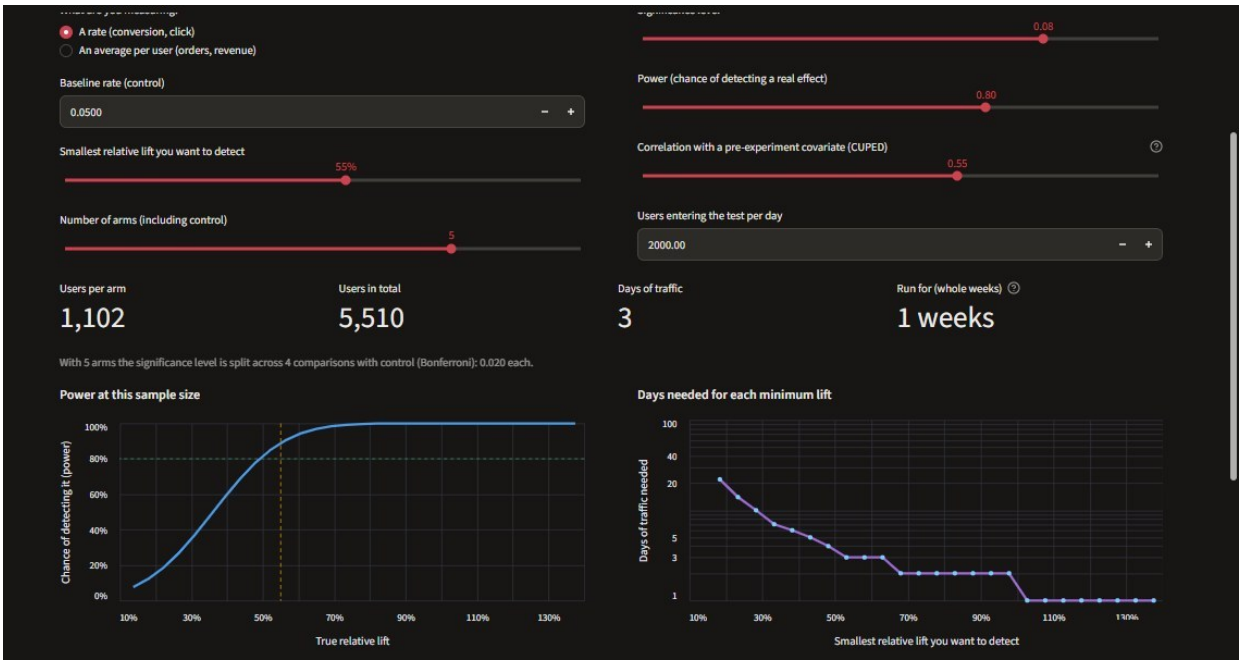


Screenshot 11. Plan an experiment at its defaults: baseline 5%, 10% lift, 2 arms, alpha 0.05, power 0.80, 2,000 users per day gives 31,231 users per arm and 5 weeks.

Control	Meaning	What happens when you change it
Start from	Load a baseline from the Hillstrom control group (conversion, visit, or spend per user) or type your own.	Fills the baseline (and standard deviation for spend) with real numbers.
What are you measuring?	A rate (conversion, click) or an average per user (orders, revenue).	A mean metric also needs a standard deviation, because variance drives the sample size.
Baseline rate / average	What the control group achieves today.	Rarer events (a lower rate) need many more users to see the same relative lift.
Smallest relative lift you want to detect 1% to 100%	The smallest improvement worth being able to see.	Halve it and users needed roughly quadruple. This is the most powerful lever.
Number of arms 2 to 6	Control plus treatments.	More arms split alpha across the comparisons with control (Bonferroni), and need users for every arm: total users and days rise.
Significance level 0.01 to 0.10	Tolerated false-positive rate.	Lower alpha (stricter) needs more users; higher needs fewer.
Power 0.50 to 0.95	Chance of detecting a real effect of the planned size.	Higher power needs more users (90% instead of 80% needs about a third more).
Correlation with a pre-experiment covariate (CUPED) 0 to 0.9	How well past behaviour predicts the metric.	0 means no adjustment; 0.6 cuts the users needed by about 36%.
Users entering the test per day	Your daily traffic.	Turns users into days and weeks. Doubling traffic halves the days.
Days available 1 to 90 (second half of the page)	How long you can run.	Shows the smallest lift you could reliably detect in that time. Anything smaller will look 'not significant' even if real.



Screenshot 12. The two charts: power at this sample size (left) and days needed for each minimum lift (right), then 'what can I detect in a fixed time'.

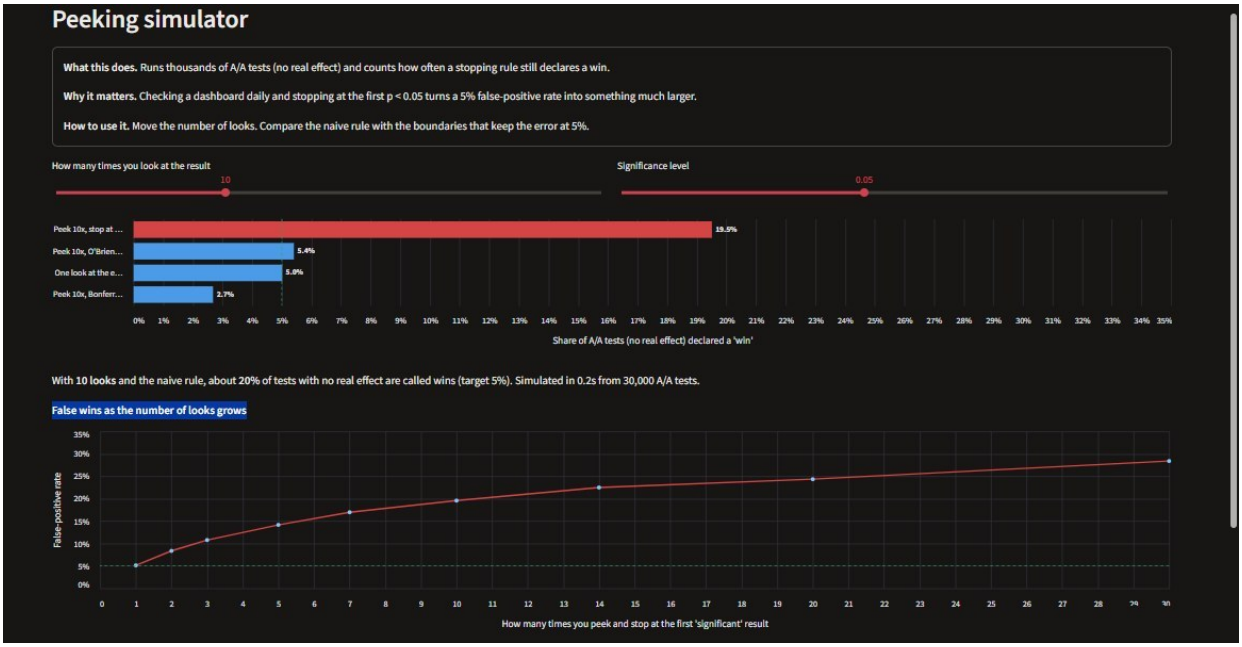


Screenshot 13. The same page with different settings (alpha 0.08, lift 55%, 5 arms, correlation 0.55): 1,102 users per arm, 5,510 in total, 3 days. The page notes that with 5 arms alpha is split across 4 comparisons (0.020 each).

Reading the power chart. The blue curve is power against the *true* lift at the planned sample size. It crosses the green 80% line at the lift you asked for (dashed amber), so effects smaller than planned are likely to be missed. **Reading the days chart.** The curve falls steeply: the last few percentage points of lift cost many days. With 14 days at 2,000 users per day (14,000 per arm) the smallest reliably detectable lift is 14.6% (screenshot).

11.4 Peeking simulator (interactive)

At a glance	
Purpose	Shows, by live simulation, why looking at a test repeatedly and stopping early produces false wins.
Output	A bar chart of false-win rates for four stopping rules and a line chart of false wins against the number of looks.



Screenshot 14. Peeking simulator with 10 looks: the naive rule declares a win in 19.5% of tests that have no real effect.

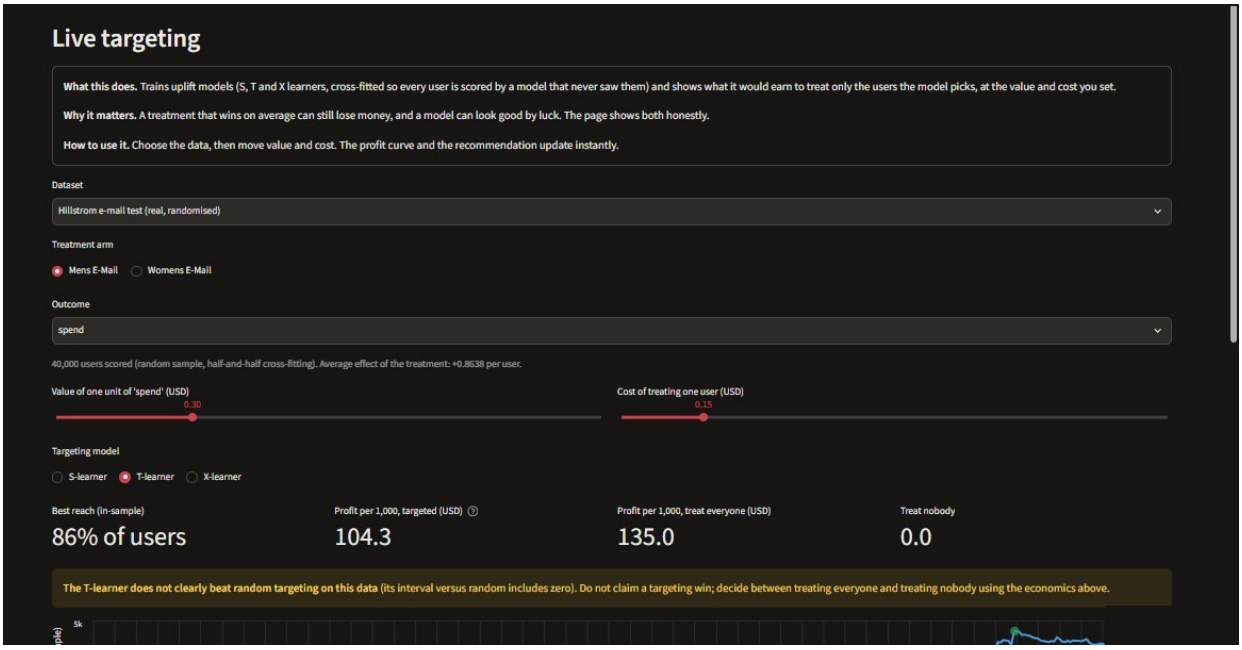
Control	Meaning	What happens when you change it
How many times you look at the result 1 to 30	The number of equally spaced looks.	1 look: 5% false wins (correct). 10 looks: about 19 to 20%. 30 looks: about 28%.
Significance level 0.01 to 0.10	The nominal error rate each rule tries to control.	Moves the target line and every rule's rate with it.

- **Peek N times, stop at first p below alpha (naive)** (red): the false-win rate climbs with every look (19.5% at 10 looks in the screenshot).
- **One look at the end (correct)**: 5.0%.
- **Bonferroni boundary**: 2.7% (safe but over-cautious, so it loses power).
- **O'Brien-Fleming boundary**: 5.4% (strict early, relaxed late; the overall error stays close to 5%).

The simulation uses 30,000 A/A tests and finishes in a fraction of a second. The page's caption lists the fixes: decide the sample size in advance (Plan page), look once, or use a sequential boundary.

11.5 Live targeting (interactive)

At a glance	
Purpose	Trains uplift models on a randomised test and shows what it would earn to treat only the users the model picks, at the value and cost you set.
Data	Hillstrom (Men's or Women's e-mail vs no e-mail), the simulated exp_008 or exp_009, and Criteo if its sample file is present.
Method	S, T and X learners, 4-fold cross-fitting on a random sample of up to 40,000 users; scoring is cached, so moving a slider is instant.



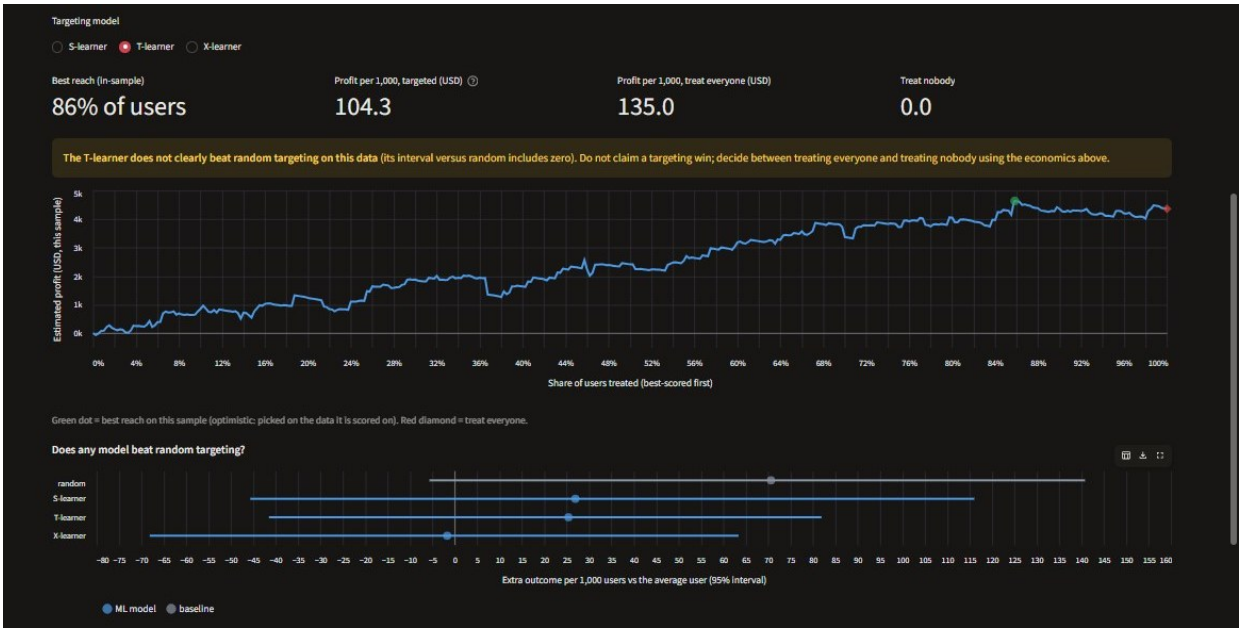
Screenshot 15. Live targeting on Hillstrom (Men's e-mail, outcome spend, T-learner): the headline numbers and the verdict banner.

Control	Meaning	What happens when you change it
Dataset, **Treatment arm** / **Experiment**, **Outcome**	Which test, which treatment, which metric.	Re-trains the models (a few seconds, shown by a spinner) because the question changed.
Value of one unit of the outcome	Money per unit (for Hillstrom spend, the margin on a dollar).	Raising it makes treating more users profitable: the best reach grows and profit rises.
Cost of treating one user	What one e-mail or offer costs.	Raising it makes the best reach shrink; when the cost exceeds what a typical user returns, the best policy is to treat nobody.
Targeting model (S, T, X)	Which learner's scores are used to rank users.	Changes the profit curve and the verdict; the Qini chart below shows all three at once.

What the output says

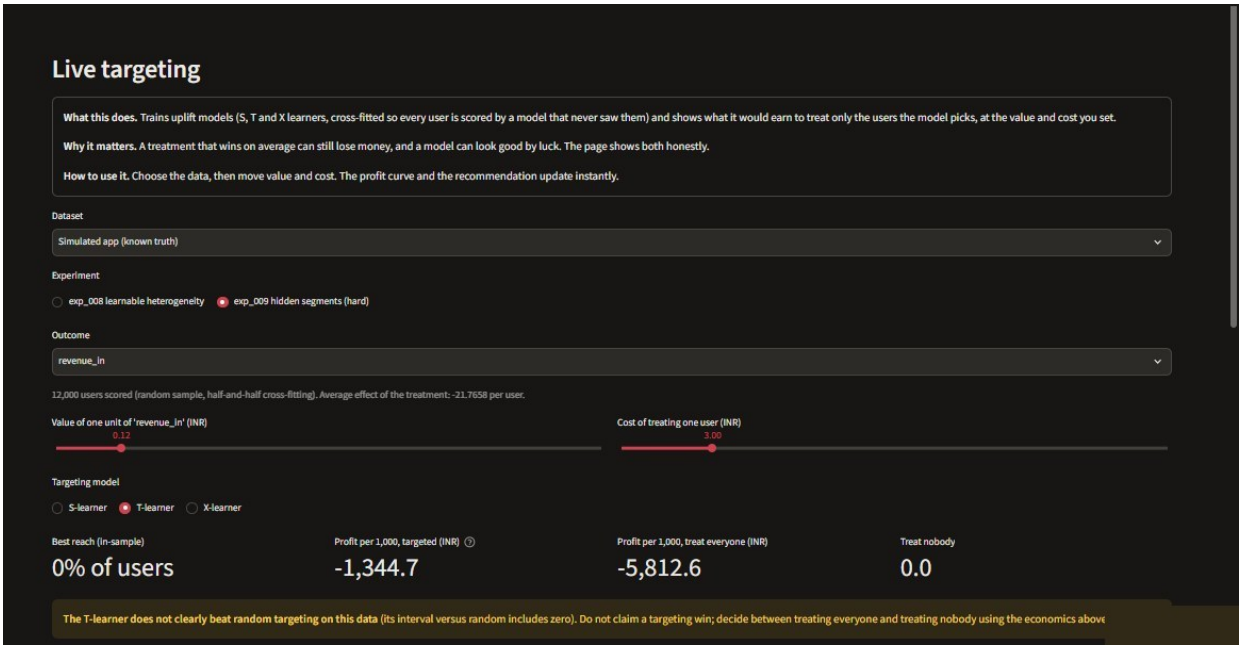
- **Best reach (in-sample)**: the share of users at the peak of the profit curve on this sample (optimistic: picked on the data it is scored on).
- **Profit per 1,000, targeted**: the *honest* figure: the cut-off is chosen on one random half of users and scored on the other half.
- **Profit per 1,000, treat everyone** and **treat nobody** (always 0): the two simple alternatives.

- **A verdict banner in plain language.** Yellow: 'The model does not clearly beat random targeting on this data ... decide between treating everyone and treating nobody.' Green: targeting the top share earns more than treating everyone. Blue: the model ranks users better than random, but at these economics treating everyone is as good.
- **The profit curve** (green dot = best reach, red diamond = treat everyone) and **the Qini chart** with a bootstrap interval for each model and for random.

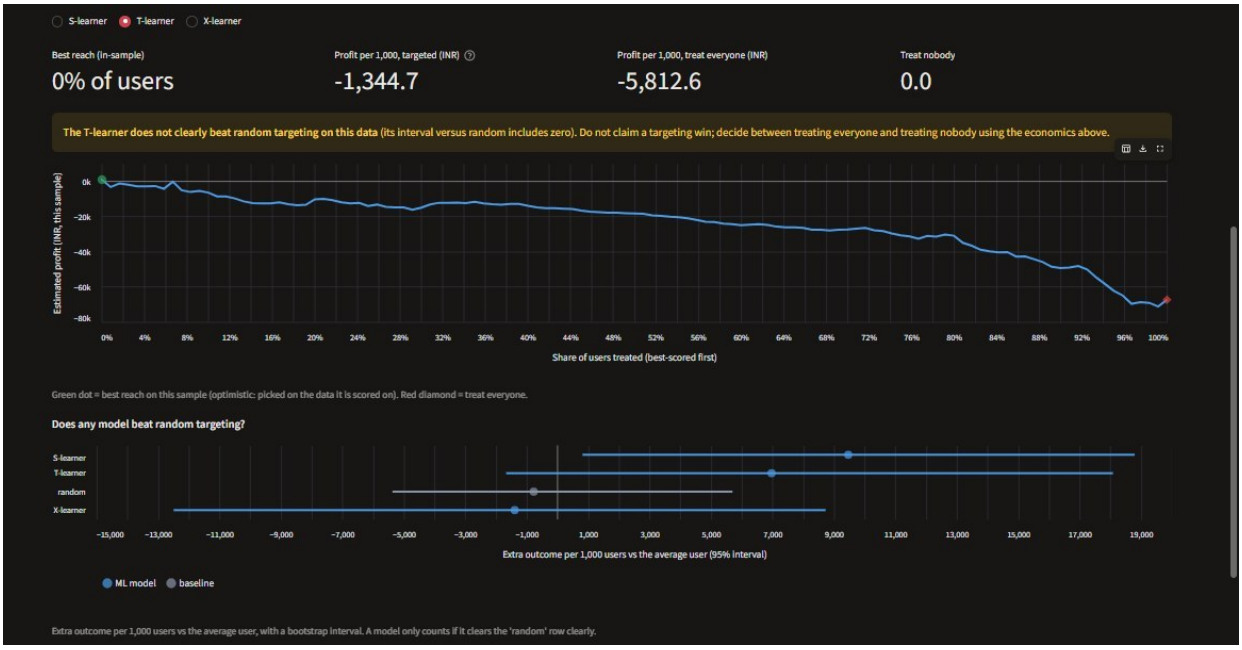


Screenshot 16. Profit curve and the Qini chart for Hillstrom. Every model's interval overlaps random's, so the page warns not to claim a targeting win.

Worked example (Hillstrom, Men's e-mail, spend, T-learner, value 0.30, cost 0.15). Treating everyone earns about USD 135 per 1,000 users and the targeted policy about USD 104, so targeting does not pay on this data, and the banner says the T-learner does not clearly beat random. The profit curve climbs almost steadily to 100% of users: nearly every user is worth e-mailing at this cost.



Screenshot 17. Simulated exp_009 (hidden segments) on revenue: best reach 0% of users; treating everyone loses INR 5,812.6 per 1,000 users and the targeted policy loses less.



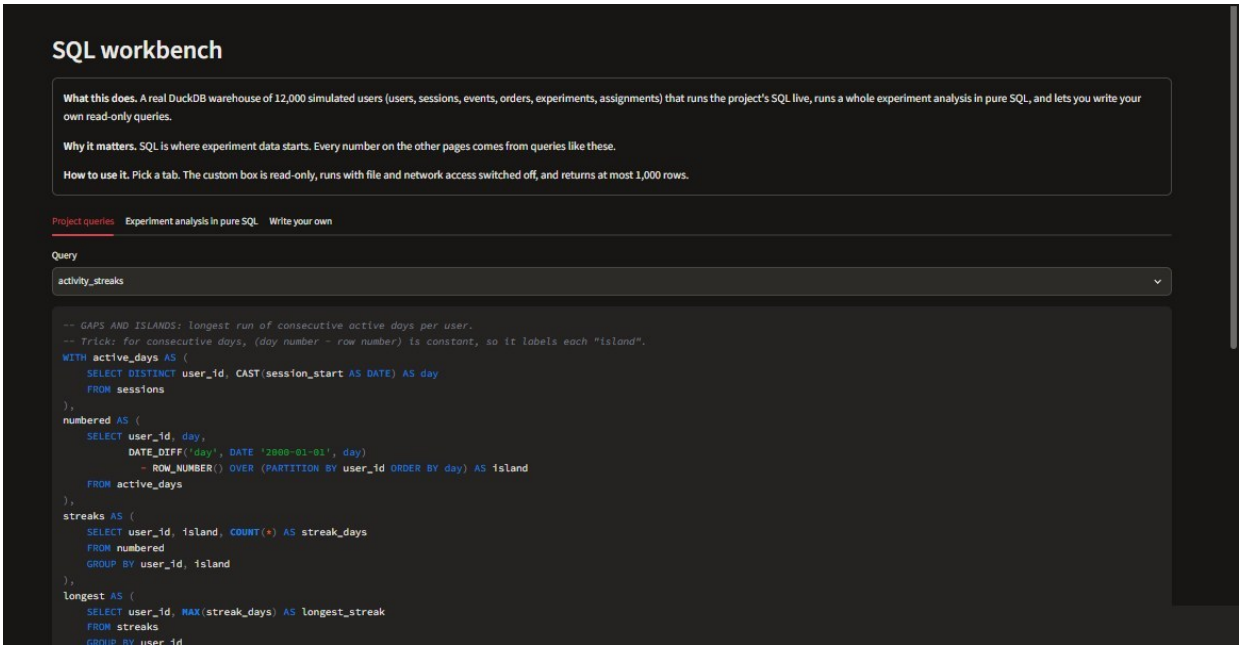
Screenshot 18. exp_009 profit curve falls as more users are treated, and no model clearly clears the random row on the Qini chart.

> **In plain words**
> **Why the 'targeted' figure is not 0 when the best reach is 0%.** 'Best reach' is read from the whole sample's curve; the 'targeted' profit uses a cut-off chosen on one half and scored on the other, so noise can pick a cut-off that is not exactly zero. The page shows both so the optimism and the noise are visible.

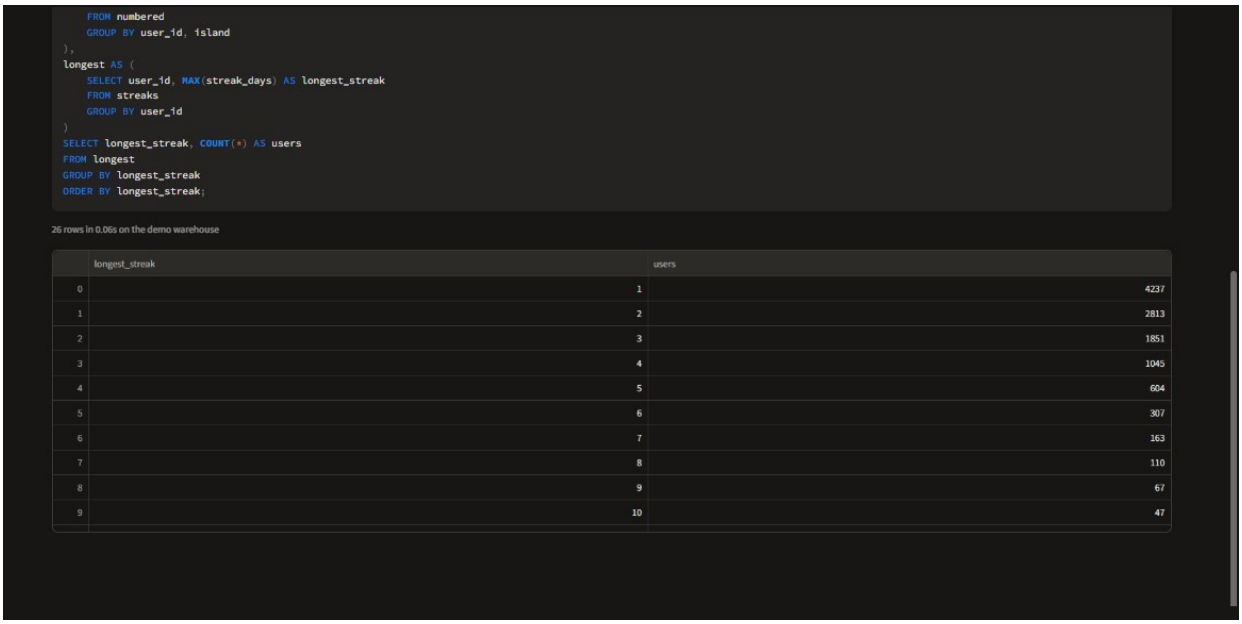
Two simulated tests behave differently on purpose: exp_008's heterogeneity depends on *observable* channel and device, so models can learn it; exp_009's depends on a *hidden* segment, so they cannot. A platform that never says 'no model works here' is not trustworthy.

11.6 SQL workbench (interactive)

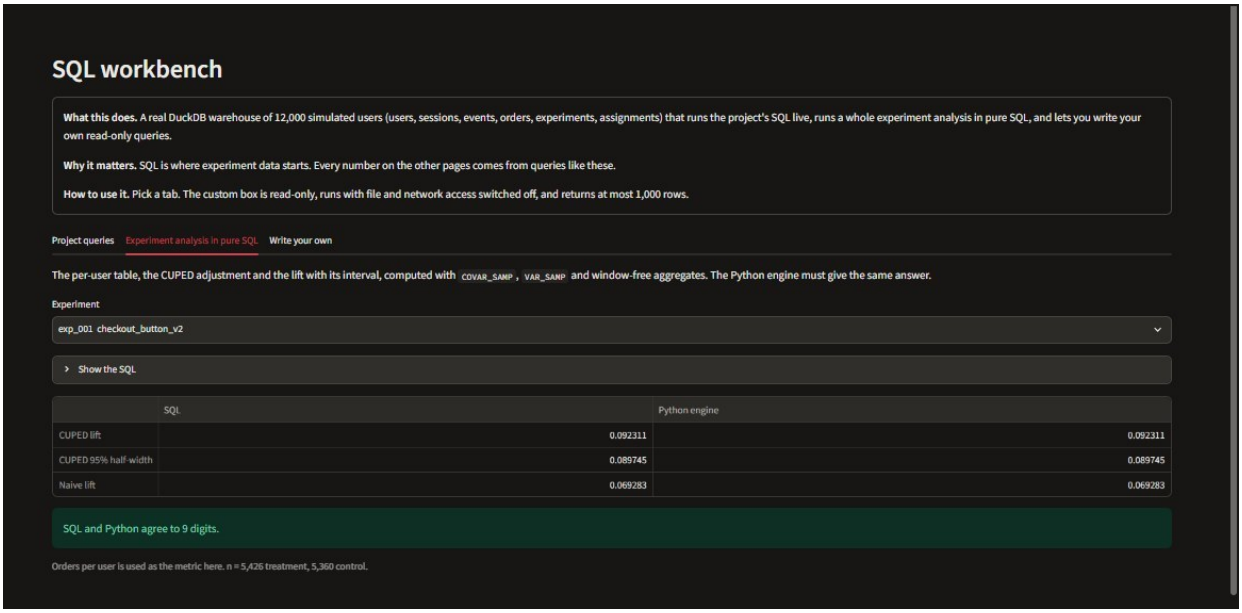
At a glance	
Purpose	A live SQL environment on a 12,000-user simulated warehouse: run the project's queries, run a whole experiment analysis in SQL, or write your own.
Safety	Read-only; file and network access switched off; 1 GB memory cap; at most 1,000 rows returned; only simulated data is inside.



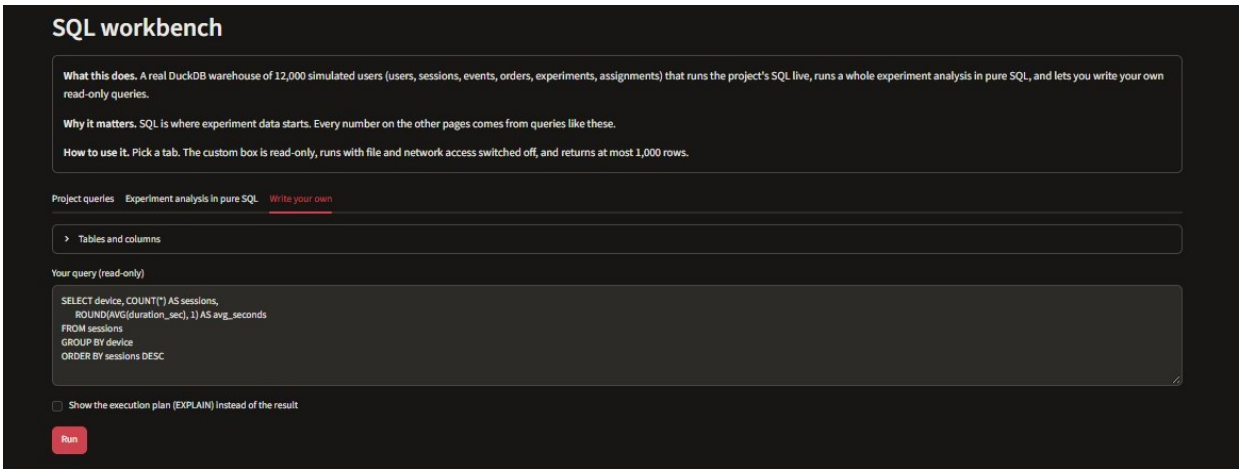
Screenshot 19. Tab 1, 'Project queries': pick a query, see its SQL and its result. Here activity_streaks (gaps and islands).



Screenshot 20. The result of the query: the number of users for each longest-streak length (26 rows in 0.06 seconds on the demo warehouse).



Screenshot 21. Tab 2, 'Experiment analysis in pure SQL': the SQL result and the Python engine agree to 9 digits for exp_001.



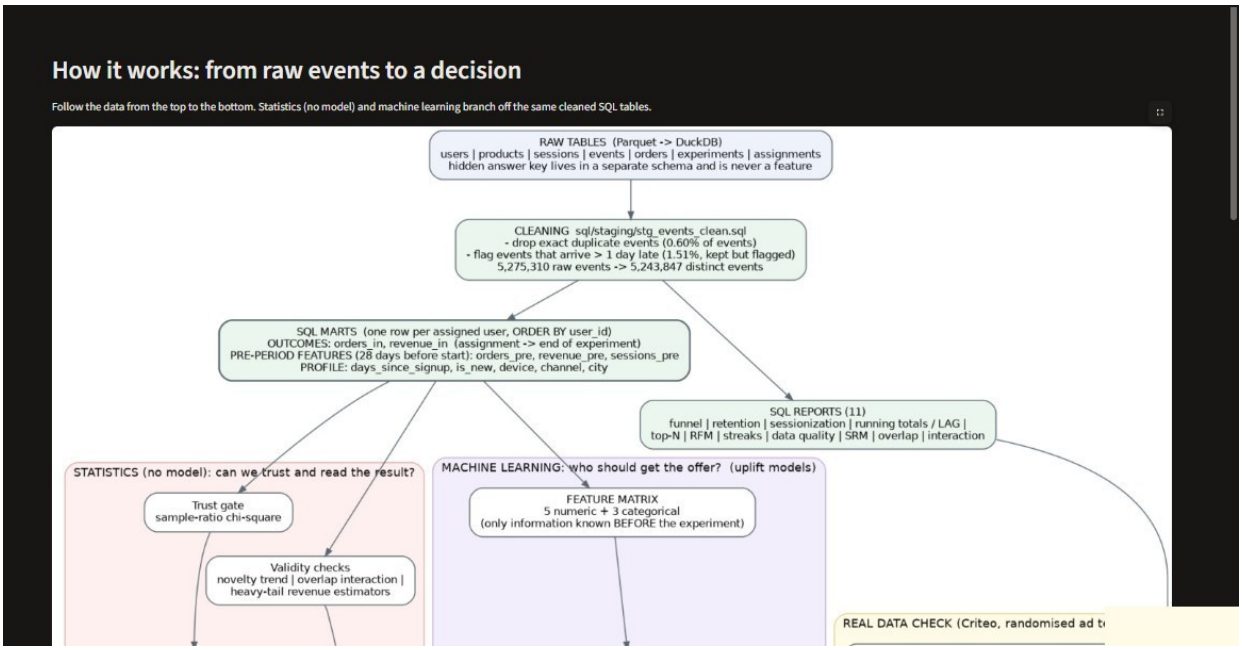
Screenshot 22. Tab 3, 'Write your own': a read-only editor, an EXPLAIN checkbox and a Run button.

Control	What it does
Query (tab 1)	Chooses one of the 11 analysis reports; the page shows its SQL and the result (first 200 rows).
Experiment (tab 2)	Chooses which of the nine experiments the SQL analyses. The table compares SQL and Python for CUPED lift, CUPED 95% half-width and naive lift; a green box confirms agreement to 9 digits.
Your query (tab 3)	A text box. Only one SELECT/WITH/EXPLAIN/DESCRIBE/SHOW statement is accepted; 'DROP TABLE users' is refused with a clear message.
Show the execution plan (EXPLAIN)	Shows how DuckDB would run the query instead of the result: useful to talk about query tuning.
Run	Executes the query (nothing runs until you click).
Tables and columns	An expandable catalogue of every table and its columns.

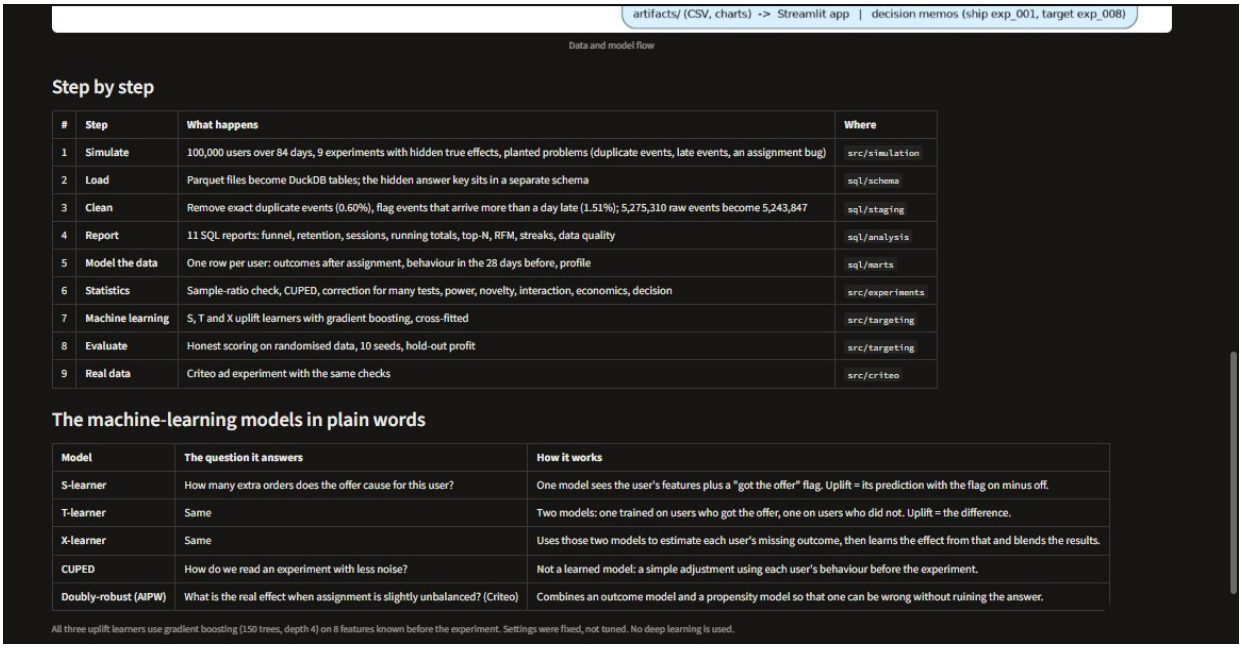
> **Interview tip**
> Interview line: "*I wrote the experiment analysis in SQL as well as Python, and a test checks they agree to nine digits. The public SQL box has a text guard, a locked connection and limits.*"

11.7 How it works

At a glance	
Purpose	The full data-and-model flow as one diagram, a nine-step table and a table of the machine-learning models in plain words.
Controls	None



Screenshot 23. 'How it works: from raw events to a decision': raw tables, cleaning, marts, SQL reports, then the statistics branch (no model) and the machine-learning branch.

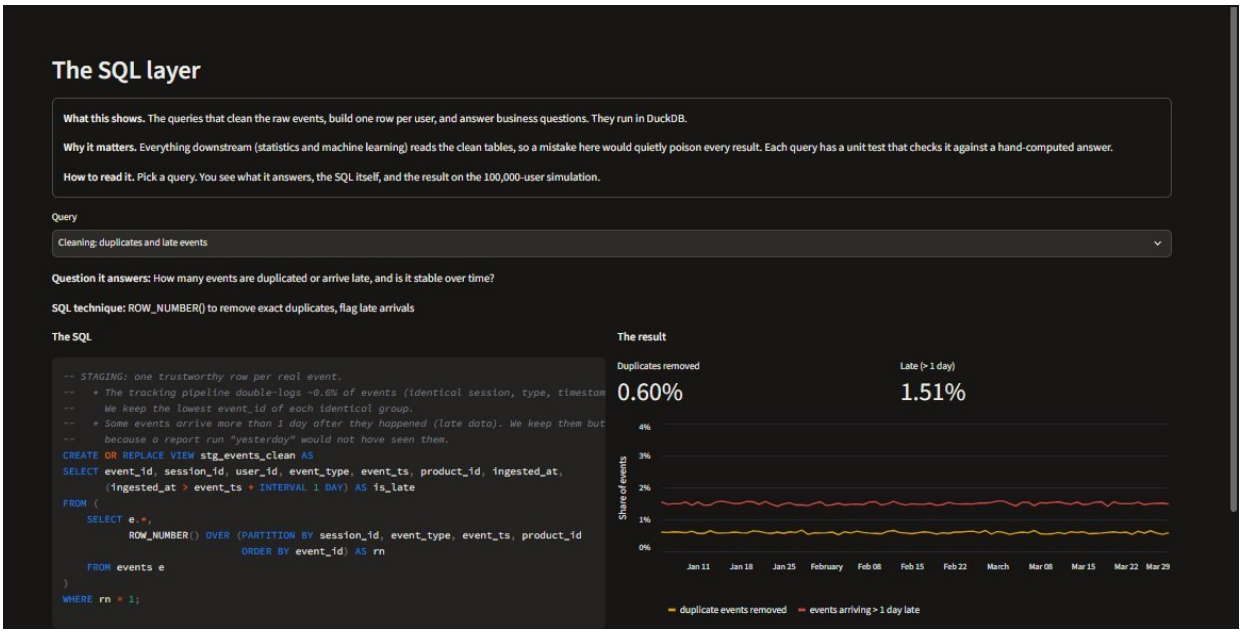


Screenshot 24. The 'Step by step' table and 'The machine-learning models in plain words'.

The page reports the cleaning numbers (0.60% duplicates, 1.51% late, 5,275,310 raw events becoming 5,243,847), the nine steps with the folder where each lives, and a table that explains S-, T- and X-learners, CUPED and the doubly-robust (AIPW) estimator in one sentence each. It also states that all three uplift learners use gradient boosting (150 trees, depth 4) on 8 features known before the experiment, with fixed, untuned settings and no deep learning.

11.8 SQL layer

At a glance	
Purpose	Browse the original eleven SQL reports on the full 100,000-user simulation: what each answers, the SQL technique, the SQL itself and a chart of its result.
Control	**Query** drop-down: one entry per report (plus the cleaning view and the per-user table).
Reads from	sql/*.sql and the result CSVs in artifacts/sql/

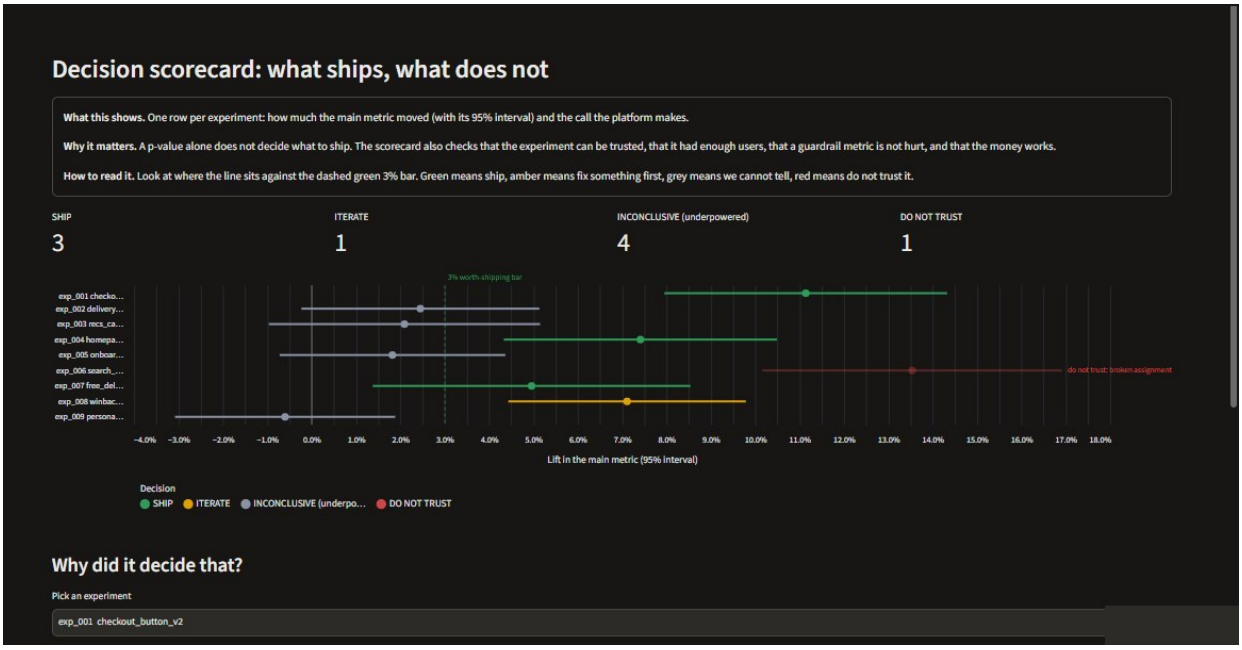


Screenshot 25. The SQL layer showing the cleaning view: the question it answers, the technique (ROW_NUMBER to remove duplicates, flag late arrivals), the SQL and a chart of the duplicate and late rates over time.

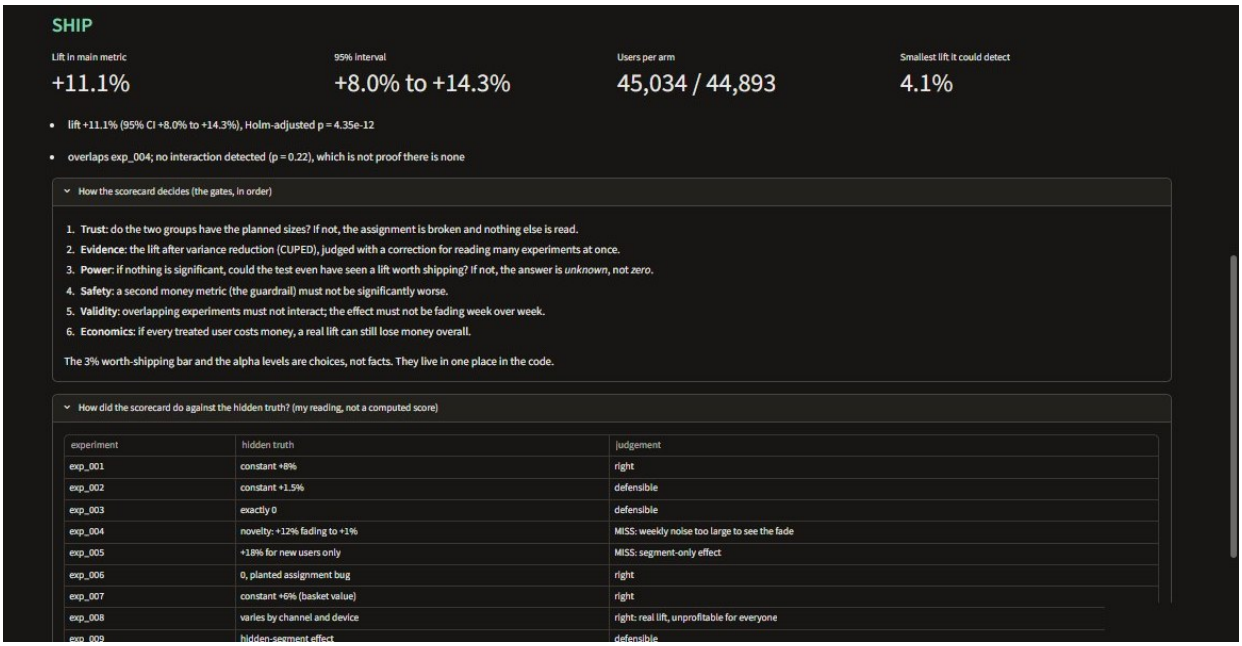
This page differs from the SQL workbench: here the results are **precomputed on the full 100,000-user data**; the workbench runs queries **live** on a smaller 12,000-user warehouse. The cleaning chart shows both rates flat over time (about 0.6% duplicates and 1.5% late), which is what a healthy monitor with a constant planted fault looks like. Each query also has the metric cards for its key numbers (here, Duplicates removed 0.60% and Late (more than 1 day) 1.51%).

11.9 Decision scorecard

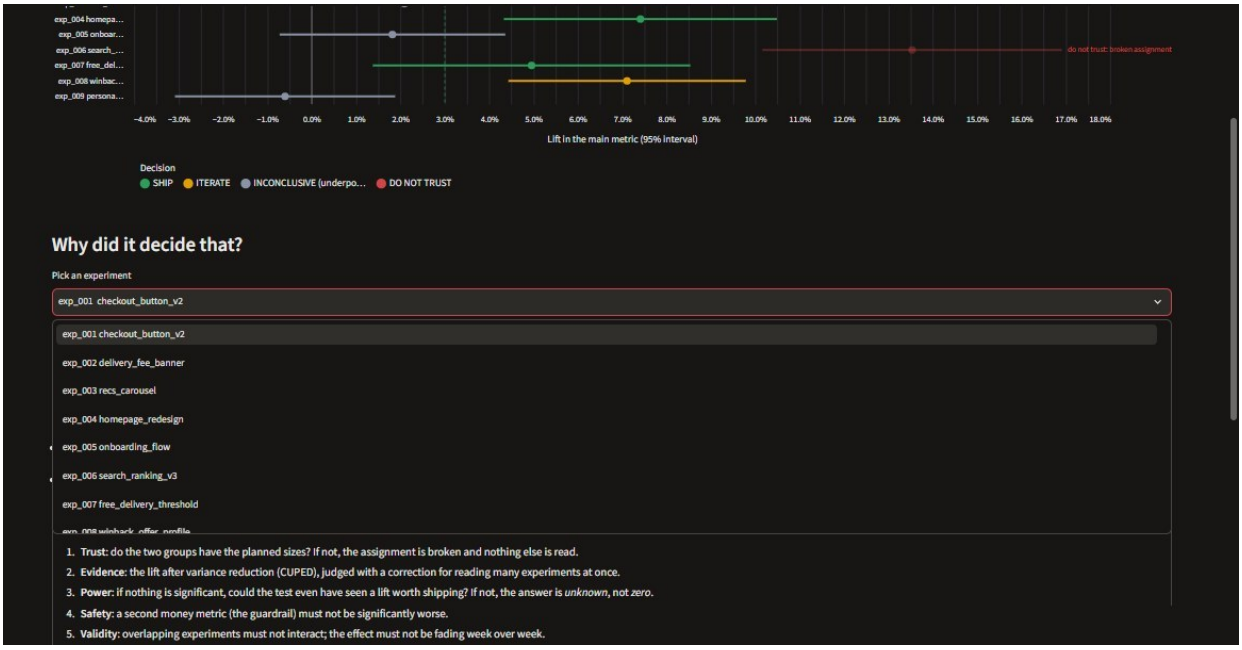
At a glance	
Purpose	One row per experiment: how much the main metric moved (with its 95% interval) and the decision the platform makes.
Control	**Pick an experiment** drop-down in 'Why did it decide that?'.



Screenshot 26. The scorecard: counts per decision (SHIP 3, ITERATE 1, INCONCLUSIVE 4, DO NOT TRUST 1) and the interval chart for all nine experiments.



Screenshot 27. 'Why did it decide that?' for exp_001: SHIP, lift +11.1% (+8.0% to +14.3%), 45,034 vs 44,893 users, detectable lift 4.1%; the gates; and the comparison with the hidden truth.

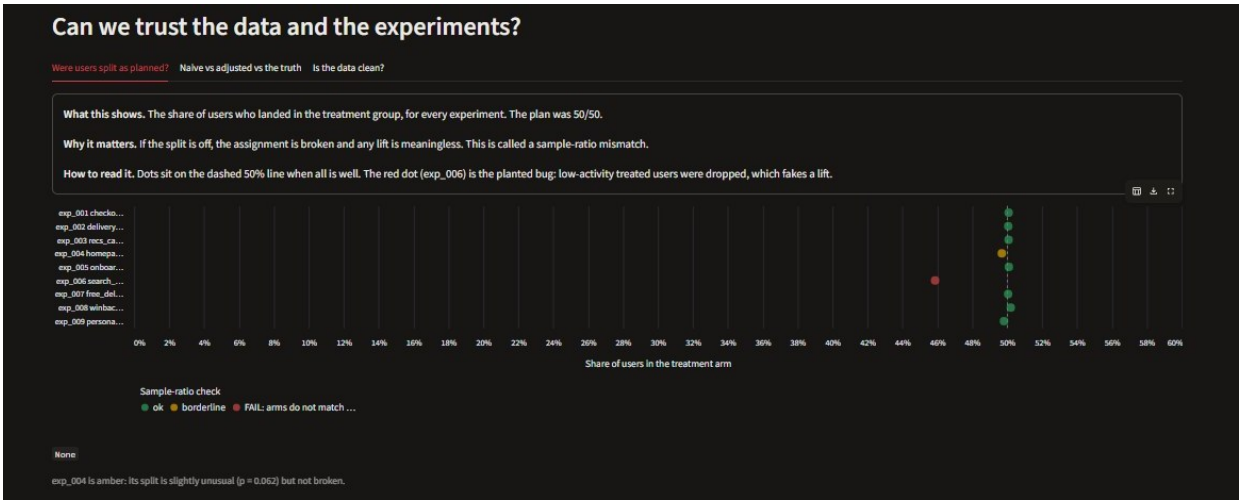


Screenshot 28. The experiment selector lists all nine experiments with their names.

Reading the counts. SHIP 3 (exp_001, exp_004, exp_007), ITERATE 1 (exp_008), INCONCLUSIVE 4 (exp_002, 003, 005, 009), DO NOT TRUST 1 (exp_006). Every decision with the exact reasons appears when you pick the experiment: for example, exp_001 ships but notes that it *overlaps exp_004 with no interaction detected (p = 0.22), which is not proof there is none*. The expandable **How the scorecard decides** lists the six gates, and **How did the scorecard do against the hidden truth?** is the table in section 7.12, including the two honest misses.

11.10 Trust checks

At a glance	
Purpose	Three checks on the data and the experiments.
Tabs	Were users split as planned? / Naive vs adjusted vs the truth / Is the data clean?



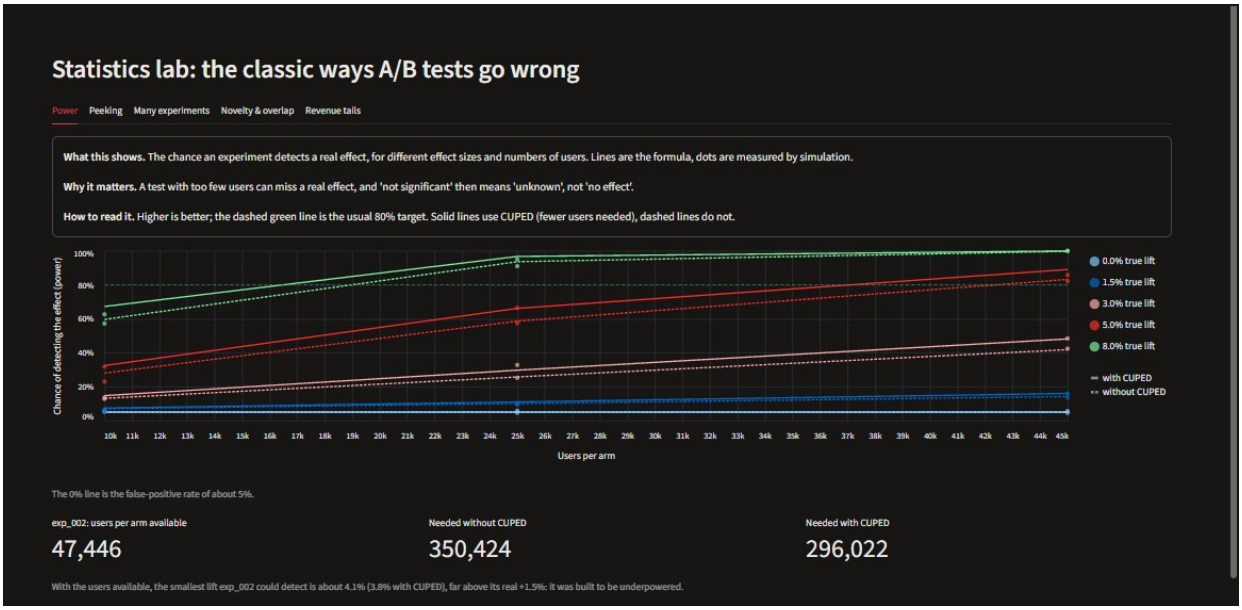
Screenshot 29. Tab 1: the share of users in the treatment arm for every experiment. The planned share is 50% (dashed line); exp_006 (red) is the planted bug, exp_004 is amber (p = 0.062).

- **Were users split as planned?** Green dots sit on the 50% line. Amber = borderline sample-ratio p (below 0.10). Red = failed (below 0.001). exp_006 sits near 46%.

- **Naive vs adjusted vs the truth** (no screenshot): for every experiment, the naive estimate, the variance-reduced (CUPED) estimate and the true built-in effect (green diamond). CUPED gives the same answers with tighter intervals; for exp_006 the truth is 0 yet both estimates say +13% to +19%, because CUPED cannot repair a broken experiment. exp_004 and exp_005 are special cases (an effect that fades; an effect only for new users).
- **Is the data clean?** (no screenshot): the duplicate rate and late-event rate by day, found with the data-quality SQL; both lines should be flat.

11.11 Statistics lab

At a glance	
Purpose	The classic ways A/B tests go wrong, each shown as a chart computed by simulation.
Tabs	Power / Peeking / Many experiments / Novelty and overlap / Revenue tails

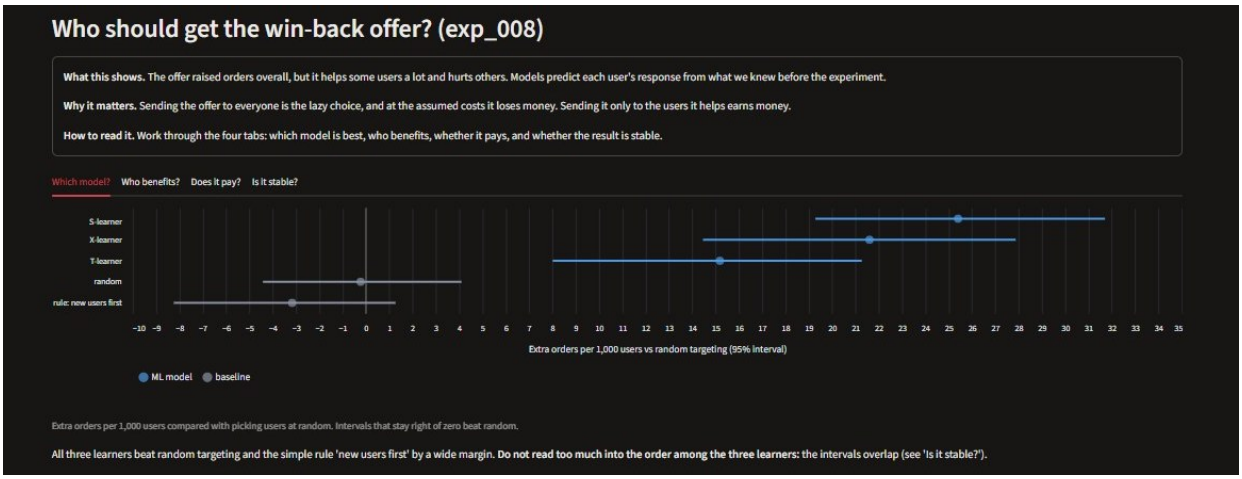


Screenshot 30. Tab 'Power': power against users per arm for five true lifts (0%, 1.5%, 3%, 5%, 8%); solid lines use CUPED, dashed lines do not; dots are measured by simulation.

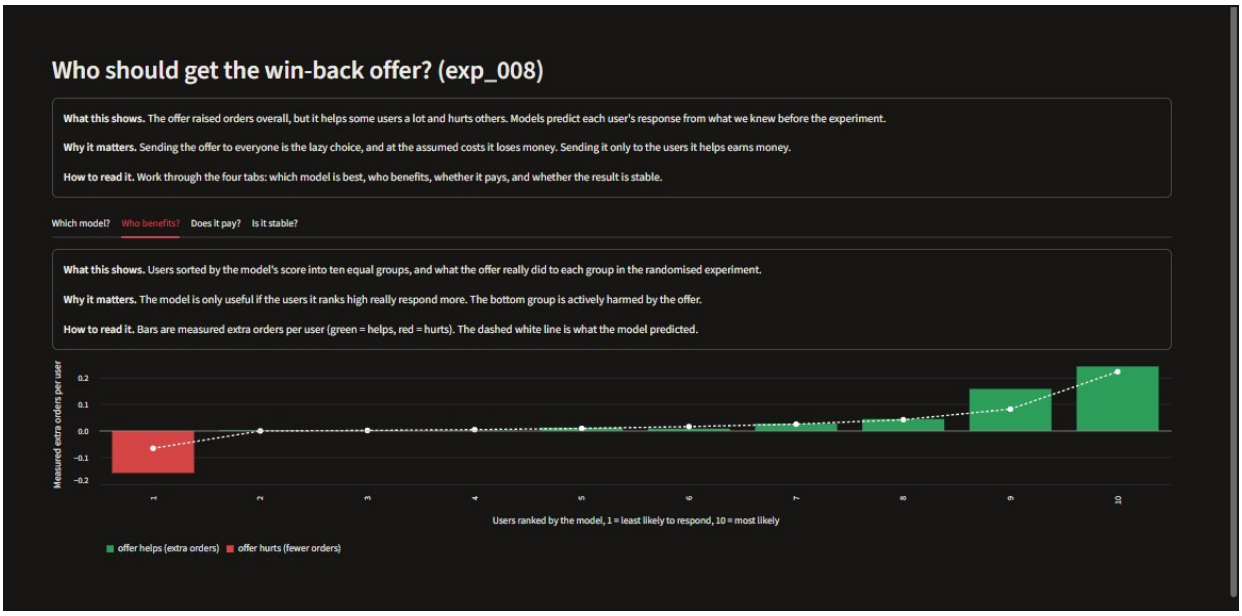
Tab	What it shows	Key reading
Power	Chance of detecting a real effect against sample size, formula (lines) vs simulation (dots), with and without CUPED. Below: exp_002 needs 350,424 users per arm without CUPED, 296,022 with it, but has 47,446.	The 0% line is the false-positive rate, about 5%. A small effect needs a huge sample.
Peeking	The same peeking study as the interactive page, on the real 100,000-user warehouse (28 daily looks).	Peeking daily gives about 25% false wins; Bonferroni and O'Brien-Fleming bring it back near 5% or below.
Many experiments	Chance of at least one false win among m null experiments: measured (red), the formula 1 - 0.95^m (pink) and with Holm (green). A table shows which experiments survive each correction.	20 harmless tests: about 6 in 10 chance of a false winner without correction.
Novelty and overlap	Left: exp_004's lift by exposure week. Right: the exp_001 x exp_004 interaction.	Week 3 is lower but the intervals overlap and the trend test cannot confirm a fade (p = 0.19). Interaction +0.0126, p = 0.22: none detected, not proof of none.
Revenue tails	Four estimators of exp_007's revenue lift against the true +6%, and how concentrated revenue is: 74.7% of users spent nothing, the top 1% hold 20.9% of revenue and the top 10% hold 76.1%.	All four agree; winsorising measures a slightly different quantity.

11.12 Who gets the offer? (exp_008)

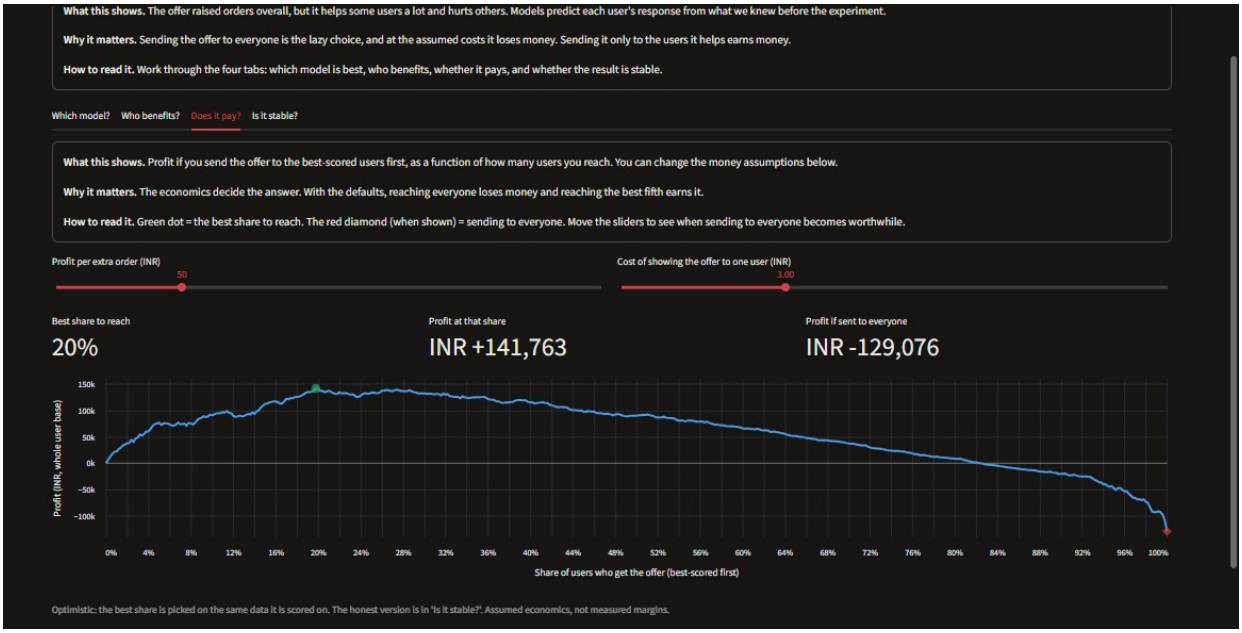
At a glance	
Purpose	The full uplift story for the win-back offer on the 100,000-user simulation, in four tabs.
Controls	Two sliders in the 'Does it pay?' tab: **Profit per extra order (INR)** 20 to 150, default 50; **Cost of showing the offer to one user (INR)** 0 to 10, default 3.



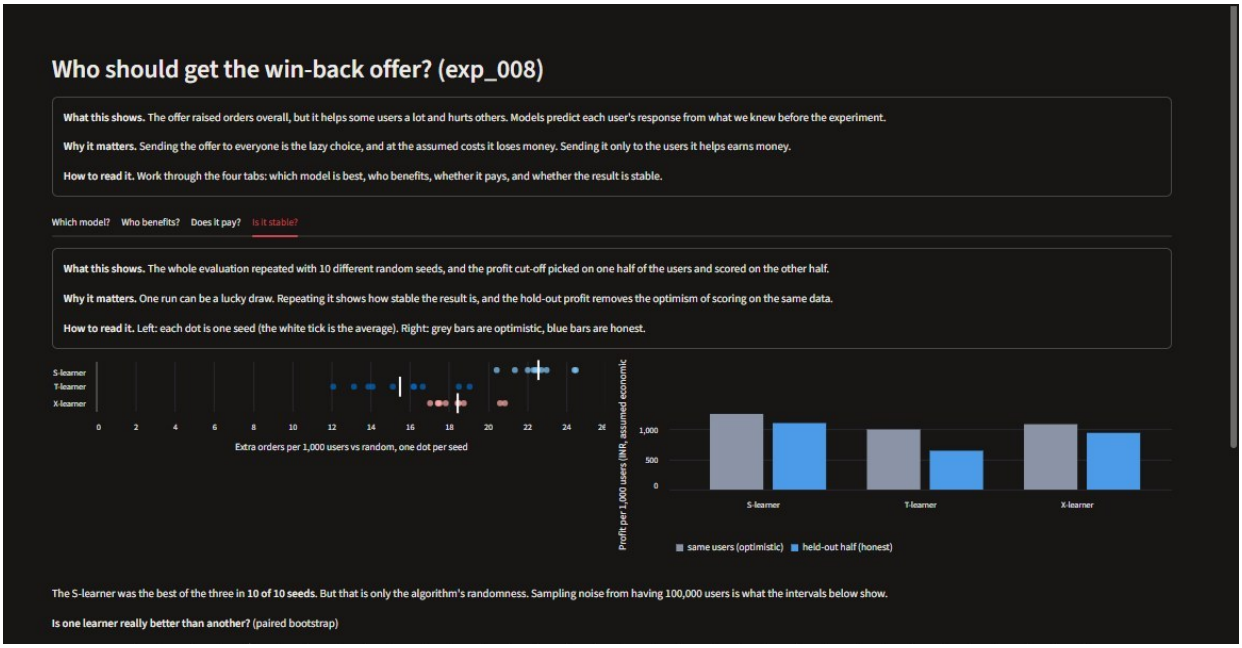
Screenshot 31. Tab 'Which model?': extra orders per 1,000 users compared with random targeting, with 95% intervals. All three learners beat random and the simple rule 'new users first'.



Screenshot 32. Tab 'Who benefits?': users ranked into ten equal groups by the S-learner's score; bars are the measured effect of the offer (green helps, red hurts), the white line is what the model predicted.



Screenshot 33. Tab 'Does it pay?' at INR 50 per extra order and INR 3 per user: best reach 20% earns INR +141,763; sending to everyone loses INR 129,076.

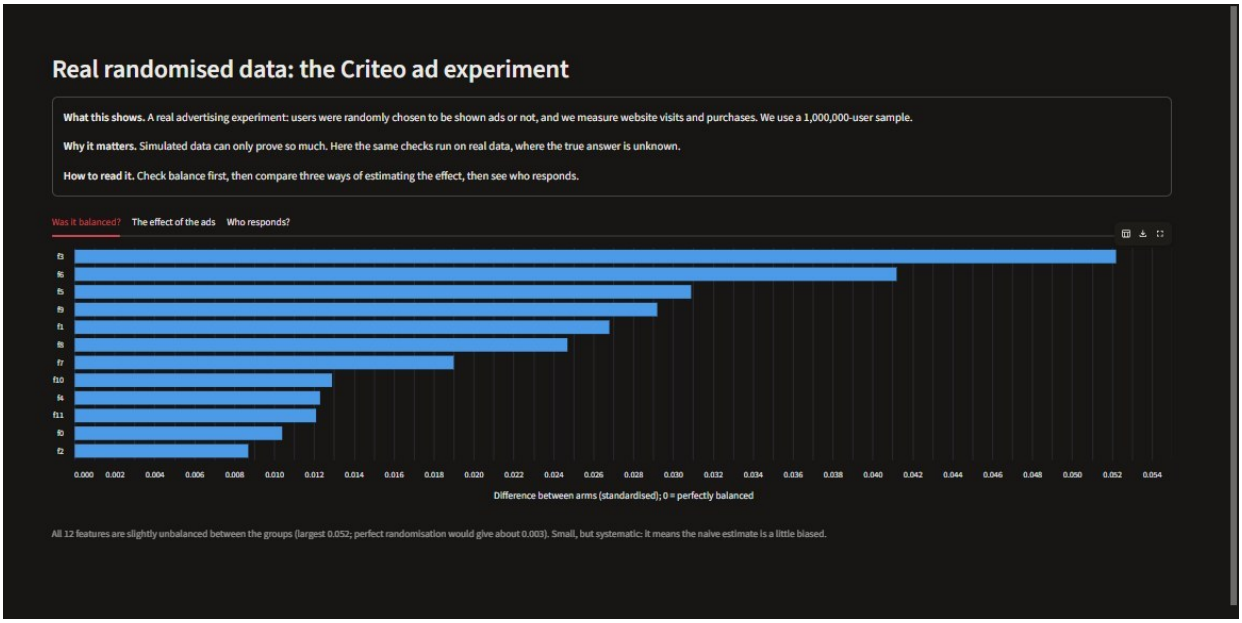


Screenshot 34. Tab 'Is it stable?': ten seeds (left; each dot is one seed) and optimistic (grey) vs honest hold-out (blue) profit (right).

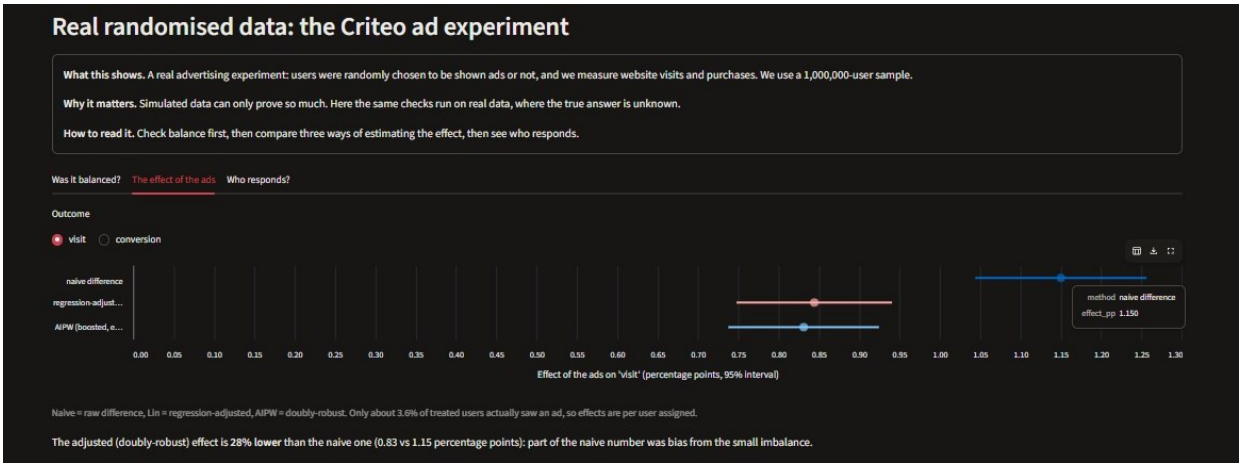
Tab	What to take from it
Which model?	S, X and T learners all beat random targeting and the 'new users first' rule by a wide margin; do not read too much into their order because the intervals overlap.
Who benefits?	The bottom group (decile 1) is *harmed* by the offer (about -0.16 orders per user); deciles 2 to 7 are near zero; the top group gains about +0.24. The model's predicted values follow the measured ones, rising at the top.
Does it pay?	With the defaults the best reach is about 20% of users and profit about INR 141,763 (on the whole 100,000-user base); everyone loses about INR 129,076. Raise 'Profit per extra order' past about INR 88 or lower the cost under about INR 1.71 and sending to everyone becomes defensible. The page notes that the best share is picked on the data it is scored on, so it is optimistic.
Is it stable?	The S-learner was best in 10 of 10 seeds (algorithm randomness only). The paired bootstrap shows S beats T clearly, S vs X is marginal, X vs T cannot be separated. Hold-out (blue) profit is lower than same-data (grey) profit: that gap is the optimism removed.

11.13 Real data (Criteo)

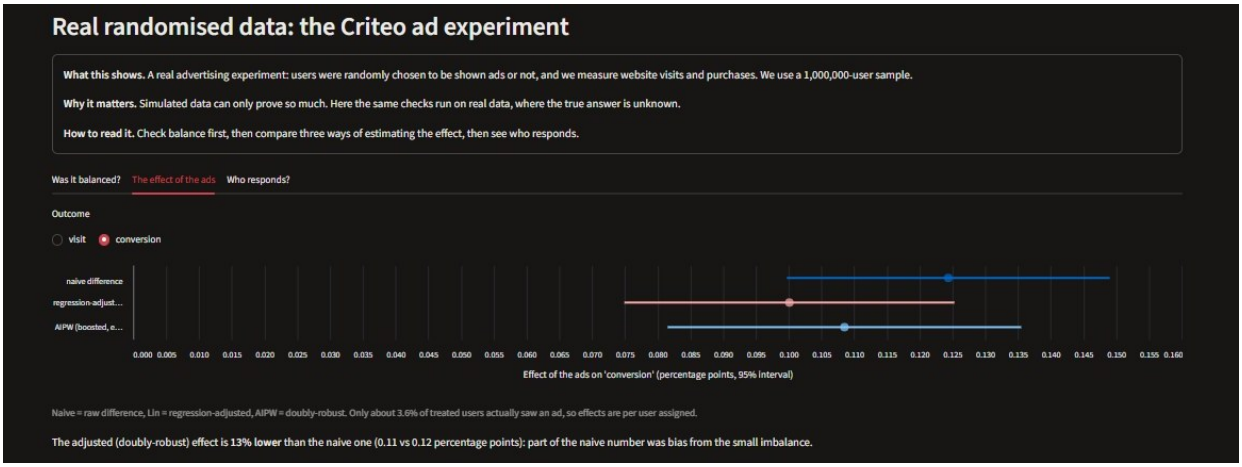
At a glance	
Purpose	The same checks on a real randomised advertising experiment (a 1,000,000-user sample).
Controls	**Outcome** radio (visit or conversion) in the 'effect of the ads' tab.
Tabs	Was it balanced? / The effect of the ads / Who responds?



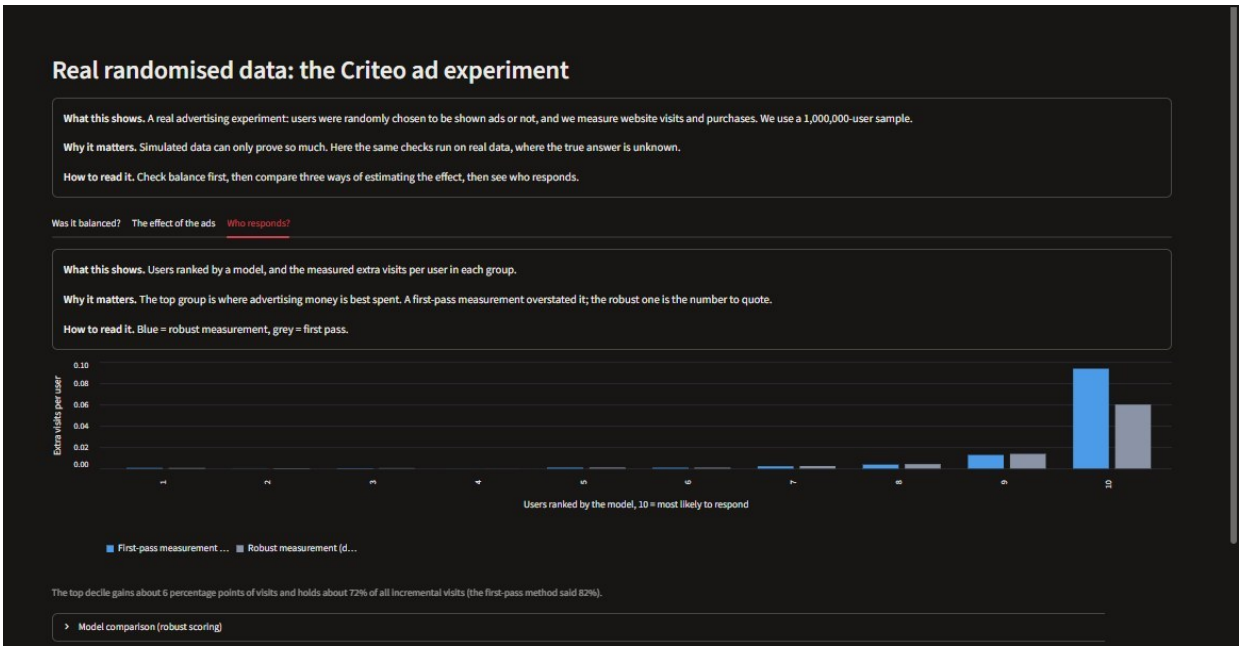
Screenshot 35. Tab 'Was it balanced?': standardised difference between the arms for each of 12 features; perfect randomisation would give about 0.003 and the largest here is 0.052.



Screenshot 36. Tab 'The effect of the ads', outcome visit: naive +1.15 percentage points, regression-adjusted and doubly-robust about +0.84 and +0.83 (28% lower than naive).



Screenshot 37. Outcome conversion: naive 0.12 vs adjusted 0.11 percentage points (13% lower).



Screenshot 38. Tab 'Who responds?': the measured extra visits per user in each predicted decile; the top decile gains about 6 percentage points and holds about 72% of incremental visits (the first-pass number was 82%).

Why it matters. The lesson is the *direction* of the correction: the naive effect was inflated by a small, systematic imbalance in the randomisation, and the doubly-robust estimate is the one to quote. The page also states that only about 3.6% of treated users saw an ad, so effects are per user assigned, not per ad seen.

11.14 Platform validation

At a glance	
Purpose	Checks the platform's own statistics by repeating the whole analysis in 100 freshly simulated worlds of 20,000 users each.
Controls	**Experiment** drop-down in 'One experiment, 100 simulated worlds' (lower on the page): shows the 100 individual estimates for one experiment.

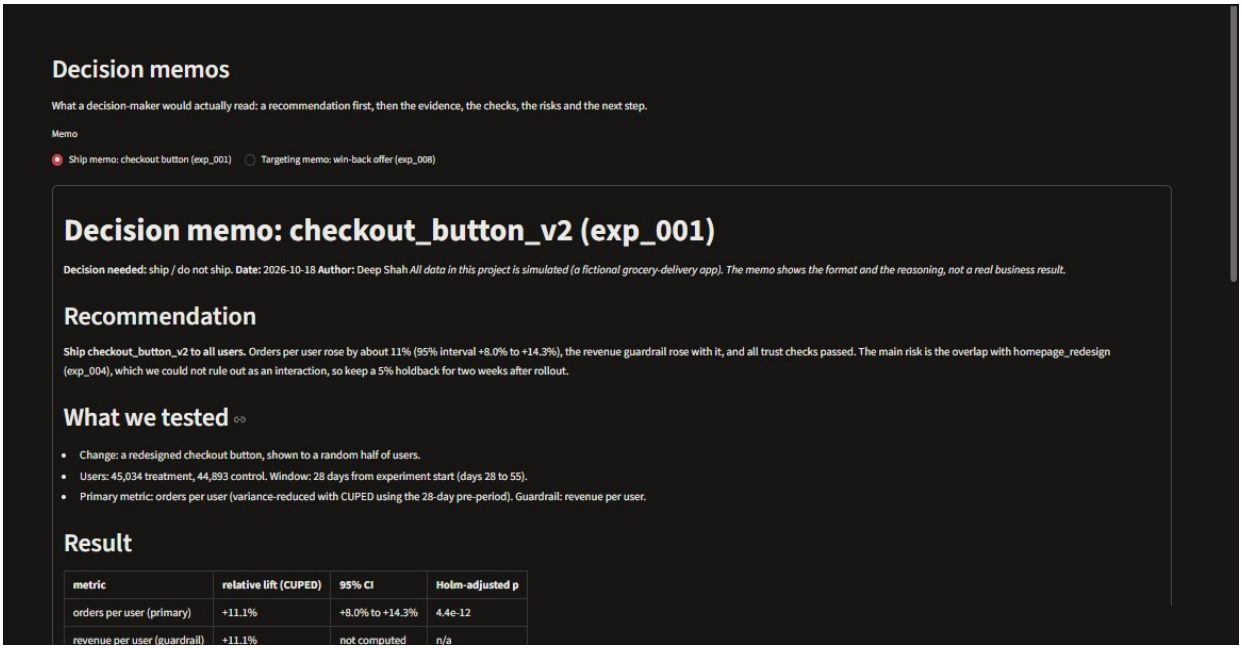


Screenshot 39. Platform validation: clean experiments unbiased (Yes), interval coverage 94% to 100%, planted bug flagged (Yes). Red = the planted-bug experiment exp_006.

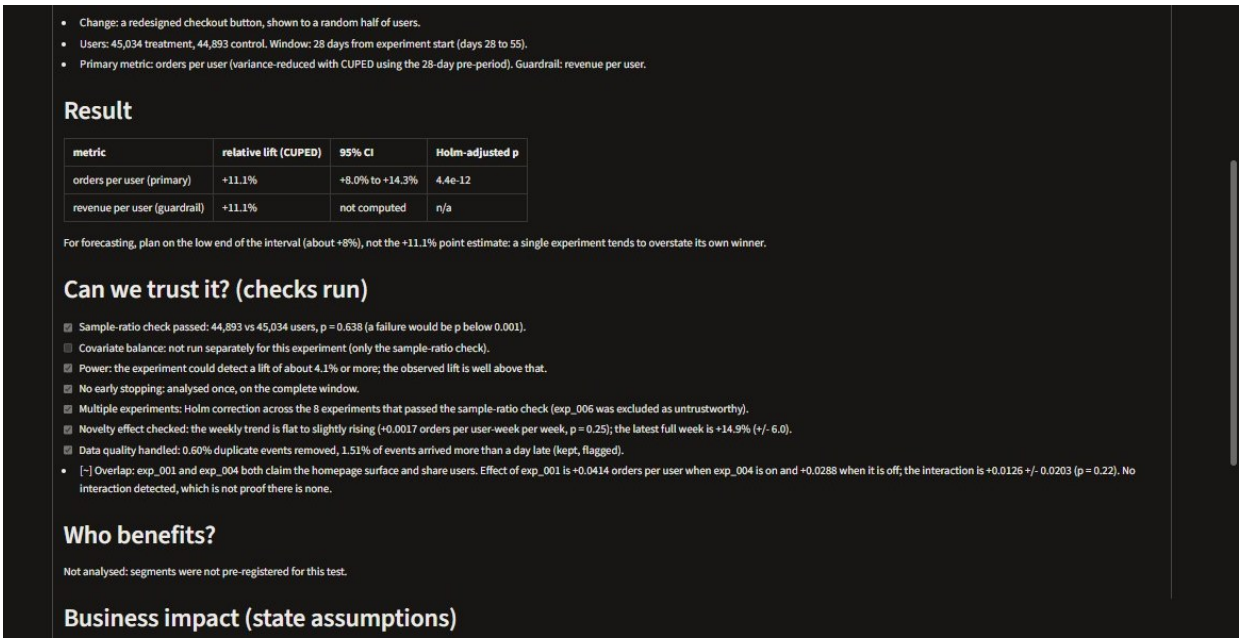
- **Do the 95% intervals contain the truth?** Each dot is the share of worlds where the interval covered the true effect; the green line is the 95% target. exp_006 (red) is far below: its intervals almost never contain the truth.
- **Is the average estimate correct?** The average error with plus or minus 2 standard errors; lines that cross the zero line are unbiased. exp_006 is wrong by roughly +13% and +19% (CUPED and naive).
- **Planted bug flagged?** The page counts it as flagged when its coverage is below 50%.

11.15 Decision memos

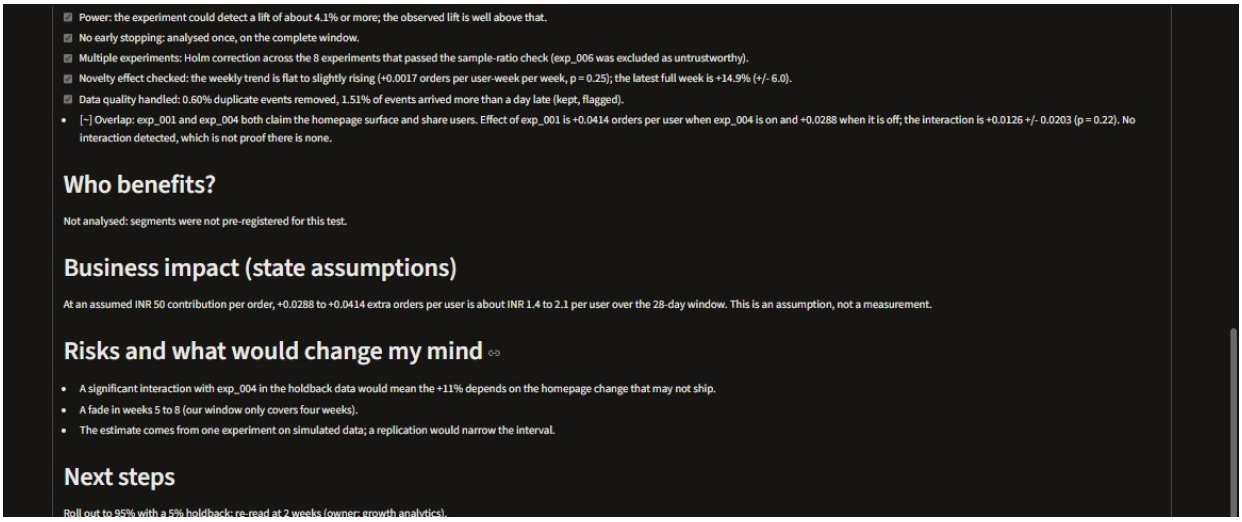
At a glance	
Purpose	What a decision-maker would actually read: the recommendation first, then evidence, checks, risks and next step.
Control	**Memo** radio: 'Ship memo: checkout button (exp_001)' or 'Targeting memo: win-back offer (exp_008)'.



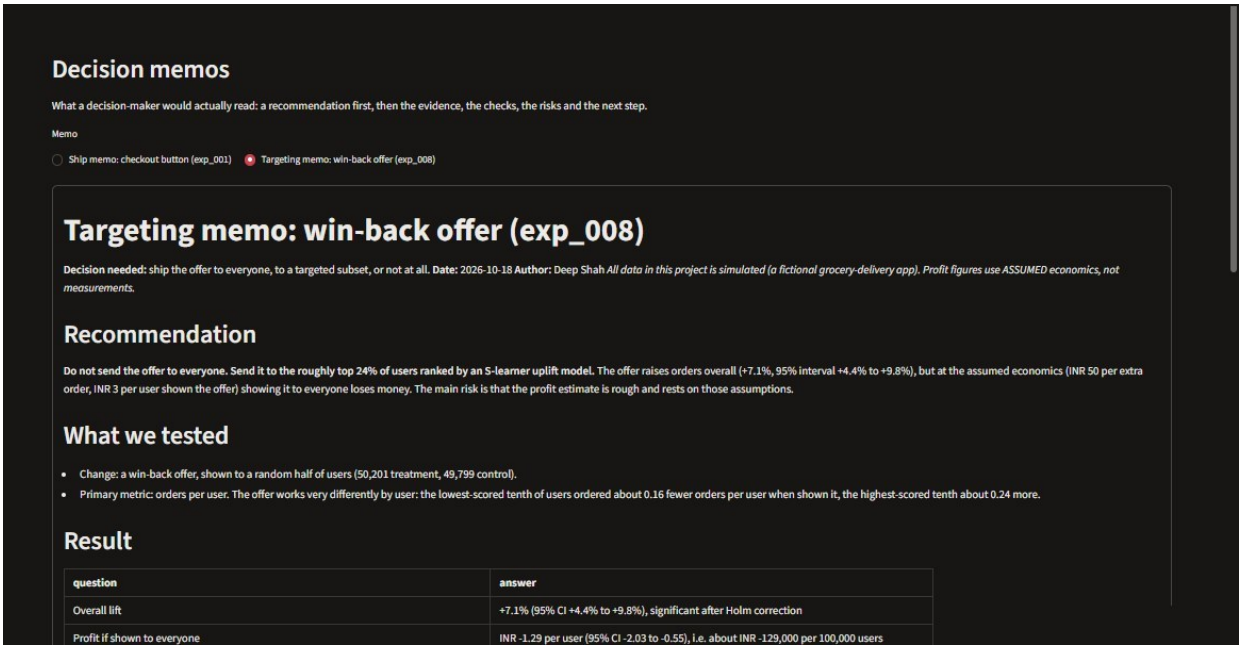
Screenshot 40. The ship memo for exp_001: decision needed, recommendation, what we tested and the result table.



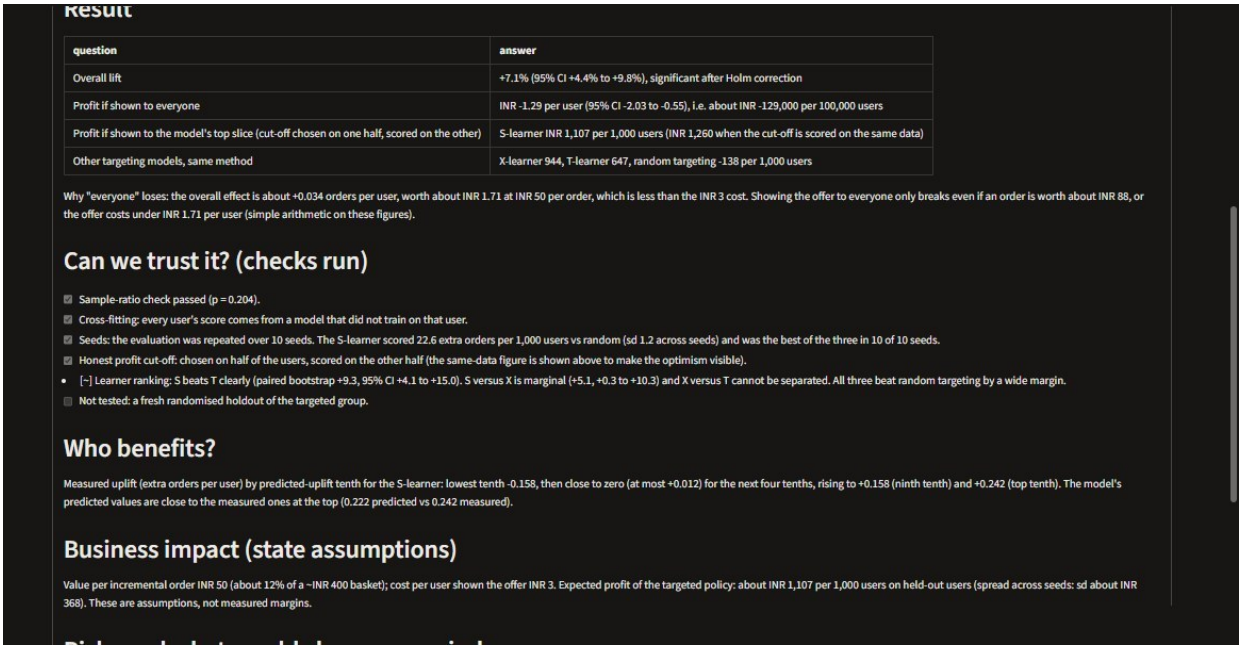
Screenshot 41. Can we trust it? A checklist of the checks that were run (and the one not run: covariate balance), then who benefits and the business impact.



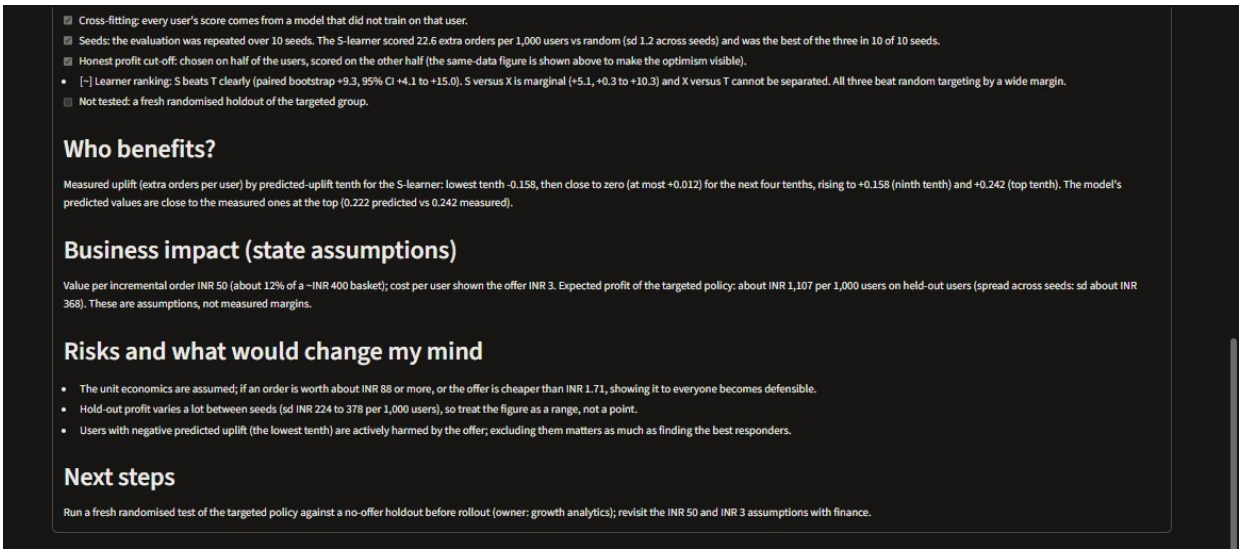
Screenshot 42. Risks and what would change my mind, and the next steps.



Screenshot 43. The targeting memo for exp_008: 'Do not send the offer to everyone. Send it to roughly the top 24% ranked by the S-learner.'



Screenshot 44. The targeting memo's result table, trust checks (cross-fitting, 10 seeds, honest hold-out) and the break-even arithmetic.

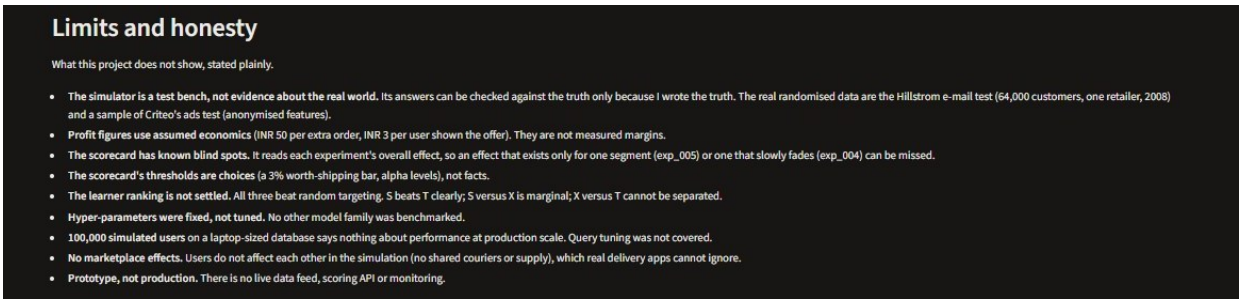


Screenshot 45. The targeting memo's risks and next steps: a fresh randomised test of the targeted policy before rollout.

Memo section	Why it is there
Decision needed / Recommendation	Leads with the answer and the main risk, in two sentences.
What we tested	The change, the users, the window, the primary and guardrail metrics.
Result	A small table; also a reminder to plan on the low end of the interval (about +8% for exp_001), because a single experiment tends to overstate its winner.
Can we trust it? (checks run)	A ticked list: sample-ratio, covariate balance (marked not run), power, no early stopping, Holm across experiments, novelty, data quality, overlap.
Who benefits?	exp_001: 'not analysed: segments were not pre-registered'. exp_008: the decile story.
Business impact (state assumptions)	Money only with its assumptions stated (INR 50 per order, INR 3 per user).
Risks and what would change my mind	Concrete triggers, for example a significant interaction with exp_004 or a fade in weeks 5 to 8.

Memo section	Why it is there
Next steps	An owner and a date: roll out to 95% with a 5% holdback and re-read at 2 weeks; test the targeted policy against a no-offer holdout.

11.16 Limits and honesty



Screenshot 46. The Limits and honesty page: what the project does not show, stated plainly.

- The simulator is a test bench, not evidence about the real world; the real data are Hillstrom (64,000 customers, one retailer, 2008) and a Criteo sample (anonymised features).
- Profit figures use assumed economics (INR 50 per extra order, INR 3 per user), not measured margins.
- The scorecard has known blind spots: an effect that exists only for one segment (exp_005) or that slowly fades (exp_004) can be missed.
- Its thresholds (3% bar, alpha levels) are choices, not facts.
- The learner ranking is not settled: all three beat random; S beats T clearly; S vs X is marginal; X vs T cannot be separated.
- Hyper-parameters were fixed, not tuned; no other model family was benchmarked.
- 100,000 simulated users on a laptop-sized database says nothing about production scale; query tuning was not covered.
- No marketplace effects: users do not affect each other in the simulation.
- Prototype, not production: no live data feed, scoring API or monitoring (the REST API in v2 exposes the analysis but not a persisted uplift model).

> **Interview tip**
> A reviewer's trust goes *up* when a project lists its own weaknesses. This page is deliberately in the menu, not hidden in a README.

PART D

What happens when you change a parameter

The seven levers behind every result, then real what-if experiments with numbers computed by the platform's own engine.

12. The seven levers behind every result

Every control in the app is a handle on one of seven things. Learn these and any page becomes predictable.

Lever	Where you see it	More of it means...	The catch
1. Sample size (users per arm)	Plan page; the Detectable lift on Analyze	Narrower intervals, smaller detectable lift, higher power.	Costs days. Needed users grow with the *square* of how small a lift you want to see.
2. Effect size (the lift you care about)	Plan: smallest relative lift	Easier to detect; far fewer users.	Small effects (+1.5%) can need hundreds of thousands of users.
3. Noise (variance of the metric)	Baseline rate or standard deviation; CUPED correlation	Wider intervals, more users needed.	Reduce it with pre-experiment covariates (CUPED), not with wishful thinking.
4. Significance level (alpha)	Analyze, Plan, Peeking sliders	Lower alpha means fewer false wins but needs more users.	Raising alpha buys 'significance' by accepting more false wins.
5. Power	Plan slider	Higher power: fewer missed real effects, more users.	80% is the convention, not a law.
6. Number of looks and arms	Peeking slider; Plan arms slider	More looks inflate false wins; more arms need a stricter alpha per comparison and more users.	Use sequential boundaries or fewer arms.
7. Economics (value and cost)	Analyze and Targeting value/cost; Who gets the offer sliders	Higher value or lower cost moves a decision toward SHIP and a wider reach.	A significant lift can still lose money; assumptions must be stated.

Key idea

One equation ties levers 1 to 5 together: n per arm = 2 * ((z for alpha + z for power) * noise / (lift * baseline))^2. Change any one of them and the others must move to keep it true.

12.1 Direction of every control (quick reference)

Page	Control	Raise it and...	Lower it and...
Analyze	Significance level	easier to call a win; more false wins	harder to call a win; fewer false wins
Analyze	Smallest lift worth shipping	more 'INCONCLUSIVE' (a test must detect a bigger bar); more notes that the interval dips below the bar	fewer inconclusives; a 'NO EFFECT FOUND' becomes possible
Analyze	Value per unit	profit per user rises; ITERATE can turn into SHIP	profit falls; SHIP can turn into ITERATE
Analyze	Cost per treated user	profit falls; SHIP can turn into ITERATE	profit rises
Analyze	Adjustment columns (add)	interval can narrow (if predictive)	interval stays wide
Plan	Smallest lift	far fewer users and days	far more users and days
Plan	Baseline rate	(for a fixed relative lift) fewer users	many more users for rare events
Plan	Power / (stricter) alpha	more users	fewer users
Plan	CUPED correlation	fewer users (times 1 - rho squared)	no saving at 0
Plan	Arms	more total users and days; stricter alpha per comparison	fewer
Plan	Users per day	fewer days	more days
Peeking	Number of looks	naive false wins climb (5% at 1 look, about 19% at 10, about 28% at 30)	approach 5%
Targeting	Value per unit	treat more users; profit up	treat fewer
Targeting	Cost per user	treat fewer users; eventually nobody	treat more; eventually everyone
Who gets the offer?	Profit per extra order	past about INR 88, sending to everyone is defensible	targeting matters more
Who gets the offer?	Cost of showing the offer	best reach shrinks	best reach grows; under about INR 1.71, everyone

13. What-if experiments (computed numbers)

Every table in this chapter was produced by running the platform's own functions for this guide. You can reproduce any row by setting the same values on the live pages.

13.1 Plan page: what each lever costs in users and days

Baseline conversion 5%, a 10% relative lift (5.0% to 5.5%), 2 arms, alpha 0.05, power 0.80, no covariate, 2,000 users per day, unless stated.

Smallest lift to detect	Users per arm	Days (at 2,000 per day)
5%	122,121	123
10%	31,231	32
20%	8,155	9
50%	1,468	2

A 5% lift needs about four times the users of a 10% lift, and a 50% lift needs only about 1,500 per arm.

Setting	Value	Users per arm
Power	70%	24,559
Power	80%	31,231
Power	90%	41,810
Significance level	0.01	46,471
Significance level	0.05	31,231
Significance level	0.1	24,601
CUPED correlation	0.0	31,231
CUPED correlation	0.3	28,420
CUPED correlation	0.6	19,988
Baseline rate	1%	163,092
Baseline rate	5%	31,231
Baseline rate	20%	6,507

Number of arms	Users per arm	Total users	Days
2	31,231	62,462	32
3	37,821	113,463	57
5	44,371	221,855	111

With 3 arms alpha is split across 2 comparisons with control, and with 5 arms across 4, so users per arm rise as well as the total. **What can I detect in a fixed time?** At 5% baseline and 2,000 users a day: after 7 days (7,000 per arm) the smallest reliably detectable lift is about 20.6%; after 14 days 14.6%; after 28 days 10.3%.

In plain words
The two screenshots in section 11.3 are the live version of this: defaults give 31,231 per arm; alpha 0.08, lift 55%, 5 arms and correlation 0.55 give 1,102 per arm.

13.2 Peeking: false wins by the number of looks

Looks	1	2	3	5	7	10	14	20	30
False wins (no real effect)	5.0%	8.3%	10.6%	14.1%	16.8%	19.4%	22.0%	24.0%	28.0%

Simulated on 40,000 A/A tests. At 10 looks the other rules give (from the page): Bonferroni about 2.7%, O'Brien-Fleming about 5.4%.

13.3 Analyze page: economics decide the verdict (Hillstrom, conversion)

Same data, same statistics. Only the money assumptions change. Net value is USD per user if the e-mail goes to everyone; the decision is what the platform returns.

Raising the cost of an e-mail (value USD 30 per conversion):

Cost per e-mail	Men's e-mail: net value	Men's decision	Women's e-mail: net value	Women's decision
USD 0.00	+0.204	SHIP	+0.093	SHIP
USD 0.05	+0.154	SHIP	+0.043	SHIP
USD 0.15	+0.054	SHIP	-0.057	ITERATE
USD 0.30	-0.096	ITERATE	-0.207	ITERATE
USD 0.50	-0.296	ITERATE	-0.407	ITERATE

Break-even is where net value reaches zero: about USD 0.20 per e-mail for Men's and about USD 0.09 for Women's (the point estimates). A decision flips to ITERATE only when the whole interval is below zero, which is why Men's at USD 0.30 and Women's at USD 0.15 flip but a net value close to zero does not always.

Changing the value of a conversion (cost USD 0.15 per e-mail):

Value per conversion	Men's net value	Men's decision	Women's net value	Women's decision
USD 5	-0.116	ITERATE	-0.134	ITERATE
USD 10	-0.082	ITERATE	-0.119	ITERATE
USD 20	-0.014	SHIP	-0.088	ITERATE
USD 30	+0.054	SHIP	-0.057	ITERATE
USD 60	+0.258	SHIP	+0.037	SHIP

Subtle but important
Look at the row 'USD 20': Men's net value is slightly negative (-0.014) yet the decision is still SHIP. The interval around it includes zero, so the e-mail is **not proven** unprofitable and shipping is not blocked. Only a *provably* negative interval blocks it. That is a deliberate design choice, written in the reasons.

13.4 Analyze page: the other controls

Experiment	Setting changed	Result
Hillstrom, conversion	Significance level 0.001, 0.01, 0.05, 0.10	No change: Holm-adjusted p is 3e-13 (Men's) and 1.6e-04 (Women's), far below every level; both stay significant.
Hillstrom, conversion	Planned split changed from equal to 50/25/25 (the true split is about 1/3 each)	Sample-ratio p goes from 0.904 to essentially 0, and both arms become **DO NOT TRUST**.
Hillstrom, spend	Adjust for recency, history, mens, womens, newbie	Interval width ratio 0.997 (Men's) and 1.000 (Women's): about 0.3% narrower and unchanged: past behaviour explains almost none of the spend.
Simulated exp_001	No adjustment vs adjusting for orders_pre	Half-width 0.0940 to 0.0897: about 4.5% narrower; lift +6.9% naive, +9.2% adjusted (the SQL page reports both).

The 'smallest lift worth shipping' bar changes the label, not the statistics. On the 12,000-user demo warehouse, exp_003 (a true null) gives:

Smallest lift worth shipping	Detectable lift (MDE)	Decision for exp_003 (true effect 0)
3%	12.7%	INCONCLUSIVE (underpowered)
5%	12.7%	INCONCLUSIVE (underpowered)
10%	12.7%	INCONCLUSIVE (underpowered)
15%	12.7%	NO EFFECT FOUND
20%	12.7%	NO EFFECT FOUND

When the bar is below what the sample can detect, the honest answer is **INCONCLUSIVE (underpowered)**. Once you say you only care about large lifts, the same sample is able to detect them, and the label becomes **NO EFFECT FOUND** (no effect of that size).

13.5 The effect of sample size: demo warehouse vs full warehouse

The Analyze page's simulator uses 12,000 users so it loads in seconds. The Scorecard page uses 100,000. Smaller samples are less powerful, so some verdicts differ. Below, the demo warehouse (no economics switched on, adjusted for the matching pre-period column):

Experiment	Users (treat / control)	SRM p	Lift	Interval narrower by	Demo decision	Scorecard (100,000 users)
exp_001	5,426 / 5,360	0.525	+9.2%	5%	SHIP	SHIP
exp_002	5,741 / 5,679	0.562	-0.9%	8%	INCONCLUSIVE	INCONCLUSIVE
exp_003	5,743 / 5,677	0.537	+3.6%	7%	INCONCLUSIVE	INCONCLUSIVE
exp_004	5,340 / 5,446	0.307	+10.0%	7%	SHIP	SHIP
exp_005	6,104 / 5,896	0.0576	-1.9%	9%	INCONCLUSIVE	INCONCLUSIVE
exp_006	4,921 / 5,619	1.05e-11	+10.1%	10%	DO NOT TRUST	DO NOT TRUST
exp_007	5,642 / 5,778	0.203	+7.3%	17%	INCONCLUSIVE	SHIP
exp_008	5,941 / 6,059	0.281	+19.7%	9%	SHIP	ITERATE (economics)
exp_009	5,975 / 6,025	0.648	-5.2%	8%	INCONCLUSIVE	INCONCLUSIVE

- **exp_006** is DO NOT TRUST at both sizes: a broken assignment is visible even in a small sample.

- **exp_007** (a true +6% on basket value) is INCONCLUSIVE with 12,000 users but SHIP with 100,000: a heavy-tailed metric needs a large sample. This is the whole point of the Plan page.
- **exp_008** reads SHIP here only because no costs were entered. The page's default cost of INR 3 per user for exp_008 would turn it into ITERATE, as on the Scorecard page.
- **exp_005** has a borderline sample-ratio p (0.058): the page would add a note, not block.

13.6 Live targeting: how cost moves the best reach (Hillstrom, Men's e-mail, spend, T-learner)

Value per dollar of spend = 0.30. Profit is per 1,000 users on the held-out half (the honest figure).

Cost per e-mail	Best reach (in-sample)	Targeted profit per 1,000	Treat everyone per 1,000	Reading
USD 0.00	99%	231.1	285.0	everyone: treat all
USD 0.15	86%	104.2	135.0	treating everyone beats targeting (as on the page)
USD 0.30	7%	-15.2	-15.0	break-even: roughly nobody
USD 0.60	1%	-4.4	-315.0	treat nobody
USD 1.00	0%	0.0	-715.0	treat nobody

As the cost rises the best reach falls from 99% to about 0%, and treating everyone goes from clearly positive to clearly negative. The targeted policy never meaningfully beats the simple options on this data (see the Qini table: every model's interval versus random includes zero), so the platform's advice follows the economics: e-mail everyone if it pays, nobody if it does not.

Model	Qini per 1,000 users	Difference from random (95% interval)	Clearly beats random?
random	+70.6	(baseline)	n/a
T-learner	+24.0	-132 to +67	No
S-learner	+14.8	-147 to +60	No
X-learner	-3.4	-193 to +23	No

Why intervals matter
Even the 'random' row scores far from zero (about +70): with heavy-tailed spend, a score on 40,000 users is very noisy. This is exactly why the page shows intervals and a verdict instead of a ranking.

PART E

Explaining and defending the project

How to describe it in 60 seconds and in 5 minutes, resume bullets you can honestly write, and the questions an interviewer is likely to ask.

14. Explaining the project

14.1 The 60-second version

> **Say this**
> "I built an experimentation platform that turns A/B-test data into a trust-checked decision: ship, iterate, inconclusive, or do not trust. It checks that the split was fair, reduces noise with CUPED, corrects for multiple tests, checks power, guardrails and unit economics, and says why. The SQL layer is DuckDB with window functions, cohorts and a whole experiment analysis in SQL that matches my Python to nine digits. I validated the statistics by re-simulating the experiments 100 times with known truths, and ran the same engine on two real randomised datasets, Hillstrom and Criteo. On Criteo the naive effect was inflated by about 28% because of a small randomisation imbalance; on Hillstrom no uplift model beat random, and the platform says so instead of forcing a story. It is deployed as a FastAPI service in Docker and a live Streamlit app with 167 tests."

14.2 The five-minute walk-through (a suggested order)

1. **The problem** (30 s): reading an A/B test correctly is harder than running it; list three ways it goes wrong (broken split, underpowered, peeking).
2. **The data** (30 s): a simulator with a known truth as a test bench, plus two real randomised datasets.
3. **The SQL** (60 s): staging with ROW_NUMBER, per-user marts with a 28-day pre-period, reports with window functions, and the analysis in SQL. Mention the two bugs you found by reading (RFM ties, retention censoring).
4. **The statistics** (90 s): SRM first, then CUPED, Holm, power, peeking, guardrails and economics, ending in a decision rule. Mention the two honest misses.
5. **The uplift** (60 s): why uplift is different from prediction, S/T/X learners, cross-fitting, Qini, honest hold-out profit, and the 'no model beats random' result on Hillstrom.
6. **The engineering** (30 s): one engine used by the API and the app, tests, CI, Docker, the SQL security guard.

> **Interview tip**
> If you have only two minutes, spend them on **SRM, CUPED, the decision rule and what the platform got wrong**. Interviewers remember honesty about limits.

15. Resume bullets you can honestly write

Every statement below is supported by something in the repository (a test, a result or the live app). Pick three or four for a one-page resume. Replace nothing with a bigger-sounding number.

#	Bullet	Evidence
1	Built an A/B-testing decision engine (sample-ratio mismatch, CUPED, Holm correction, power/MDE, peeking boundaries, guardrails, unit economics) that returns SHIP / ITERATE / INCONCLUSIVE / DO NOT TRUST with reasons; validated on 100 re-simulated worlds (unbiased estimates, 94% to 100% interval coverage, planted assignment bug flagged).	Platform validation page; tests
2	Designed a DuckDB SQL layer: de-duplicated and flagged 5.3M events, 11 analytic reports (window functions, gaps and islands, cohorts, RFM, sessionisation, QUALIFY) each unit-tested against hand-computed answers, plus a full CUPED experiment analysis in SQL that matches Python to 9 digits.	SQL workbench; tests/sql
3	Ran the engine on real randomised data: found a Criteo ad-test effect inflated 28% by a small randomisation imbalance using doubly-robust (AIPW) estimation; on the Hillstrom e-mail test showed that no uplift model beat random targeting rather than over-claiming.	Real data pages; Live targeting
4	Built S/T/X uplift models with cross-fitting and honest hold-out profit evaluation; on simulated data a targeted policy earned INR 1,107 per 1,000 users while sending to everyone lost INR 1.29 per user (assumed economics).	Who gets the offer? page; targeting memo
5	Shipped the platform as a FastAPI service (Docker, GitHub Actions on Python 3.11/3.12) and a live 16-page Streamlit app, with 167 automated tests and a read-only SQL workbench protected by a text guard and a locked DuckDB connection.	Live URL; CI; tests

Avoid

Do **not** write: 'improved conversion by X%' (no real business ran this), 'production system' (it is a deployed prototype without monitoring), or any simulated number without the word simulated. Put the live URL next to the project name, and open it a few minutes before an interview because free apps sleep.

15.1 Skills lines this project supports

SQL: DuckDB, window functions, CTEs, QUALIFY, cohort analysis, data cleaning and data-quality monitoring. **Experimentation / causal inference:** A/B test design and analysis, SRM, CUPED, multiple-testing correction, power analysis, sequential testing, uplift modelling, doubly-robust estimation. **Engineering:** FastAPI, Docker, pytest, GitHub Actions, Streamlit.

16. Interview questions and model answers

Q1. What is an A/B test and what can go wrong?

Random assignment to control and treatment lets you attribute a difference to the change. It goes wrong when the split is not really random (sample-ratio mismatch), when the sample is too small to see the effect, when you stop early on a lucky result (peeking), when you run many tests and take the best, or when a real win costs more than it earns. My platform has a check for each.

Q2. What is sample-ratio mismatch and why check it first?

If a 50/50 plan gives 46/54, a chi-square goodness-of-fit test flags it. It means assignment or logging is broken, so the groups are not comparable and any lift is meaningless however significant it looks. I stop the analysis at p below 0.001 and add a note below 0.10. In my simulator exp_006 had planted dropped users and was flagged DO NOT TRUST.

Q3. Explain CUPED. Why is it unbiased?

It subtracts the part of each user's outcome that is predicted by their behaviour before the test: y minus theta times (x minus mean x), with theta equal to cov(y,x)/var(x). Because x was measured before assignment it has the same expected value in both arms, so the average effect is unchanged, while the variance shrinks by a factor of 1 minus rho squared. I extended it to several covariates by regression. On Hillstrom spend it barely helped, because spend is almost always zero, and the platform reports that.

Q4. Holm vs Bonferroni vs Benjamini-Hochberg?

Bonferroni uses alpha/m for every test. Holm is a step-down version that is uniformly at least as powerful and controls the same family-wise error. BH controls the false-discovery rate (the share of wins that are false), which is less strict. I use Holm because I want to avoid *any* false ship decision among the experiments I read together.

Q5. How do you size an experiment?

From the baseline, the smallest lift worth detecting, alpha and power: for a rate, n per arm is (z alpha/2 + z power) squared times the sum of the two Bernoulli variances, divided by the squared difference. Halving the lift roughly quadruples the sample. A covariate with correlation rho cuts it by 1 minus rho squared. I round the duration up to whole weeks. My Plan page does this live.

Q6. Why does peeking inflate false positives?

Each look is another chance for the test statistic to cross the line by luck. With ten looks the false-positive rate is about 19%, with thirty about 28%. Fixes are to fix the sample size in advance, use a stricter boundary at each look, or use a sequential boundary such as O'Brien-Fleming, which is strict early and relaxed late. I calibrate it by Monte Carlo.

Q7. What does INCONCLUSIVE mean and how is it different from NO EFFECT?

Not significant does not mean no effect. If the sample could not have detected a lift as small as the one worth shipping (the MDE is bigger than the bar), the answer is unknown, so INCONCLUSIVE. Only when the test could have detected it do I say NO EFFECT FOUND.

Q8. Why uplift modelling rather than predicting who will buy?

A purchase propensity model finds people who will buy anyway. Uplift estimates the *causal* effect of the offer per user, so it finds persuadables and excludes sure things, lost causes and sleeping dogs (who are driven away). In my data the lowest-scored tenth ordered about 0.16 fewer orders when shown the offer.

Q9. How do you evaluate an uplift model when you never see an individual's effect?

With a randomised experiment I build a per-user pseudo-effect y times (t/p minus (1-t)/(1-p)), whose average is the treatment effect. Sorting users by score and cumulating it gives a Qini-style gain curve; the area above random targeting, in extra outcome per 1,000 users, is the metric, with bootstrap intervals. I also cross-fit so no user is scored by a model that saw them, and I pick the profit cut-off on one half and score it on the other.

Q10. Explain doubly-robust (AIPW) estimation.

It combines an outcome model and a propensity model: the estimate is correct if either is correct. On Criteo, a small covariate imbalance made the naive effect inflated; the AIPW effect was 28% lower for visits. I analyse by assignment, not exposure, to avoid post-treatment bias.

Q11. Your main data is simulated. Why should I trust it?

You should not trust the simulated numbers as facts about the world, and the platform says so. The simulator is a test bench: because I wrote the truth I can grade the platform, and I did, over 100 fresh worlds. Then I ran the same engine on two real randomised datasets, where the truth is unknown but the same checks apply.

Q12. Where does your platform fail?

Two documented misses. exp_005's +18% for new users only is hidden in the overall average, and exp_004's novelty fade is too noisy to see at weekly resolution. Both are heterogeneity, which the scorecard does not examine. Next steps would be a pre-registered segment gate with correction, and a sequential or Bayesian view.

Q13. How do you handle heavy-tailed revenue?

I compare four estimators: naive, CUPED, winsorised at a pooled cap (which changes the quantity measured, so I show it next to the uncapped ones), and a bootstrap. I also split the lift into purchase rate and revenue per buyer, noting that buyers are a post-treatment group so the split is descriptive.

Q14. Walk me through a SQL window-function problem you solved.

Streaks: for consecutive days, day number minus ROW_NUMBER is constant, so it labels each island. Sessionisation: LAG gives the previous event time, a gap over 30 minutes starts a session, a running SUM numbers them. Top-N per group: DENSE_RANK with QUALIFY, and I note that ties can return more than N rows. Retention: I build a grid of observable (cohort, week) pairs so unfinished weeks do not drag retention down.

Q15. What SQL bugs did you find in your own work?

RFM scoring with NTILE split tied users (most buyers have one order) into different scores by user_id; I switched to RANK so ties share a score and added a test with eight tied users. Retention pooled cohorts that could not yet have been observed at week 8; I added a grid of observable weeks with explicit zeros and two tests.

Q16. How would you productionise this?

Persist the final uplift model with a version and a model card behind a scoring endpoint, schedule the SQL pipeline, add automatic SRM and data-quality alerts, log every decision with its inputs, monitor model drift, and add a pre-registration step for hypotheses and segments. The engine is already pure functions behind an API, with tests and CI.

Q17. How is the public SQL box kept safe?

Three layers: a text guard that allows one read-only statement and rejects write and file-reading keywords, a DuckDB connection with external access disabled and configuration locked so it cannot be re-enabled, and limits (1 GB memory, 1,000 rows) on a database that holds only simulated data. A test proves the locked connection cannot read a file.

Q18. Statistical versus practical significance?

A tiny lift can be significant with enough users and still not be worth shipping. I let the user set the smallest lift worth shipping and the unit economics; a significant lift that is provably unprofitable becomes ITERATE. In my win-back offer example a real +7.1% order lift lost INR 1.29 per user.

Q19. What would you do next?

A pre-registered segment analysis gate, a Bayesian probability-to-beat-control view, CUPAC (a model-based covariate), a real transactional dataset such as Olist for the SQL layer, a persisted uplift model behind a scoring endpoint, and monitoring.

PART F

Appendices

Glossary, formulas, commands, troubleshooting and what to do next.

Appendix A. Glossary

Term	Meaning
A/A test	An experiment where both groups get the same experience; any 'winner' is a false positive. Used to check a testing setup.
A/B test	A randomised comparison of a control and a treatment.
AIPW (doubly robust)	An effect estimator that combines an outcome model and a propensity model; correct if either is right.
Alpha	The tolerated false-positive rate (usually 0.05).
Arm	One group of an experiment (control or a treatment).
Assignment	The act (and table) of placing a user in an arm.
Average treatment effect (ATE)	The average difference the treatment causes.
Bonferroni	A multiple-testing correction that divides alpha by the number of tests.
Bootstrap	Resampling the data with replacement many times to see how much an estimate varies.
Confidence interval	A range that would contain the true value in about 95% of repeated experiments.
Control	The group that keeps the old experience.
Covariate	A pre-experiment measurement used to reduce noise or to describe users.
Cross-fitting	Scoring each user with a model trained on other users, to avoid overfitting.
CUPED	Controlled-experiment Using Pre-Experiment Data: variance reduction by subtracting what past behaviour predicts.
Decile	One tenth of the users after ranking them by a score.
Delta method	A way to get the standard error of a ratio (such as relative lift).
DuckDB	An analytical SQL database that runs inside a program.
False discovery rate (FDR)	The expected share of declared wins that are false.
Family-wise error rate (FWER)	The chance of at least one false win among a set of tests.
Gradient boosting	A model made of many small decision trees added together.
Guardrail	A second metric that must not get worse.
Heterogeneous effect	A treatment effect that differs between users.
Holm	A step-down multiple-testing correction, never less powerful than Bonferroni.
Intent-to-treat	Analysing users by the arm they were assigned to, not by what they did afterwards.
IPW	Inverse-propensity weighting: weighting outcomes by the inverse chance of the assignment received.
Lift	The relative change of the treatment mean over the control mean.
MDE	Minimum detectable effect: the smallest relative lift a sample can reliably detect.
Mart	A table prepared for analysis, here one row per user.
Novelty effect	A lift that fades as the newness wears off.
O'Brien-Fleming boundary	A sequential-testing threshold that is strict early and relaxed late.
p-value	How surprising the data would be if there were no effect.
Peeking	Checking results repeatedly and stopping at the first significant one.

Term	Meaning
Pre-period	The days before an experiment starts (28 here).
Power	The chance of detecting an effect of a given size.
Propensity	The probability of receiving the treatment.
Qini curve / area	A gain curve for uplift models; the area above random targeting.
Randomisation	Assigning users to arms by chance.
RFM	Recency, frequency, monetary value: a customer segmentation.
S/T/X learner	Three recipes for uplift models using ordinary prediction models.
SMD	Standardised mean difference: how different two groups are on a feature, in standard deviations.
SRM	Sample-ratio mismatch: arm sizes that differ from the plan by more than chance.
Staging	A cleaned layer of data that all later queries read.
Treatment	The group that gets the new experience.
Uplift	The causal effect of a treatment on one user (or group).
Welch t-test	A two-group comparison that does not assume equal variances.
Winsorising	Capping extreme values at a chosen limit.

Appendix B. Formula cheat-sheet

Quantity	Formula	Notes
Relative lift	`lift = mean_t / mean_c - 1`	Treatment mean over control mean, minus 1
Delta-method SE	`se = sqrt( (se_t / mean_c)^2 + (mean_t * se_c / mean_c^2)^2 )`	95% interval: lift +/- 1.96 se
Welch t	`t = (mean_t - mean_c) / sqrt(var_t/n_t + var_c/n_c)`	No equal-variance assumption
Sample-ratio test	`X2 = sum (observed - expected)^2 / expected`	Chi-square, arms minus 1 degrees of freedom
CUPED	`theta = cov(y,x)/var(x); y_adj = y - theta*(x - mean x)`	Variance times (1 - rho^2); mean unchanged
Family-wise error	`1 - (1 - alpha)^m`	5.0%, 22.6%, 40.1%, 64.2% for m = 1, 5, 10, 20
Holm	`reject p(i) if p(i) <= alpha / (m - i + 1), stop at first failure`	p sorted ascending
Sample size (rate)	`n = (z_a + z_pw)^2 * (p0(1-p0) + p1(1-p1)) * (1-rho^2) / (p1-p0)^2`	z_a = z for 1 - alpha/2; z_pw = z for power
Sample size (mean)	`n = 2 * ((z_a + z_pw) * sd * sqrt(1-rho^2) / (lift*mean))^2`	Per arm
MDE	`(z_a + z_pw) * sqrt(2) * sd * sqrt(1-rho^2) / (sqrt(n) * mean)`	Smallest detectable relative lift
Net value per user	`value * (mean_t - mean_c) - cost_per_treated`	With a Welch interval
Interaction (2x2)	`(effect of A when B on) - (effect of A when B off)`	Zero means the effects add
Uplift pseudo-effect	`tau_i = y_i * ( t_i/p - (1 - t_i)/(1 - p) )`	Average equals the average treatment effect
Profit curve	`profit(k) = value * sum(tau, top k users) - cost * k`	Peak = best reach
AIPW	`tau_i = mu1 - mu0 + t(y - mu1)/e - (1-t)(y - mu0)/(1 - e)`	mu = outcome models, e = propensity

Appendix C. Commands

Run the live app and API on your computer

```
cd growth-experimentation-platform
.venv\Scripts\activate                      # Windows PowerShell
pip install -r requirements.txt -r app/requirements.txt -r api/requirements.txt
streamlit run app/streamlit_app.py           # http://localhost:8501
uvicorn api.main:app --reload                # http://127.0.0.1:8000/docs
python -m pytest                             # 167 tests
```

Rebuild everything from scratch

```
python -m src.simulation.simulate --n-users 100000      # about 83 MB of Parquet in data/warehouse/
python -m src.warehouse.build_warehouse                 # DuckDB database + staging view
python -m src.warehouse.run_sql                         # all 11 analysis queries -> artifacts/sql/
python -m src.experiments.first_look
python -m src.experiments.novelty --exp exp_004
python -m src.experiments.interaction
python -m src.experiments.revenue --exp exp_007
python -m src.experiments.scorecard
python -m src.experiments.validation --seeds 100
python -m src.experiments.studies all
python -m src.targeting.uplift --exp exp_008
python -m src.targeting.multiseed --exp exp_008 --seeds 10
python -m src.criteo.prepare                            # Criteo download + 1M sample (run on Colab)
python -m src.engine.make_samples                       # small sample files for the live app
```

Docker

```
docker build -t gep-api .
docker run -p 8000:8000 gep-api
curl http://127.0.0.1:8000/health
```

Update the live app after a change

```
git add -A
git commit -m "describe the change"
git push                                        # Streamlit Cloud redeploys by itself in about a
    minute
```

Appendix D. Troubleshooting

Symptom	Likely cause and fix
The live app says 'this app has gone to sleep'	Free apps sleep when idle. Click the wake-up button and wait about a minute. Open it before an interview.
A page says a file 'was not found in artifacts/'	The file is missing from the repository's artifacts folder. Check it on GitHub and commit it.
The Criteo option is missing on Analyze / Live targeting	The 100,000-row sample is not in sample_data/. Run python -m src.criteo.prepare then python -m src.engine.make_samples, commit the parquet file. The older 'Real data (Criteo)' page does not need it.
The Hillstrom option is missing	sample_data/hillstrom.csv.gz is not in the repository (it is not git-ignored). Add and commit it.
Tests fail on Windows only	Paste the output. Typical causes are path separators and line endings; they are quick to fix.
git push is rejected (non-fast-forward)	The GitHub copy has newer commits. Do not force-push; run git pull, resolve any conflict, then push.
The Docker build fails in CI	Open the Actions log; usually a missing file in the COPY lines or a package version.
The page layout looks cramped or the menu too wide	The menu width is set in the style block of app/streamlit_app.py (the clamp(15rem, 18vw, 18.5rem) values).

Appendix E. Honest limits and what to do next

Known limits (also on the Limits page)

- The simulator is a test bench, not the real world; Hillstrom is 2008 and one retailer; Criteo features are anonymised.
- Economics are assumptions. The scorecard misses effects that exist only in a segment or that fade slowly.
- There is no persisted uplift model, no monitoring, and no marketplace effects; query tuning at production scale was not covered.

A sensible roadmap

Next step	What it adds	Effort
Persist a final uplift model + a scoring endpoint + a model card	Closes the 'prototype, not production' gap; matches the CLV project's deployment story	Medium
A pre-registered segment (heterogeneity) gate with correction	Fixes the two documented misses honestly	Medium
A Bayesian view (probability to beat control, expected loss)	A common industry alternative to p-values	Small
Olist real transactional dataset for the SQL layer (star schema, order items, cohorts)	SQL on real data; query tuning and EXPLAIN at scale	Medium
CUPAC (a model-based covariate) and a sequential test mode	More noise reduction; safe early stopping	Medium
Monitoring and scheduled runs	Alerts for SRM and data quality	Medium

Final thought
The project already does what most portfolio projects skip: it checks itself, says where it is wrong, and is live. Keep the live link working and be ready to demonstrate the Analyze page by changing the cost and watching the decision flip.